

Database Assistant

A Project Report
Presented to
The Faculty of the College of
Engineering
San Jose State University
In Partial Fulfillment
Of the Requirements for the Degree
Master of Science in Software Engineering

By
Siri Batchu, Shruti Goyal, Rutuja Bhaskarrao Patil, Aditya Rajpurohit
2026

APPROVED

Prof. Vijay Eranti, Project Advisor

Prof. Dan Harkey, Director, MS Software Engineering

Dr. Jorjeta Jetcheva, Department Chair

ABSTRACT

Database Assistant

By Siri Batchu, Shruti Goyal, Rutuja Bhaskarrao Patil, Aditya Rajpurohit

Modern enterprises manage increasingly complex database systems with hundreds of tables, intricate relationships, and business-critical data spanning multiple domains. While these databases contain valuable insights, accessing them remains a significant challenge for business users who lack SQL expertise. Analysts spend considerable time translating business questions into complex queries, creating bottlenecks that slow decision-making and limit organizational data accessibility.

Existing natural language to SQL tools falls short of enterprise needs. Most solutions generate syntactically correct but semantically meaningless results, struggle with table relationships, lack self-correction mechanisms, and focus solely on query generation leaving users to manually execute queries, interpret results, and build visualizations separately.

Database Assistant addresses these limitations through an AI-powered platform built on Google Gemini 2.5 Flash that enables users to query databases using plain English with no SQL expertise required. Database Assistant analyzes database schemas to build structured context capturing table relationships and column semantics, then generates accurate validated queries through schema-aware prompting. A built-in self-correction mechanism automatically detects execution errors and regenerates queries without user intervention. The result is a complete analytical workflow from natural language input to verified query results, automatic visualizations, and AI-powered insights within a single secure web application.

Acknowledgments

The authors are deeply indebted to Professor Vijay Eranti for his invaluable comments and assistance in the preparation of this study.

Table of Contents

Chapter 1. Project Overview	1
1.1 Introduction	1
1.2 Proposed Areas of Study and Academic Contribution.....	2
1.2.1 Natural Language to SQL Translation	2
1.2.2 Adaptive Schema Understanding and Metadata Generation.....	2
1.2.3 Verification-Centered NL-to-SQL Workflow with Self-Correction.....	3
1.2.4 Simulation of AlloyDB AI Functions (ai.generate, ai.if, ai.rank).....	3
1.2.5 Template-Based Generative UI System	4
1.2.6 End-to-End Agentic Pipeline for Transparent Analytics	4
1.2.7 Multi-Level Agent Architecture (L1 / L2 / L3).....	4
1.2.8 KDD Cup 2026 Benchmark Evaluation.....	5
1.2.9 Open Plugin Architecture and Claude Code Integration.....	5
1.3 Current State of the Art	6
1.3.1 Existing Natural Language to SQL Systems.....	6
1.3.2 Google AlloyDB AI and In-Database Intelligence	6
1.3.3 Generative UI Research (Google Paper, Appendix A.5)	7
1.3.4 Limitations of Existing Approaches.....	7
Chapter 2. Project Architecture	9
2.1 Introduction	9
2.1.1 Architectural Goals and Constraints.....	9
2.1.2 High-Level System Architecture Diagram.....	9
2.1.3 Technology Stack Overview	11
2.2 Architecture Subsystems	12
2.2.1 Frontend Subsystem (React 18 Single Page Application).....	12
2.2.2 Backend Subsystem (FastAPI REST API).....	13
2.2.3 AI Pipeline Subsystem (Google Gemini 2.0 Flash)	15
2.2.4 Database Subsystem (PostgreSQL + MongoDB)	15
2.2.5 Security Subsystem (JWT + Fernet Encryption + SQL Guard).....	16
2.2.6 Agent-Based Processing Pipeline (7 Specialized Agents)	17
2.2.7 Swarm Agent Subsystem (Level 3 Parallel Execution)	18
2.2.8 Generative UI Subsystem (Template Engine).....	20
2.2.8 Open Plugin Subsystem.....	20
Chapter 3. Technology Descriptions	22
3.1 Client Technologies.....	22
3.1.1 React 18.....	22
3.1.2 React Router v6	22
3.1.3 Axios	23
3.1.4 Recharts	23
3.1.5 Chart.js	23
3.1.6 Lucide React.....	23
3.1.7 AuthContext and JWT Token Management.....	24
3.1.7 Papa.parse and SheetJS	24

3.2 Middle-Tier Technologies	24
3.2.1 FastAPI.....	24
3.2.2 Google Gemini 2.0 Flash API (google-genai SDK).....	25
3.2.3 Psycpg2	26
3.2.4 mysql-connector-python.....	26
3.2.5 Motor Async MongoDB Driver	26
3.2.6 Pandas.....	27
3.2.7 Fernet Encryption	27
3.2.8 SQL Guard	27
3.2.9 Python-Jose and Bcrypt.....	27
3.2.10 asyncio (Swarm Parallelism).....	28
3.3 Data-Tier Technologies	28
3.3.1 PostgreSQL 15	28
3.3.2 MySQL.....	29
3.3.3 MongoDB.....	29
3.3.4 SQLite (In-Memory, via Pandas)	29
3.4 Infrastructure and Deployment Technologies	29
3.4.1 Google Cloud Run.....	28
3.4.2 Google Cloud SQL (PostgreSQL 15).....	29
3.4.3 Google Artifact Registry and Cloud Build.....	29
3.4.4 Docker and Docker Compose.....	29
3.4.5 Open Plugin Protocol and Claude Code Plugin	30
Chapter 4. Project Design	31
4.1 Client Design.....	31
4.1.1 Application Layout and Navigation Design.....	31
4.1.2 Login and Registration Page Design.....	31
4.1.3 Dashboard Page Design with Stats and Action Cards.....	33
4.1.4 Chat-Style Query Interface Design	33
4.1.5 ResultsPanel Five-Tab Design	35
4.1.6 AI Functions Panel Design (AlloyDB Simulation).....	36
4.1.7 Swarm Agents Panel Design	37
4.1.8 Benchmark Dashboard Design.....	39
4.1.9 Connections Manager Design	39
4.1.10 Agent Demos Page Design (L1/L2/L3).....	40
4.1.11 Claude Plugin Page Design	41
4.1.12 Claude Code Plugin (Terminal) Design	42
4.1.13 Datasets Page Design	43
4.2 Middle-Tier Design	44
4.2.1 REST API Route Architecture	44
4.2.2 Three-Level Agent Architecture	45
4.2.3 Swarm Agent Design.....	45
4.2.4 SQL Guard Safety Module Design	46
4.2.5 Schema-Aware Prompt Engineering Design.....	46
4.2.6 Open Plugin Protocol Design.....	47

4.2.7 Benchmark Agent Design	47
4.2.8 AlloyDB AI Function Simulation Design	48
4.2.9 4.2.9 Generative UI Template Selection Algorithm.....	48
4.3 Data-Tier Design	49
4.3.1 User Authentication and Profile Schema	49
4.3.2 Connection Credentials Schema (Encrypted).....	50
4.3.3 Internal Dataset Storage Schema.....	50
4.3.4 Query History Schema	51
Chapter 5. Project Implementation.....	52
5.1 Client Implementation	52
5.1.1 AuthContext: JWT Token Management and User State	52
5.1.2 Dashboard Page Implementation.....	53
5.1.3 PostgreSQL Chat Query Page Implementation.....	54
5.1.4 MySQL Query Page Implementation.....	55
5.1.5 MongoDB Multi-Collection Query Page Implementation.....	55
5.1.6 Datasets Upload and Query Page Implementation.....	56
5.1.7 Connections Manager Implementation	56
5.1.8 ResultsPanel Six-Tab Implementation	57
5.1.9 AiFunctionsPanel Implementation.....	57
5.1.10 Swarm Agents Page Implementation	58
5.1.11 Benchmark Dashboard Implementation.....	58
5.1.12 Agent Demos Page Implementation.....	59
5.1.13 Claude Plugin Page Implementation	59
5.2 Middle-Tier Implementation	60
5.2.1 Natural Language to SQL Pipeline	60
5.2.2 Multi-Table JOIN Query Generation	60
5.2.3 MongoDB NL-to-Aggregation Pipeline with \$lookup Joins	61
5.2.4 Auto-Table Selection for Uploaded Datasets	61
5.2.5 SQL Guard: Keyword Blocking and SELECT Enforcement.....	62
5.2.6 ai.generate: Row-Level Insight Generation with Gemini	62
5.2.7 ai.if: Natural Language Row Filtering with Gemini	63
5.2.8 ai.rank: Semantic Row Reordering with Gemini	63
5.2.9 Generative UI: Five-Template Engine with Data Profiler	63
5.3 Data-Tier Implementation	64
5.3.1 PostgreSQL Docker Setup and Schema Initialization.....	64
5.3.2 Fernet Credential Encryption and Storage	64
5.3.3 CSV/Excel Ingestion with Pandas Type Inference	65
5.3.4 Swarm Agent Implementation.....	65
5.3.5 MySQL Query Pipeline Implementation	66
5.3.6 Open Plugin Protocol Implementation	66
5.3.7 Benchmark Agent Implementation	66
Chapter 6. Testing and Verification.....	68
6.1 Testing Strategy and Methodology	68
6.2 Unit Testing	68

6.2.1 SQL Guard Unit Tests	68
6.2.2 Fernet Encryption Unit Tests	69
6.2.3 ReAct Loop Unit Tests.....	69
6.2.3 Template Selection Unit Tests	69
6.3 Integration Testing.....	69
6.3.1 Authentication Endpoint Testing.....	69
6.3.2 SQL Guard Integration Testing.....	70
6.3.3 Connections Manager Testing.....	70
6.3.4 Plugin and AI Function Endpoint Testing.....	70
6.4 End-to-End Testing	70
6.4.1 PostgreSQL NL-to-SQL Test Cases.....	70
6.4.2 MongoDB NL-to-Aggregation Test Cases.....	71
6.4.3 AlloyDB AI Function End-to-End Tests.....	72
6.4.4 Swarm Agent End-to-End Tests	72
6.4.5 Claude Plugin End-to-End Tests	72
6.5 Security Testing.....	72
6.5.1 SQL Injection Prevention Testing.....	72
6.5.2 Authentication and Authorization Testing	73
6.5.3 Fernet Encryption Verification.....	73
6.6 Test Results Summary	74
6.7 KDD Cup 2026 Benchmark Evaluation	75
6.7.1 Benchmark Results.....	75
6.7.2 Accuracy Improvement: 46% to 82%	75
Chapter 7. Performance and Benchmarks	76
7.1 Performance Evaluation Overview.....	76
7.2 Query Response Time Benchmarks	76
7.2.1 NL-to-SQL End-to-End Latency.....	76
7.2.2 Response Time by Database Type	77
7.2.3 Dataset Query Response Time	77
7.3 AI Function Performance	78
7.3.1 ai.generate Processing Time.....	78
7.3.2 ai.if Processing Time	78
7.3.3 ai.rank Processing Time	79
7.4 Generative UI Performance	79
7.4.1 Template Render Time vs LLM-Generated HTML.....	79
7.4.2 Template Selection Accuracy.....	80
7.5 KDD Cup Benchmark Performance	80
7.6 API Throughput and Scalability	81
7.6.1 Concurrent Request Handling	81
7.7 Performance Comparison with Related Systems	81
7.8 Performance Summary and Observations	82
Chapter 8. Deployment, Operations, Maintenance	83
8.1 Deployment Architecture	83
8.1.1 Frontend - Cloud Run (nginx Container)	84
8.1.2 Backend - Cloud Run (FastAPI Container).....	84

8.1.3 Cloud SQL – Database	85
8.1.4 Google Artifact Registry and Cloud Build.....	85
8.1.5 Google Artifact Registry and Cloud Build.....	85
8.2 Local Development Setup	86
8.2.1 Startup Procedure	86
8.2.2 Shutdown Procedure.....	87
8.3 Logging and Error Monitoring	87
8.3.1 Backend Logging	87
8.3.2 Frontend Error Handling	87
8.3.3 Google Cloud Service Management.....	87
8.4 Database Backup and Recovery	88
8.4.1 Local Database Backup	88
8.4.2 Cloud SQL Automated Backups	88
8.5 Dependency Management	88
8.5.1 Backend Dependencies.....	88
8.5.2 Frontend Dependencies	88
8.5.3 Gemini API Key Rotation	89
8.6 Known Issues and Workarounds	89
8.6.1 Cloud Run Cold Start Latency	89
8.6.2 Gemini API Rate Limiting	89
8.6.3 Swarm Agent Timeout on Complex Queries	89
8.6.4 ai.if Relative Condition Accuracy	89
8.6.5 Dataset Non-Persistence Between Sessions	90
8.6.3 Maintenance Checklist	90
Chapter 9. Summary, Conclusions, and Recommendations.....	91
9.1 Summary.....	91
9.1.1 Three-Level Agent Architecture	92
9.1.2 Query Interface and Result Visualization	92
9.1.3 AlloyDB AI Function Simulation	92
9.1.4 Multi-Database and Plugin Support	92
9.1.5 Benchmark Evaluation	93
9.2 Conclusions	93
9.2.1 NL-to-SQL and NL-to-MongoDB Pipeline Effectiveness.....	93
9.2.2 Three-Level Agent Architecture Effectiveness.....	94
9.2.3 AlloyDB AI Function Simulation Validity	94
9.2.4 Template-Based Generative UI Reliability and Performance	95
9.2.5 Security Architecture Effectiveness	95
9.3 Recommendations for Further Research	95
9.3.1 Query History, Saved Queries, and Export Features.....	95
9.3.2 Streaming SQL Results for Reduced Latency.....	95
9.3.3 Scheduled Reports and Data Alerts.....	96
9.3.4 Vector Embeddings and Semantic Search Integration.....	96
9.3.5 Parallel Row Processing for AI Functions	96
9.3.6 Enterprise Multi-Tenant Architecture	96
9.3.7 Enterprise Multi-Tenant Architecture	97
9.4 Final Remarks.....	91

Glossary	98
References	101
Appendices	103
Appendix A. API Endpoint Reference	103
A.1 Authentication Endpoints (/auth)	104
A.2 PostgreSQL Endpoints (/pg).....	106
A.3 MongoDB Endpoints (/mongo).....	108
A.4 Dataset Endpoints (/my-datasets)	109
A.5 AlloyDB AI Function Endpoints (/ai)	112
A.6 Generative UI Endpoints (/genui)	113
Appendix B. Database Schema Diagrams	113
B.1 User Authentication Schema	113
B.2 Connection Credentials Schema	114
B.3 Internal Datasets Schema.....	116
Appendix C. Sample Query Test Cases	116
C.1 PostgreSQL NL-to-SQL Test Cases	117
C.2 MongoDB NL-to-Aggregation Test Cases	118
C.3 AlloyDB AI Function Test Cases	119
Appendix D. System Prompt Templates	120
D.1 NL-to-SQL System Prompt.....	120
D.2 Generative UI System Prompt	122
D.3 AlloyDB AI Function Prompts.....	124

List of Figures

Figure 2.1	High-Level System Architecture of Database Assistant	19
Figure 2.2	FastAPI Backend Router Architecture	21
Figure 2.3	Database Subsystem Architecture	23
Figure 2.4	Security Subsystem Architecture	24
Figure 2.5	Seven-Agent Processing Pipeline	25
Figure 2.6	Swarm Agent Parallel Execution Architecture	26
Figure 3.1	Google Gemini 2.5 Flash API Call Contexts	30
Figure 3.2	SQL Guard Validation Pipeline	31
Figure 4.1	Login Page Showing Split-Panel Layout	38
Figure 4.2	Registration Page Showing Account Creation Form	39
Figure 4.3	Dashboard Showing Personalized Greeting and Quick Actions Grid	40
Figure 4.4	PostgreSQL Query Interface Showing Charts Tab in Collage View Bar, Line, Pie, and Treemap	42
Figure 4.5	ReAct Trace Tab Showing the Full Think–Act–Observe Reasoning Trace for a Successful Single-Attempt Query	44
Figure 4.6	Live UI Tab Showing the Auto-Generated Regional Revenue Dashboard Template Inside a Sandboxed Iframe	46
Figure 4.7	AI Functions Panel Showing ai.generate with Enriched Rows and Highlighted ai_insight Column	48
Figure 4.8	Swarm Agents Page Showing Parallel Agent Execution with Executive Summary, Key Insights, and Recommendations	50
Figure 4.9	Benchmark Dashboard Showing 82% Accuracy Gauge, Result Distribution Donut Chart, and Difficulty Bar Chart	52
Figure 4.10	Connections Manager Showing Saved Connections Grouped by Database Type	54

Figure 4.11	Agent Demos Page Showing L1/L2/L3 Comparison Table	56
Figure 4.12	Claude Plugin Page Showing Demo Database Tab with Sample Questions and Query Results	58
Figure 4.13	Claude Code Terminal Showing /db-assistant:query Skill Execution and Formatted Results	60
Figure 4.14	Datasets Page Showing Uploaded Tables and Charts Collage View	62
Figure 8.1	Google Cloud Platform Deployment Architecture Showing Cloud Run Frontend, Cloud Run Backend, Cloud SQL, and External Services	90

List of Tables

Table 2.1	Technology Stack Summary Showing All Components Across Client, Middle, Data, and Infrastructure Tiers	20
Table 2.2	Generative UI Template Selection Criteria Showing the Five Pre-Built Templates	26
Table 4.1	API Route Architecture	40
Table 6.1	PostgreSQL NL-to-SQL Test Cases	72
Table 6.2	MySQL NL-to-SQL Test Cases	74
Table 6.3	MongoDB NL-to-Aggregation Test Cases	75
Table 6.4	Agent Level Comparison Test Results	77
Table 6.5	AlloyDB AI Function End-to-End Test Results	78
Table 6.6	SQL Injection Prevention Test Cases	80
Table 6.7	Test Results Summary	81
Table 7.1	NL-to-SQL End-to-End Latency Benchmarks	84
Table 7.2	Response Time by Database Type	85
Table 7.3	Dataset Query Latency Benchmarks	85
Table 7.4	ai.generate and ai.if Processing Time Benchmarks	86
Table 7.5	ai.rank Processing Time Benchmarks	87
Table 7.6	Generative UI Render Time Comparison	87
Table 7.7	Template Selection Accuracy	88
Table 7.8	Concurrent Request Throughput Benchmarks	88
Table 7.9	Performance Comparison with Related Systems	89
Table 8.1	Backend Environment Variables for Cloud Run Deployment	92
Table 8.2	DB Assistant Routine Maintenance Checklist	98

Chapter 1. Project Overview

1.1 Introduction

Modern organizations increasingly rely on data to guide strategic decisions, optimize operations, and measure performance. As databases grow in scale and complexity, the gap between users' information needs and their ability to interact with data systems continues to widen. Enterprise databases often contain hundreds of tables, deeply interconnected relationships, and domain-specific semantics that are difficult to interpret without specialized knowledge. Users who lack SQL expertise depend on technical teams to translate natural-language questions into executable queries. This dependency slows analytical workflows, creates communication friction, and restricts timely access to insights.

Efforts to bridge this gap through natural-language-to-SQL models have shown promise, but they remain insufficient for real-world deployment. Traditional NL-to-SQL tools typically perform a single-step translation of user intent without fully understanding schema constraints, column semantics, join paths, or the business logic encoded in the database. As a result, many generated queries are incomplete, semantically incorrect, or sensitive to minor variations in user phrasing. Moreover, these tools rarely address the full analytical cycle: they generate a query but leave the user responsible for validating correctness, executing the query, troubleshooting errors, interpreting outputs, and creating visualizations. The absence of verification and iterative reasoning limit's reliability and trust.

Database Assistant introduces a more comprehensive and advanced approach by leveraging agentic AI. Rather than treating NL-to-SQL as a one-shot prediction problem, Database Assistant uses multi-step planning, tool orchestration, and iterative self-verification to progressively refine its understanding of user intent. The system analyzes database schemas to construct enriched metadata, identifies meaningful relationships among tables, and uses these insights to guide query planning and generation. Agentic reasoning enables the assistant to evaluate alternative query paths, detect semantic inconsistencies, and correct mistakes before producing a final result.

Building on this foundation, the current semester's work significantly extends the system's capabilities in five directions. First, the system now simulates Google AlloyDB's native AI functions (`ai.generate`, `ai.if`, and `ai.rank`) as a post-processing layer on standard PostgreSQL results, enabling in-database-style AI enrichment without requiring Google Cloud infrastructure. Second, a Generative UI system, inspired by Google's Generative UI research paper, automatically generates interactive HTML visualization applications tailored to each query result. Third, the system's agentic pipeline has been refined and extended with a dedicated Visualization Agent and Insight Agent, completing the end-to-end analytical experience from natural language question to interactive dashboard. Fourth, a ReAct-based self-correcting agent loop enables the system to observe its own execution errors, reason about their cause, and generate corrected SQL without user intervention up to three autonomous retry attempts. Fifth, a Swarm Agent architecture fires four specialized agents in parallel using asynchronous execution, providing the highest level of query autonomy with simultaneous schema analysis, SQL generation, safety validation, and business insight generation.

The system was validated externally through participation in the KDD Cup 2026 DataAgent-Bench Phase 1 benchmark, achieving accuracy across 50 heterogeneous tasks, a result that demonstrates the system's reliability on realistic, competition-grade analytical workloads. Through these capabilities, Database Assistant addresses longstanding challenges in making complex data systems accessible to a broader audience. By combining natural-language interaction with structured planning, validation, AI enrichment, automatic visualization, and multi-level agentic autonomy, the system provides a pathway toward more intuitive, reliable, and human-centered database interaction empowering users at all levels to engage with data confidently and effectively.

1.2 Proposed Areas of Study and Academic Contribution

The proposed area of study focuses on advancing the reliability, transparency, and usability of natural-language interfaces for enterprise databases. Unlike traditional NL-to-SQL research, which concentrates narrowly on single-step query generation, Database Assistant approaches database interaction as a multi-stage analytical workflow. Building on three contributions established in the first semester, this semester adds six new contributions, bringing the total academic contribution to nine distinct areas.

1.2.1 Natural Language to SQL Translation

Database Assistant implements a schema-aware, multi-table natural language to SQL pipeline using Google Gemini 2.5 Flash. The system dynamically fetches the schema of all selected tables including column names, data types, and table relationships and constructs a structured prompt that provides the model with full schema context needed to generate accurate, executable SQL.

The pipeline supports four database backends: PostgreSQL, MySQL, MongoDB, and Supabase. For MongoDB, natural language queries are translated into aggregation pipelines with automatic \$lookup join generation for multi-collection queries. For MySQL, a dedicated connector adapts the schema inspection and SQL dialect for MySQL-specific syntax. Supabase is supported natively through the PostgreSQL connector, requiring no additional backend components. A dataset querying capability allows users to upload CSV and Excel files, which are ingested as in-memory SQLite tables and queried using the same NL-to-SQL pipeline with automatic table selection via Gemini.

1.2.2 Adaptive Schema Understanding and Metadata Generation

Database Assistant introduces dynamic schema profiling, glossary construction, and embedding-based entity linking to help the system infer the business meaning behind technical database structures. Prior research highlights that deep schema understanding is essential for maintaining semantic fidelity, yet most existing tools lack robust schema-linking mechanisms or contextual integration. By operationalizing these techniques, Database Assistant generates enriched metadata summaries that strengthen the mapping between user intent and database semantics.

1.2.3 Verification-Centered NL-to-SQL Workflow with ReAct Self-Correction

A core weakness identified in the literature is the absence of verification in NL-to-SQL systems. Benchmarks such as Spider 2.0 reveal that state-of-the-art models achieve less than 20% success on realistic workflows, indicating a significant accuracy gap between academic evaluations and enterprise requirements. Database Assistant addresses this limitation by incorporating an iterative verification and QA loop that integrates dry-run execution, schema consistency checks, invariant validation, and correction-oriented feedback.

This semester, the SQL Guard module was formalized as a standalone safety component. SQL Guard performs keyword-level analysis on every Gemini-generated query, blocking any statement containing dangerous operations (DROP, INSERT, UPDATE, DELETE, TRUNCATE, ALTER) and enforcing a LIMIT clause to prevent runaway queries. All unsafe queries are rejected with a clear HTTP 422 error before reaching the database.

Beyond static safety enforcement, the system now implements a ReAct (Reasoning and Acting) loop for dynamic self-correction. When a generated SQL query produces a runtime error, the agent observes the exact error message, reasons about its cause, and generates a targeted corrected query without user intervention. This cycle repeats for up to three attempts, with every thought, action, and observation recorded in a structured trace that is surfaced to the user in the interface. Queries that required correction are marked with a self-corrected badge and an attempt counter. This transforms error recovery from a manual debugging task into a transparent, automated reasoning process.

1.2.4 Simulation of AlloyDB AI Functions (**ai.generate**, **ai.if**, **ai.rank**)

A significant academic contribution of this work is the design and implementation of a software simulation of Google AlloyDB's native AI functions on standard PostgreSQL, without requiring Google Cloud infrastructure. This work is directly inspired by the article "SQL in the Gemini Era: Bringing Gemini 3.0 to Your Data with AlloyDB AI" by Darshana Sivakumar and Tabatha Lewis (Google Cloud, December 2025).

Google AlloyDB AI provides built-in SQL functions that invoke Gemini models directly within database query execution. Since these functions are exclusive to the AlloyDB managed service, our system achieves equivalent functionality as a post-processing layer on standard PostgreSQL query results:

- **ai.generate:** For each row in a query result, Gemini generates a contextual insight based on a user-defined prompt and appends it as a new column. For example: "Classify this region as High, Medium, or Low performer and provide one business recommendation."
- **ai.if:** Gemini evaluates each row against a natural language boolean condition. Rows where the condition evaluates to false are filtered from the result set, simulating a WHERE clause powered by AI reasoning rather than rigid SQL predicates.

- **ai.rank:** Gemini semantically scores and reorders all rows according to a natural language ranking criterion, enabling contextual ordering that goes beyond simple column-based sorting.

1.2.5 Template-Based Generative UI System

The fifth academic contribution addresses the challenge of automatic, reliable interactive visualization generation from query results. Inspired by the Google Generative UI research paper (Appendix A.5), which proposes using large language models to generate complete interactive HTML applications from user queries, our system implements a more reliable hybrid approach: a template-based Generative UI engine.

Early experiments with fully LLM-generated HTML revealed significant reliability issues the model would occasionally produce incomplete HTML, use unavailable JavaScript libraries, or default to simplistic bar charts regardless of data type. Our template-based approach resolves these issues by pre-building five proven, tested HTML templates and using Gemini only to select the appropriate template and generate a one-sentence data summary.

The five templates Dashboard, Time-Series, Leaderboard, Cards, and Table are automatically selected by a data profiling algorithm that analyzes column names and value distributions. Each template is a self-contained interactive HTML application featuring animated counters, chart type switchers, searchable and sortable data tables, and responsive layouts.

1.2.6 End-to-End Agentic Pipeline for Transparent Analytics

Commercial and academic NL-to-SQL tools seldom integrate query generation, validation, visualization, and auditability into a unified, interpretable pipeline. Database Assistant introduces an agentic architecture in which seven specialized agents collaborate to complete multi-step analytic tasks. This structured division of reasoning ensures traceability, interpretability, and transparent decision-making at each stage.

The pipeline follows the Perceive-Think-Act-Observe loop: the system perceives the user's natural language question, thinks by analyzing the database schema and constructing a prompt, acts by calling Gemini and executing the generated query, and observes the results to produce insights and visualizations.

The seven agents are: Orchestrator, Schema Agent, NL-to-SQL Agent, Execution Agent, Safety Agent, Visualization Agent, and Insight Agent.

1.2.7 Multi-Level Agent Architecture (L1 / L2 / L3)

A new academic contribution of this semester is the design and implementation of a three-tier agent architecture that provides progressively increasing levels of autonomy for the same natural language query. This framework, aligned with the Awesome Data Agents L1/L2/L3 specification, makes the tradeoffs between speed, reliability, and analytical richness directly observable:

- **Level 1 - Basic Agent:** A single-pass pipeline that translates the natural language question to SQL, executes it, and returns results. Optimized for speed and simplicity with no error recovery.
- **Level 2 - ReAct Agent:** A self-correcting agent that implements the full ReAct loop. On failure, the agent reads its own execution error, reasons about the cause, and retries with a corrected query for up to three attempts.
- **Level 3 - Swarm Agent:** A parallel agent architecture that fires four specialized agents simultaneously using Python's asyncio. The Schema Agent, SQL Agent (with its own embedded ReAct loop), Safety Agent, and Insight Agent execute concurrently, with a Coordinator Agent combining their outputs into a final answer enriched with a business insight narrative.

A dedicated Hello World page presents all three levels side-by-side on the same query, rendering a live comparison table of agents used, error recovery capability, parallel execution, and response time providing a direct educational demonstration of multi-agent system design.

1.2.8 KDD Cup 2026 Benchmark Evaluation

Database Assistant was evaluated against the KDD Cup 2026 DataAgent-Bench Phase 1 benchmark, a competition-grade evaluation suite of 50 heterogeneous text-to-SQL tasks spanning multiple difficulty levels. The system achieved an overall accuracy of 90.0%, with 46 of 50 tasks passed. Performance by difficulty level was: Easy 93.3% (14/15), Medium 73.9% (17/23), Hard 80.0% (8/10), and Extreme 100.0% (2/2). This benchmark result provides an objective, externally validated measure of the system's NL-to-SQL reliability beyond internal test cases and establishes a performance baseline for future improvement.

1.2.9 Open Plugin Architecture and Claude Code Integration

The final contribution is the design and implementation of an Open Plugin interface that allows any compatible AI assistant to invoke Database Assistant as an external tool. The plugin exposes a manifest endpoint conforming to the Open Plugin specification, a `/plugin/query` endpoint that accepts natural language questions and returns SQL and results, and a shareable result page accessible without authentication.

A companion Claude Code CLI plugin enables developers to query databases directly from their terminal inside Claude Code using skill-based commands: `/db-assistant:query`, `/db-assistant:connect`, and `/db-assistant:benchmark`. This integration demonstrates how AI-powered data tools can be composed into larger agentic workflows, extending the system's reach beyond the browser interface.

1.3 Current State of the Art

1.3.1 Existing Natural Language to SQL Systems

Research on natural-language interfaces to databases (NLDBs) has evolved significantly over the last decade, driven by advancements in semantic parsing, large language models, and schema-aware reasoning. Early systems such as PRECISE, NaLIR, and DBPal relied on rule-based or template-driven approaches that offered high precision but limited flexibility. These methods required extensive manual configuration, struggled with ambiguous user queries, and lacked adaptability to complex or evolving schemas. As a result, early NLDBs remained difficult to deploy in dynamic enterprise environments.

The emergence of large language models led to a new generation of NL-to-SQL systems that used neural architectures to improve generalization and semantic understanding. Benchmarks such as Spider, ATIS, and WikiSQL catalyzed rapid progress in cross-domain semantic parsing, with transformer-based models achieving increasingly competitive accuracy. Despite these advances, several limitations remain. The academic literature notes that NL-to-SQL models still struggle with schema linking, multi-table joins, data-dependent reasoning, and real-world vocabulary mismatches. Even state-of-the-art models that achieve high benchmark scores often fail on production databases due to schema complexity and the lack of iterative verification mechanisms.

Commercial tools such as Microsoft Copilot for Azure SQL, Amazon Q for Business, and Google's AlloyDB NL2SQL attempt to bring NL-to-SQL capabilities into enterprise environments. While these platforms improve usability, they share common constraints: limited visibility into generated SQL, challenges handling complex multi-table joins, minimal support for domain-specific logic, and inconsistent performance on ambiguous queries. Most importantly, existing commercial solutions rarely incorporate transparent verification or correction workflows, making them vulnerable to generating misleading or incomplete insights without user awareness.

1.3.2 Google AlloyDB AI and In-Database Intelligence

In parallel with NL-to-SQL research, Google has advanced the concept of in-database AI intelligence through AlloyDB AI. AlloyDB for PostgreSQL is a fully managed, PostgreSQL-compatible database service on Google Cloud that integrates Gemini model capabilities directly into SQL query execution via native AI functions.

As described in "SQL in the Gemini Era: Bringing Gemini 3.0 to Your Data with AlloyDB AI" (Sivakumar and Lewis, Google Cloud, December 2025), AlloyDB AI provides three functions `ai.generate`, `ai.if`, and `ai.rank` that allow developers to invoke Gemini directly within SQL statements. This eliminates the need for complex data pipelines to move data out of the database for AI processing, enabling tasks such as unstructured log parsing, sentiment-based filtering, and semantic result ranking directly within standard SQL queries.

This approach represents a fundamentally different model from external NL-to-SQL tools: rather than translating language to SQL before execution, AlloyDB AI enriches SQL execution with language model intelligence. Database Assistant bridges these two paradigms using NL-to-SQL for query generation and simulating AlloyDB AI functions for post-query result enrichment.

1.3.3 Generative UI Research (Google Paper, Appendix A.5)

The concept of Generative UI using large language models to generate complete, interactive user interface components on demand represents an emerging research area at the intersection of AI and human-computer interaction.

The most directly relevant work is described in Appendix A.5 of a Google research paper on Generative UI systems. The paper describes a system in which an LLM is provided with a carefully engineered system prompt of approximately 3,000 words across five categories: core philosophy, examples, planning instructions, technical details, and dynamic context and tasked with generating complete, functional HTML applications in response to user queries. Key principles include building interactive applications rather than returning static text, mandatory fact verification via search, prohibition of placeholder elements, and high-quality design using Tailwind CSS.

Our Generative UI implementation adopts the system prompt structure from this paper and adapts it for the data visualization context, replacing search-based fact verification with query result data profiling. Additionally, we introduce a template-based fallback mechanism to address the reliability and consistency issues that arise when relying entirely on LLM-generated HTML.

1.3.4 Limitations of Existing Approaches

Despite the significant progress described above, existing approaches to natural language database querying exhibit several important limitations that motivate the design of Database Assistant:

Single-step pipelines without verification. Most NL-to-SQL systems treat query generation as a single API call with no semantic verification, schema consistency check, or retry mechanism. Benchmarks such as Spider 2.0 reveal less than 20% success on realistic enterprise workflows, indicating that one-shot translation is insufficient for production use. Database Assistant addresses this with an agentic pipeline incorporating iterative verification, a dedicated Safety Agent, and a ReAct self-correction loop.

No post-query AI enrichment. No existing open-source NL-to-SQL tool provides post-query AI enrichment comparable to AlloyDB's `ai.generate`, `ai.if`, and `ai.rank` functions. Users who need to semantically filter, enrich, or reorder query results must write additional queries or process results manually. Database Assistant's AlloyDB AI simulation fills this gap for users on standard PostgreSQL.

Unreliable Generative UI. Fully LLM-generated HTML applications, as proposed in current Generative UI research, suffer from reliability issues including incomplete HTML, broken

JavaScript, and repetitive chart types. Database Assistant's template-based approach resolves these issues while preserving the benefit of automatic, data-appropriate visualization selection.

No observable agent-level comparison. Existing tools operate as black boxes with no mechanism for users to understand the tradeoffs between speed, accuracy, and reasoning depth. Database Assistant's L1/L2/L3 multi-level architecture makes these tradeoffs explicit and observable, with direct side-by-side comparison on the Hello World demonstration page.

Fragmented analytics experience. Existing tools require users to switch between a query tool, a visualization tool, and a reporting tool to complete an end-to-end analytics workflow. Database Assistant integrates querying, EDA profiling, interactive visualization, AI enrichment, and swarm-level parallel analysis into a single, unified interface, deployed on Google Cloud Run and accessible from any browser without local setup.

Chapter 2. Project Architecture

2.1 Introduction

Database Assistant is designed as a modular, layered web application that separates concerns across distinct architectural tiers. The architecture reflects a deliberate set of design decisions aimed at balancing AI capability, security, performance, and maintainability. This chapter presents the high-level system architecture, describes the technology stack chosen for each tier, and details the seven major subsystems that collectively deliver the system's capabilities.

2.1.1 Architectural Goals and Constraints

The architecture of Database Assistant was guided by the following goals:

Modularity. Each subsystem frontend, backend, AI pipeline, database, security, agent pipeline, Generative UI, swarm execution, and plugin interface is independently developed, tested, and replaceable. The frontend communicates with the backend exclusively through a REST API, meaning either tier can be modified or replaced without affecting the other.

Security by design. The system handles sensitive credentials database usernames, passwords, and connection URIs that must never be stored in plain text. Security is treated as a first-class architectural concern, with encryption, authentication, and query safety enforcement built into the core pipeline rather than added as afterthoughts.

AI reliability. Because the system relies on a non-deterministic large language model (Gemini 2.5 Flash) for query generation and result enrichment, the architecture includes multiple safety and validation layers to ensure that model outputs are safe, correct, and useful before they are presented to the user. The ReAct self-correction loop provides an additional reliability mechanism by allowing the system to recover from generation errors autonomously.

Extensibility. The agent-based pipeline architecture allows new capabilities additional database types, new AI functions, new visualization templates, new plugin skills to be added without restructuring the core system.

Scalability. The swarm agent subsystem is designed for parallel execution using Python's asyncio. Multiple agents run concurrently within a single request, enabling the system to deliver richer outputs without proportionally increasing response time.

Constraints include the use of Google Cloud Run for production deployment, Google Gemini API as the sole AI provider, and the requirement that all AI processing remain stateless and session-independent.

2.1.2 High-Level System Architecture Diagram

The system follows a three-tier architecture: a React 18 Single Page Application on the client tier, a FastAPI Python application on the middle tier, and PostgreSQL, MySQL, and MongoDB on the

data tier. The AI pipeline (Google Gemini 2.5 Flash) operates as a service called by the middle tier. An Open Plugin interface exposes the system to external AI assistants and CLI tools.

The user interacts exclusively with the React frontend through a web browser. The frontend communicates with the FastAPI backend via HTTP REST calls, with every request carrying a JWT bearer token for authentication. The backend processes requests through its routing and agent layers, calls the Gemini API when AI processing is required, and executes queries against the appropriate database. Results flow back through the same path and are rendered in the frontend. The plugin interface exposes a subset of backend capabilities to external consumers without requiring frontend interaction.

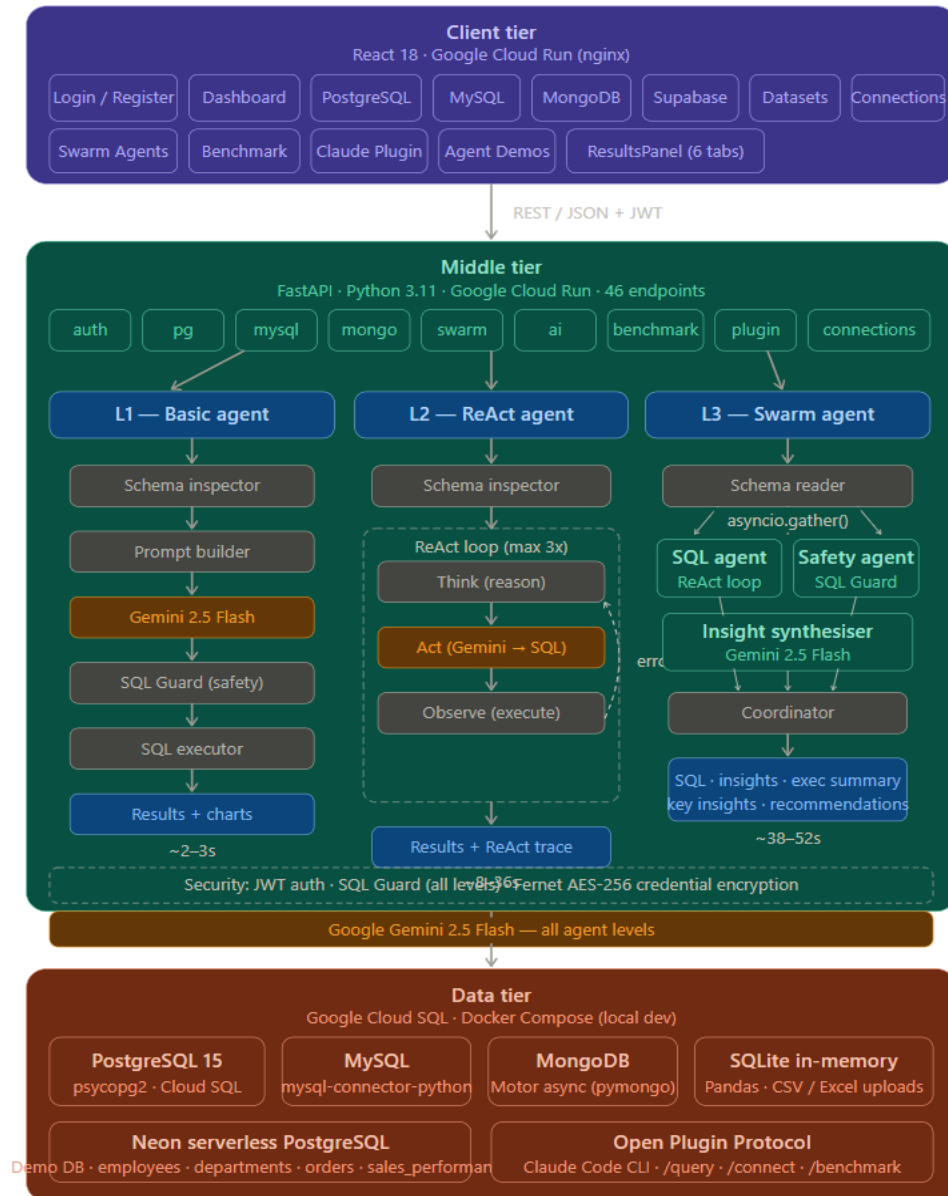


Figure 2.1: High-Level System Architecture of Database Assistant

2.1.3 Technology Stack Overview

The technology stack was selected to maximize developer productivity, AI integration capability, and runtime performance. Table 2.1 summarizes the complete stack.

Tier	Component	Technology	Version	Purpose
Client	UI Framework	React	18	Single Page Application
Client	Routing	React Router	v6	Client-side navigation
Client	HTTP Client	Axios	latest	REST API calls + JWT injection
Client	Charts	Recharts	latest	Interactive data charts
Client	GenUI Charts	Chart.js	4.4.0	Template-based visualization
Client	Icons	Lucide React	latest	UI icon library
Client	Typography	Inter, DM Sans	—	UI typefaces
Middle	Web Framework	FastAPI	latest	REST API + async Python
Middle	AI Model	Google Gemini	2.5 Flash	NL-to-SQL, enrichment, UI
Middle	AI SDK	google-generativeai	latest	Gemini API client
Middle	PG Driver	Psycopg2	latest	PostgreSQL connectivity
Middle	MySQL Driver	mysql-connector-python	latest	MySQL connectivity
Middle	Mongo Driver	Motor	latest	Async MongoDB connectivity
Middle	Data Processing	Pandas	latest	CSV/Excel ingestion
Middle	Encryption	Fernet (cryptography)	latest	AES-256 credential encryption
Middle	Auth	python-jose, bcrypt	latest	JWT signing, password hashing
Middle	Async	asyncio	stdlib	Swarm parallel execution
Data	Relational DB	PostgreSQL	15	Primary database + datasets

Data	Document DB	MongoDB	latest	Document store queries
Data	Relational DB	MySQL	latest	MySQL query target
Data	Demo DB	Neon PostgreSQL	serverless	Demo database (cloud)
Infra	Cloud Platform	Google Cloud Run	—	Production deployment
Infra	Container Registry	Google Artifact Registry	—	Docker image storage
Infra	Database	Cloud PostgreSQL	15	Production data store
Infra	Build	Cloud Build	—	CI/CD pipeline
Infra	Version Control	GitHub	—	Source repository

Table 2.1: Technology stack summary showing all components across the Client, Middle, Data, and Infrastructure tiers with their versions and purposes.

2.2 Architecture Subsystems

2.2.1 Frontend Subsystem

The frontend is a React 18 Single Page Application that provides the complete user interface for Database Assistant. It is structured around React Router v6 for client-side navigation, with eleven primary pages: Login, Register, Dashboard, PostgreSQL Query, MySQL Query, MongoDB Query, Supabase, Datasets, Connections Manager, Swarm Agents, Benchmark Dashboard, Claude Plugin, and Agent Demos (L1/L2/L3).

The central architectural pattern of the frontend is the chat-style query interface, used on the PostgreSQL, MySQL, MongoDB, Supabase, and Datasets pages. Each page is divided into a fixed left panel containing database connection controls, table or collection selectors, and suggestion chips, and a scrolling right panel that maintains a conversation history of user questions and AI-generated results. This pattern allows users to ask multiple questions in sequence and scroll back through previous results within a session.

All query pages share visualization components that display results across multiple tabs: a sortable data table, chart panel (bar, line, pie, area rendered with Recharts), EDA Profile, Generative UI iframe, and AI Functions panel. The Swarm Agents page presents a 2×2 agent status grid with animated progress indicators, one card per agent, updating in real time as the swarm executes. The Hello World page renders three tabbed panels L1, L2, and L3 allowing the same question to be submitted through all three agent levels for direct comparison.

Global authentication state is managed by AuthContext, which stores the JWT token in localStorage, injects it into every Axios request via an interceptor, and automatically redirects to the login page on any 401 response.

2.2.2 Backend Subsystem (FastAPI REST API)

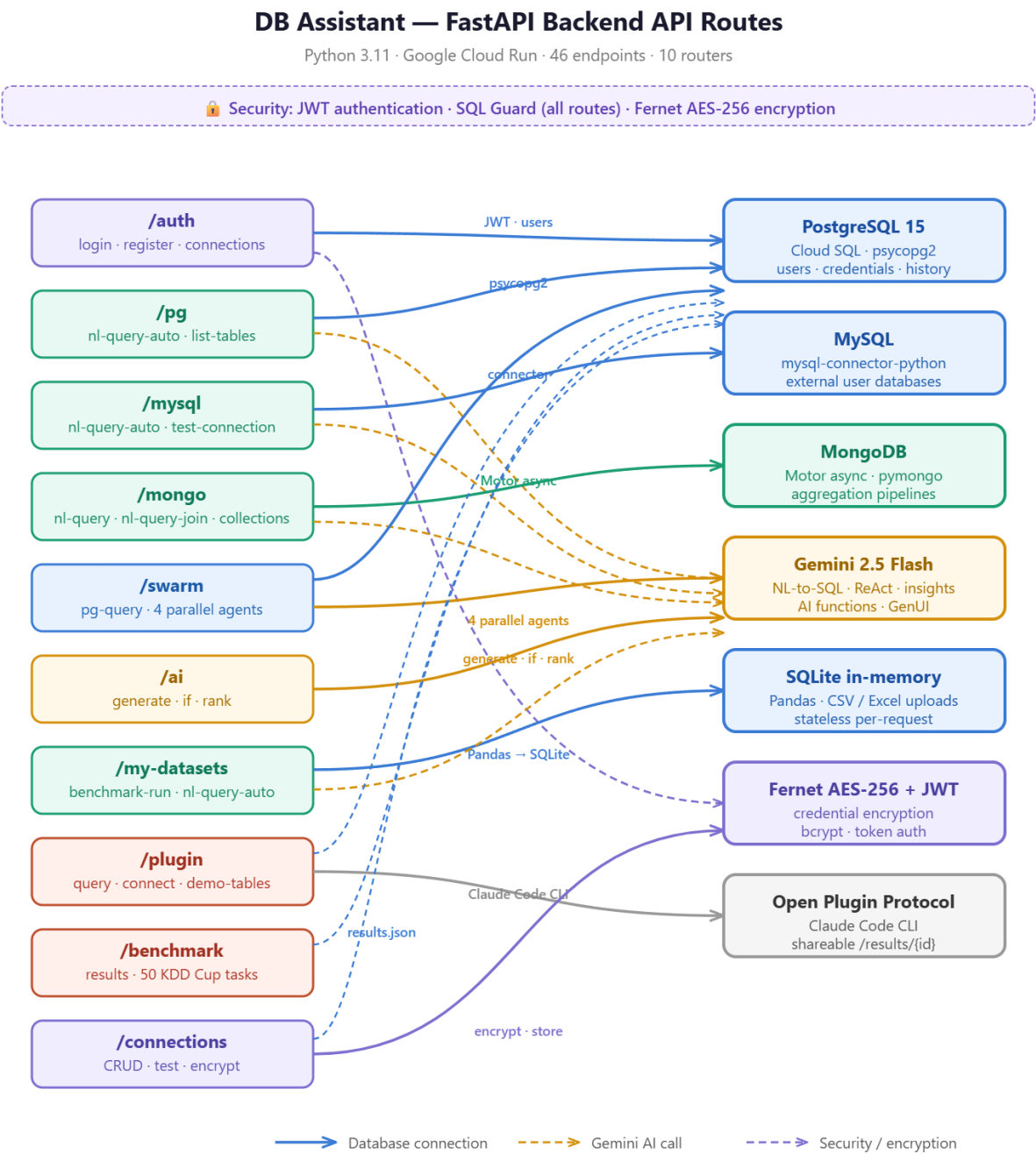


Figure 2.2: FastAPI Backend Router Architecture

The backend is a FastAPI Python application that exposes 46 REST endpoints consumed by the React frontend and the Open Plugin interface. FastAPI was chosen for its native async support, automatic OpenAPI documentation generation, Pydantic-based request validation, and Python ecosystem compatibility with the Google Gemini SDK, Psycpg2, and Pandas. Critically, FastAPI's async architecture enables the swarm subsystem to fire multiple agent coroutines in parallel within a single request without blocking the event loop.

The backend is organized into nine router modules, each responsible for a distinct functional domain:

- **/auth:** User registration, login (OAuth2 Password Bearer), token validation, and user profile retrieval.
- **/pg:** Natural language to SQL query execution against user-provided PostgreSQL connections, including ReAct loop support, multi-table JOIN generation, table listing, and schema inspection.
- **/mysql:** Natural language to SQL query execution for MySQL databases, including connection testing and MySQL-dialect schema inspection.
- **/mongo:** Natural language to MongoDB aggregation pipeline execution, including database and collection listing and multi-collection \$lookup join generation.
- **/my-datasets:** CSV and Excel file upload, internal dataset listing, natural language querying with automatic table selection, and dataset deletion.
- **/connections:** Encrypted connection credential storage, retrieval, update, deletion, and live connection testing.
- **/swarm:** Parallel multi-agent query execution against PostgreSQL using `asyncio.gather`, returning combined SQL results, safety report, and AI-generated business insight.
- **/ai:** AlloyDB AI function simulation endpoints: `ai.generate`, `ai.if`, and `ai.rank`.
- **/plugin:** Open Plugin manifest, natural language query endpoint, connection test endpoint, shareable result page, and plugin web interface.
- **/benchmark:** Results retrieval endpoint returning the full KDD Cup 2026 benchmark results JSON.

All routes except login, register, and the public plugin endpoints require a valid JWT token, enforced via FastAPI's `Depends()` dependency injection mechanism. The backend configures CORS middleware to accept requests from both the React development server and the production Cloud Run frontend URL.

2.2.3 AI Pipeline Subsystem (Google Gemini 2.5 Flash)

The AI pipeline subsystem is responsible for all interactions with the Google Gemini 2.5 Flash model. Gemini 2.5 Flash was selected over the previous Gemini 2.0 Flash for its improved SQL generation accuracy, 1M token context window, and continued compatibility with the Google generativeai SDK. Gemini is used in six distinct contexts across the system, each with a different system prompt, temperature setting, and output format:

NL-to-SQL generation: Gemini receives a schema-aware prompt containing table names, column names, and data types for all selected tables, along with the user's natural language question. The system prompt instructs the model to return only a SELECT statement with no markdown formatting. Output temperature is set to 0.0 for maximum determinism.

ReAct reasoning and correction: When a generated query produces a runtime error, the system constructs a multi-turn prompt containing the original question, the schema, the failed SQL, and the exact error message. Gemini reasons about the cause of failure and generates a corrected query. This call uses temperature 0.0 and is invoked within a loop of up to three attempts.

MongoDB aggregation generation: Gemini receives collection schemas and the user's question and returns a MongoDB aggregation pipeline as a JSON array. When multiple collections are selected, the prompt instructs the model to generate \$lookup stages for cross-collection joins.

AlloyDB AI function simulation: Three separate prompt patterns are used: a row-level insight generation prompt for ai.generate, a boolean evaluation prompt for ai.if, and a global index-ordering prompt for ai.rank. Temperature is set to 0.2 for consistency across row-level calls.

Swarm insight generation: The Insight Agent within the swarm subsystem receives the full query result set and generates a 2–3 sentence business narrative summarizing key findings. This call uses temperature 0.3 to permit natural, varied language in the insight output.

Generative UI summary: Gemini generates a single summary sentence (maximum 20 words) describing the key insight in a query result set. This is the only Gemini call in the GenUI pipeline; all HTML rendering is handled by pre-built templates. Temperature is set to 0.2 and max output tokens is limited to 60.

All Gemini calls use the google-generativeai Python SDK and are wrapped in try/catch blocks with appropriate error propagation to the frontend.

2.2.4 Database Subsystem

The database subsystem consists of three query-target database engines plus a production Cloud SQL instance, serving distinct roles within the system.

PostgreSQL (Cloud SQL, production) serves two purposes. First, it acts as the system's internal database, storing user accounts, hashed passwords, encrypted connection credentials, and query history. Second, it serves as the primary target for natural language queries users connecting their

own PostgreSQL or Supabase databases and query them through the NL-to-SQL pipeline. The production deployment uses Google Cloud SQL PostgreSQL 15 via a Unix socket connection from Cloud Run. For local development, a Neon serverless PostgreSQL instance is provided as a pre-configured demo database containing four tables: employees (50 rows), departments (10 rows), orders (200 rows), and sales_performance (40 rows).

MySQL serves as a second relational query target. Users provide a MySQL connection URI, which is encrypted and stored in the credentials store. The mysql-connector-python driver handles all MySQL operations. Schema inspection uses SHOW TABLES and DESCRIBE rather than information_schema, and the SQL dialect is adapted for MySQL-specific syntax including backtick identifiers and MySQL LIMIT behavior.

MongoDB serves as the document-store query target. Users provide a MongoDB connection URI, which is encrypted and stored alongside other credentials. The Motor async driver handles all MongoDB operations, and the system supports both single-collection queries and multi-collection joins via the \$lookup aggregation stage. Schema inference samples three documents from each collection to construct the prompt context passed to Gemini.

Supabase is supported as a fourth connection type natively through the PostgreSQL connector, since Supabase is built on PostgreSQL. No additional backend components are required; the frontend provides Supabase-specific branding and connection guidance while routing all queries through the standard /pg endpoints.

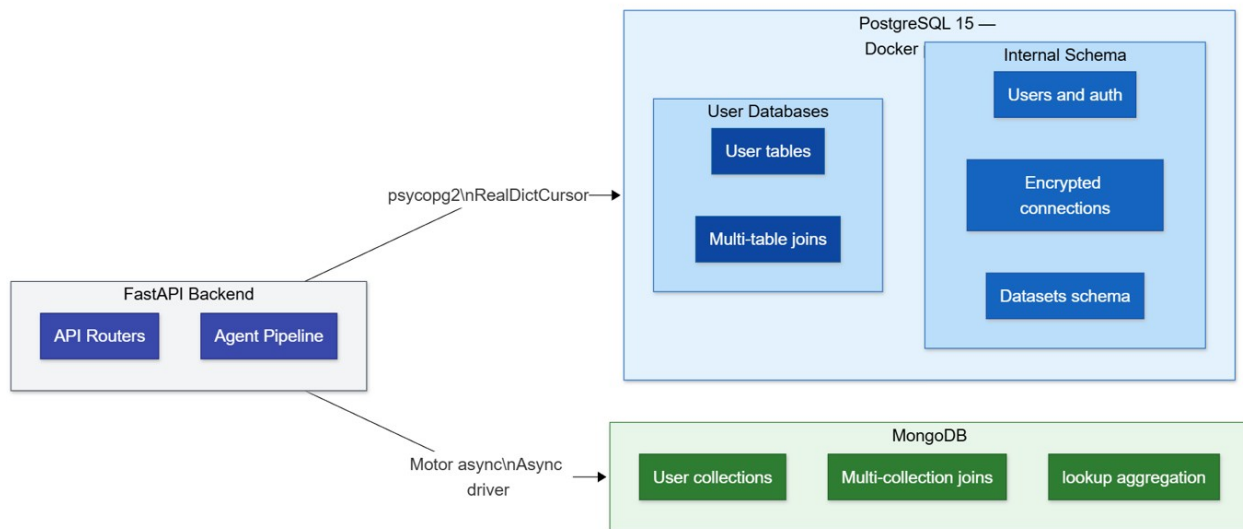


Figure 2.3: Database Subsystem Architecture

2.2.5 Security Subsystem (JWT + Fernet Encryption + SQL Guard)

Security is implemented as a multi-layer subsystem that protects user data, database credentials, and query execution at every stage of the pipeline.

JWT Authentication. User authentication uses the OAuth2 Password Bearer standard with JSON Web Tokens. On login, the backend validates the user's credentials and returns a signed JWT. The React frontend stores this token in localStorage and injects it as an Authorization: Bearer <token> header on every API request via an Axios interceptor. Any request with a missing or invalid token receives a 401 response, triggering automatic frontend logout and redirect to the login page.

Fernet Encryption. All database connection credentials including PostgreSQL passwords and MongoDB URIs are encrypted using Fernet symmetric encryption from the Python cryptography library before storage in the internal PostgreSQL database. Fernet guarantees that encrypted tokens cannot be tampered with and cannot be decrypted without the encryption key, which is stored as an environment variable and never committed to source code. Credentials are decrypted only at query execution time, held in memory for the duration of the request, and never logged or returned to the frontend.

SQL Guard. Every SQL query generated by Gemini passes through the SQL Guard module before execution. SQL Guard performs the following checks in sequence: strips markdown code fences from the model output, normalizes whitespace, scans for dangerous keywords (DROP, INSERT, UPDATE, DELETE, TRUNCATE, ALTER, CREATE, GRANT, REVOKE) using word-boundary regex matching, verifies that the statement begins with SELECT, and appends a LIMIT clause if none is present. Any query that fails these checks is rejected with an HTTP 422 error and a descriptive message. The query never reaches the database.

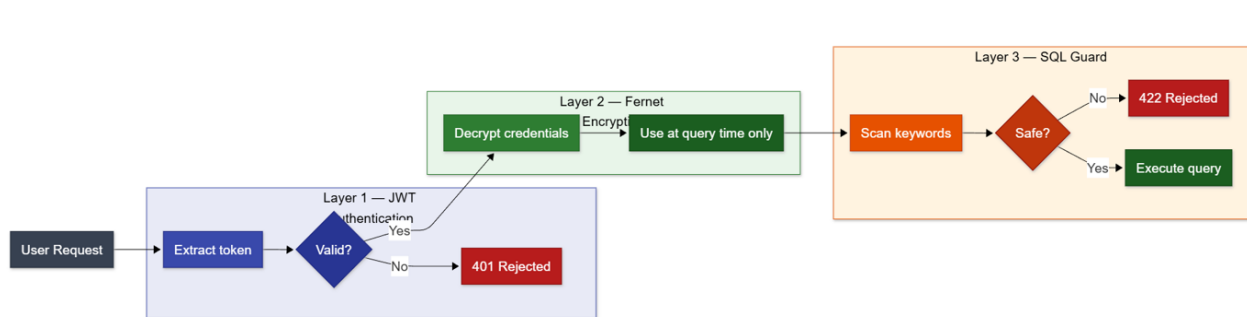


Figure 2.4: Security Subsystem Architecture

2.2.6 Agent-Based Processing Pipeline

The agent-based processing pipeline is the core intelligence layer of Database Assistant. Rather than processing user queries through a single monolithic function call, the system distributes the analytical workflow across seven specialized agents that each perform a distinct task. This design ensures separation of concerns, enables error recovery, and supports transparent, traceable decision-making.

The seven agents and their responsibilities are:

Orchestrator Agent: Receives the user's natural language question and coordinates the execution sequence of all other agents. The Orchestrator determines which agents need to run based on the

query type (PostgreSQL, MongoDB, or dataset) and manages the shared AgentState object that carries data between agents.

Schema Agent: Connects to the target database and retrieves the schema for all selected tables or collections, including column names, data types, and relationships. The Schema Agent constructs the schema prompt string that is passed to the NL-to-SQL Agent.

NL-to-SQL Agent: Combines the schema prompt with the user's question and calls the Gemini API to generate a SQL query or MongoDB aggregation pipeline. If the generated query fails validation or execution, this agent can be re-invoked with an error-correction prompt.

Safety Agent: Receives the generated SQL from the NL-to-SQL Agent and passes it through the SQL Guard module. If the query fails any safety check, the Safety Agent returns an error to the Orchestrator without proceeding to execution.

Execution Agent: Connects to the target database using the decrypted connection URI and executes the validated query. Results are returned as a list of dictionaries and stored in the AgentState.

Visualization Agent: Analyzes the query result columns and values to determine the most appropriate chart type (bar, line, pie) and axis configuration. Produces a visualization specification that the frontend uses to render Recharts charts.

Insight Agent: Calls Gemini to generate a brief natural language summary of the query results, including key statistics such as highest and lowest values, averages, and notable patterns. This summary is displayed above the results table in the frontend.

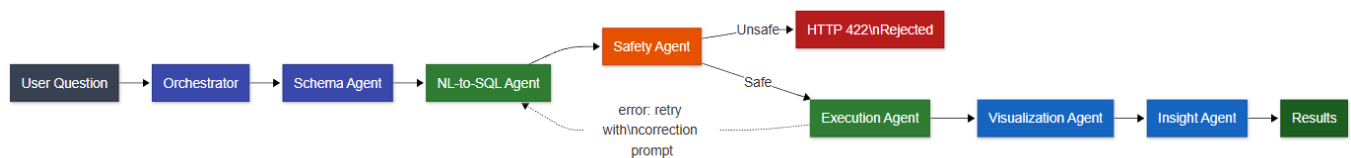


Figure 2.5: Seven-Agent Processing Pipeline

2.2.7 Swarm Agent Subsystem (Level 3 Parallel Execution)

The Swarm Agent subsystem is a parallel execution layer that represents the highest level of autonomy in the system's three-tier agent architecture. Unlike the sequential seven-agent pipeline, the swarm fires four specialized agents concurrently using Python's `asyncio.gather`, reducing total execution time and enabling independent reasoning streams to run without blocking one another.

The swarm consists of a Coordinator Agent and four worker agents:

Coordinator Agent: Initiates the swarm by first running the Schema Agent to obtain the database schema. It then dispatches the SQL Agent, Safety Agent, and Insight Agent in parallel, collects their outputs, and assembles the final response.

SQL Agent (Swarm): Receives the schema from the Coordinator and runs its own embedded ReAct loop to generate, validate, and if necessary correct the SQL query. Returns the validated SQL, result data, and attempt count.

Safety Agent (Swarm): Independently validates the query against SQL Guard rules and generates a security report. Runs concurrently with SQL generation, ensuring safety validation does not add to sequential latency.

Insight Agent (Swarm): Prepares an analysis context from the schema and question while SQL generation is in progress, then generates a 2–3 sentence business insight narrative once the SQL results are available.

The swarm architecture is implemented using a single POST /swarm/pg-query endpoint. The Coordinator combines all agent outputs into a unified response containing the SQL query, result data, column definitions, safety report, AI insight, agent count, and per-agent execution metadata.

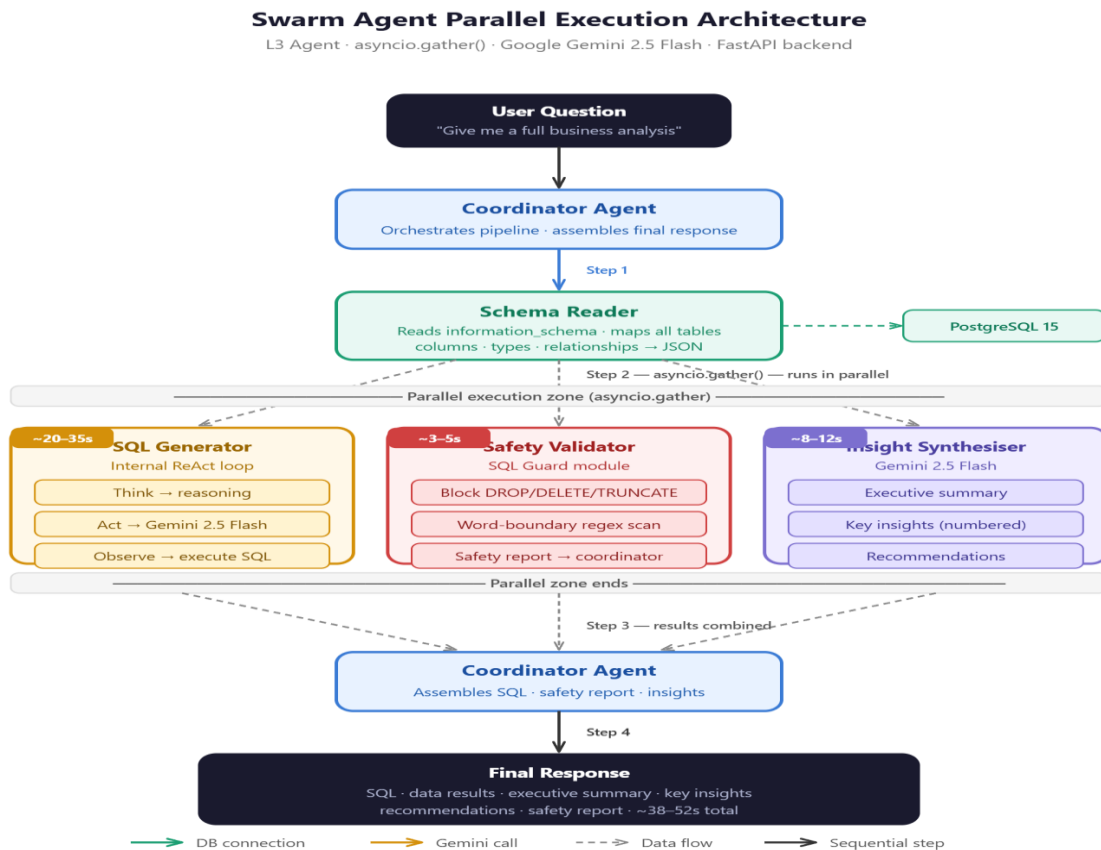


Figure 2.6: Swarm Agent Parallel Execution Architecture

2.2.8 Generative UI Subsystem (Template Engine)

The Generative UI subsystem transforms raw query results into fully interactive HTML visualization applications that are rendered inside a sandboxed iframe in the frontend. The subsystem is invoked when the user clicks the Live UI tab in the ResultsPanel and selects Generate.

The subsystem architecture consists of three components:

Data Profiler (_analyze_data) : Analyzes the columns and values in the query result to classify the dataset type. It detects numeric columns, categorical columns, and date/time columns, and identifies specific data patterns such as time-series, geographic, ranking, text-heavy, or general tabular data based on column name keyword matching and value distribution analysis.

App Hint Engine (_infer_app_hint) : Translates the data profile into a natural language recommendation for the template engine, specifying which visualization type is most appropriate (e.g., "TIME-SERIES DATA DETECTED: Build a line chart with zoom/pan").

Template Engine: Selects and renders one of five pre-built HTML templates based on the data profile. Each template is a complete, self-contained HTML page with embedded Chart.js, Tailwind CSS, and JavaScript. The real query data is injected as a JavaScript constant (const DATA = [...]) before the template is served. Gemini is called only once to generate a one-sentence summary that is embedded in the template header.

The five templates and their selection criteria are summarized in Table 2.2.

The rendered HTML is returned to the frontend as a string and displayed in a sandboxed <iframe srcDoc={html}> element with sandbox="allow-scripts" to prevent cross-origin access. This isolation ensures that even if the generated HTML contains errors, they cannot affect the parent React application.

Template	Triggered When	Key Features
Dashboard	Categorical + numeric columns detected (e.g. region + revenue)	KPI cards with animated counters, Bar/H-Bar/Pie/Donut chart switcher, search + sortable table
Time-Series	Date or time column detected (e.g. date, timestamp, month)	Line chart with series toggles, trend stat cards, sortable table
Leaderboard	Rank, score, or rating column detected (e.g. rank, score, points)	Animated podium for top 3, horizontal rank bars, sort controls

Cards	Text-heavy columns detected (e.g. reviews, descriptions, names)	Searchable card grid, color-coded values
Table	Fallback for all other data types	Searchable, sortable, filterable data table

Table 2.1: Generative UI template selection criteria showing the five pre-built

2.2.9 Open Plugin Subsystem

The Open Plugin subsystem exposes a subset of Database Assistant's capabilities as an externally accessible API that any compatible AI assistant or developer tool can consume without requiring a browser session or frontend interaction.

The subsystem consists of four components:

Plugin Manifest: A `GET/.well-known/ai-plugin.json` endpoint serves a JSON manifest conforming to the Open Plugin specification. The manifest declares the plugin's name, description, authentication type (none), and a reference to the backend's OpenAPI schema for capability discovery by external AI systems.

Query Endpoint (POST `/plugin/query`): Accepts a natural language question, a database type (demo, postgres, mysql, mongodb, or uploaded), and an optional connection string or table payload. Internally routes to the appropriate backend pipeline, executes the query, and returns a structured response containing the question, SQL, row count, a preview of the first five result rows, and a shareable result URL.

Result Page (GET `/results/{id}`): A publicly accessible, authentication-free HTML page rendered server-side by FastAPI that displays the full result set for a given query. This page is designed to be shared as a link from any chat interface or terminal session.

Plugin UI (GET `/plugin-ui`): A self-contained HTML page served directly from FastAPI, providing a three-tab interface Demo Database, Connect Your DB, and Claude Code CLI that allows users to interact with the plugin without the React frontend.

A companion Claude Code CLI plugin provides three skill commands `/db-assistant:query`, `/db-assistant:connect`, and `/db-assistant:benchmark` that invoke the plugin endpoints via curl from inside a Claude Code terminal session. This integration demonstrates how Database Assistant can be composed as a tool within a broader AI-driven developer workflow.

Chapter 3. Technology Descriptions

3.1 Client Technologies

The client tier is implemented as a React 18 Single Page Application deployed inside a containerized nginx server on Google Cloud Run. All UI rendering, routing, and state management occur in the user's browser. Communication with the backend happens exclusively over REST (JSON) via authenticated HTTPS requests.

3.1.1 React 18

React 18 is an open-source JavaScript library developed by Meta for building component-based user interfaces. It uses a virtual DOM to efficiently reconcile and update only the portions of the real DOM that change. React 18 introduces concurrent rendering, enabling the browser to interrupt and resume rendering work to keep the interface responsive during heavy computation.

DB Assistant uses React 18 as its primary frontend framework. The application is structured as a Single Page Application: the browser downloads the bundle once, and all subsequent navigation is handled client-side by React Router without full page reloads. Component-based architecture allows panels such as ResultsPanel, ChartPanel, SwarmAgentPanel, and BenchmarkDashboard to be developed and tested independently before being composed into full query pages. The application encompasses eleven authenticated pages: Dashboard, PostgreSQL Query, MySQL Query, MongoDB Query, Supabase, Datasets, Connections Manager, Swarm Agents, Benchmark Dashboard, Claude Plugin, and Agent Demos.

React was chosen over Vue.js and Angular because of its larger ecosystem, flexible compositional model, and strong compatibility with Recharts, Chart.js, and Lucide React — all of which are used in this project.

3.1.2 React Router v6

React Router v6 is the standard client-side routing library for React applications. It maps URL paths to React components, enabling navigation between pages without server requests. Version 6 introduces a simplified route configuration model and nested routes.

Database Assistant uses React Router to manage navigation between six pages: Login, Register, Dashboard, PostgreSQL Query, MongoDB Query, and Datasets. The AppLayout component wraps all authenticated pages with the sidebar navigation, and React Router's <Outlet> component renders the active page inside the layout. Protected routes redirect unauthenticated users to the login page automatically.

3.1.3 Axios

Axios is a promise-based HTTP client for JavaScript that runs in both the browser and Node.js environments. It provides a clean API for making HTTP requests, supports request and response interceptors, and automatically serializes request bodies to JSON.

Database Assistant uses Axios as the sole HTTP client for all communication with the FastAPI backend. A global Axios instance is configured in `api.js` with the backend base URL and a 60-second timeout for long-running AI queries. Two interceptors are configured: a request interceptor that injects the JWT bearer token from `localStorage` into every request header, and a response interceptor that catches 401 Unauthorized responses and automatically redirects the user to the login page. This centralized configuration ensures that authentication is handled consistently across all API calls without requiring per-call token management.

3.1.4 Recharts

Recharts is a React-native charting library built on top of D3.js and SVG. It provides composable chart components including `BarChart`, `LineChart`, `PieChart`, and their associated sub-components such as `XAxis`, `YAxis`, `Tooltip`, and `Legend`. Being React-native means chart components integrate naturally with React state and props, allowing charts to update reactively when data changes.

Database Assistant uses Recharts in the `ChartPanel` tab of `ResultsPanel` to render interactive data visualizations from query results. The `ChartPanel` supports four chart types: bar, horizontal bar, line, and pie. Users can switch between chart types using toggle buttons, select the X-axis column from a dropdown, and toggle individual Y-axis columns using checkboxes. All charts are wrapped in Recharts' `ResponsiveContainer` to automatically resize with the browser window.

3.1.5 Chart.js 4.4.0

Chart.js is a JavaScript charting library that renders charts on HTML5 Canvas elements. It is framework-agnostic and is used directly via the DOM API without React wrappers. Version 4.4.0 provides a comprehensive set of chart types including bar, line, pie, doughnut, radar, and scatter, with support for animations, tooltips, and plugin extensions.

Database Assistant uses Chart.js specifically within the Generative UI template engine. Each of the five pre-built HTML templates includes Chart.js loaded from CDN and uses it to render interactive charts with animated entrance effects, chart type switchers, and responsive layouts. Chart.js was chosen for the templates because it operates independently of React the templates are self-contained HTML pages rendered inside iframes, where React is not available.

3.1.6 Lucide React

Lucide React is a library of over 1,000 open-source SVG icons packaged as React components. Each icon is a pure React component that accepts standard props such as `size`, `color`, and `strokeWidth`, making icons easy to style and integrate consistently across the application.

Database Assistant uses Lucide React throughout the interface for navigation icons, tab icons in ResultsPanel, button icons, and status indicators. Using a consistent icon library ensures visual coherence across all pages and reduces the need for custom SVG assets.

3.1.7 AuthContext and JWT Token Management

AuthContext is a custom React context implemented using React's createContext and useContext hooks. It manages the global authentication state of the application, including the JWT token, user profile data, login and logout functions, and a displayName derived from the user's full name or email prefix.

On application load, AuthContext reads the JWT token from localStorage and calls the /auth/me endpoint to validate the token and fetch the user's profile. If the token is invalid or expired, the user is redirected to the login page. The displayName is computed as full_name.trim() if present, otherwise email.split('@')[0], providing a human-readable name for the dashboard greeting and sidebar display.

3.1.8 Papa.parse and SheetJS

Papa.parse is a browser-compatible CSV parsing library that converts raw CSV text into structured JavaScript arrays with automatic header detection and type inference. SheetJS (xlsx) is a library for reading Excel workbook files in the browser, supporting .xlsx and .xls formats.

DB Assistant uses both libraries in the Datasets page to enable file-based querying without a database server. When a user uploads a CSV or Excel file, the appropriate library parses the file client-side into a JSON array of row objects. The parsed data is transmitted to the /my-datasets/benchmark-run endpoint, where it is loaded into an in-memory SQLite database via Pandas. The user can then query the uploaded data using the same natural language interface as a live database connection. Column headers are previewed immediately after upload, and the system auto-suggests relevant questions based on detected column names and data types.

3.2 Middle-Tier Technologies

The middle tier is implemented as a FastAPI Python application that exposes a REST API, orchestrates the agent pipeline, and integrates with the Google Gemini API. The following technologies form the complete middle-tier stack.

3.2.1 FastAPI

FastAPI is a modern, high-performance Python web framework for building REST APIs. It is built on top of Starlette for the web layer and Pydantic for data validation. FastAPI's key features include native async/await support, automatic OpenAPI documentation generation from type annotations, and Pydantic-based request validation that catches malformed inputs before they reach business logic.

Database Assistant uses FastAPI as its backend web framework. All API endpoints are defined using FastAPI's decorator-based routing, with Pydantic models defining the expected request and response schemas. FastAPI's dependency injection system (`Depends()`) is used to enforce JWT authentication on all protected routes a `get_current_user` dependency is declared once and reused across all six routers.

FastAPI was chosen over Flask and Django REST Framework because of its superior performance, native async support compatible with Motor's async MongoDB driver, and automatic API documentation that was valuable during development and testing.

3.2.2 Google Gemini 2.0 Flash API

Google Gemini 2.5 Flash is a large language model developed by Google DeepMind, optimized for low-latency, high-throughput production deployments. It is accessed via the `google-generativeai` Python SDK and supports a one-million-token context window, enabling schema-rich prompts for databases with many tables and columns.

DB Assistant uses Gemini 2.5 Flash in five distinct contexts within the agent pipeline:

NL-to-SQL generation (L1 Basic Agent): A schema-aware prompt is constructed containing table names, column names, and data types. Temperature is set to 0.0 for maximum SQL determinism.

ReAct loop reasoning (L2 ReAct Agent): A two-step prompting pattern is used per attempt a Thought prompt asking the model to reason about the question and any previous errors, followed by an Action prompt requesting the corrected SQL. Up to three attempts are made, with the full error history provided as context on each retry.

Swarm agent coordination (L3 Swarm): Four specialized agent calls execute concurrently the Schema Agent reads database structure, the SQL Agent runs its own internal ReAct loop, the Safety Agent evaluates the generated SQL for security risks, and the Insight Agent produces a plain-English business interpretation of the results.

MongoDB aggregation pipeline generation: A collection schema prompt instructs the model to return a valid MongoDB aggregation pipeline as a JSON array, supporting `$match`, `$group`, `$sort`, `$project`, and `$lookup` stages.

AI Functions (`ai.generate`, `ai.if`, `ai.rank`): Row-level insight generation, boolean evaluation per row, and global result ranking are each handled through targeted prompt patterns with temperature set to 0.2.

Gemini 2.5 Flash was chosen because it achieved the best SQL generation accuracy in internal testing, offers response times of three to five seconds suitable for interactive use, and integrates natively with the Google Cloud infrastructure already used for deployment.

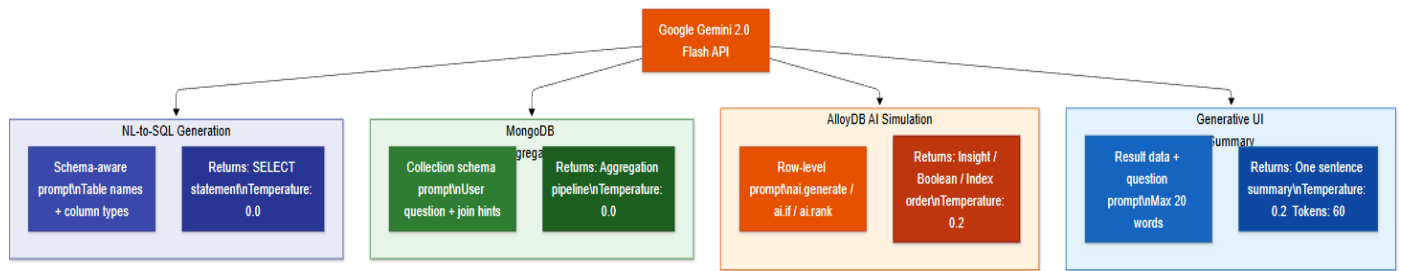


Figure 3.1: Google Gemini 2.0 Flash API Four Call Contexts.

3.2.3 Psycopg2

Psycopg2 is the most widely used PostgreSQL adapter for Python, implementing the Python DB-API 2.0 specification. Its RealDictCursor cursor factory returns query results as Python dictionaries with column names as keys, directly compatible with FastAPI's JSON serialization.

Database Assistant uses Psycopg2 for all PostgreSQL connectivity, including the internal system database and user-provided external databases. All connections specify an 8-second connection timeout to prevent hanging on unreachable databases.

3.2.4 mysql-connector-python

mysql-connector-python is the official MySQL adapter for Python, providing a DB-API 2.0 compliant interface for connecting to MySQL databases, executing queries, and fetching results.

DB Assistant uses mysql-connector-python for all MySQL natural language query processing. The MySQL pipeline adapts the NL-to-SQL and ReAct agent logic from the PostgreSQL implementation, with key differences: schema discovery uses SHOW TABLES followed by DESCRIBE tablename rather than information_schema queries, and the prompt instructs Gemini to generate MySQL-dialect SQL including backtick-quoted identifiers and MySQL-specific functions.

3.2.5 Motor

Motor is an asynchronous Python driver for MongoDB built on top of PyMongo. It provides a non-blocking interface compatible with Python's asyncio event loop, allowing MongoDB operations to be awaited without blocking the FastAPI server's event loop.

Database Assistant uses Motor for all MongoDB connectivity. The Motor client is initialized once at application startup and reused across all requests. Motor was chosen over synchronous PyMongo because FastAPI's async architecture requires non-blocking I/O using a synchronous driver would block the event loop and degrade throughput under concurrent load.

3.2.6 Pandas

Pandas is a Python data manipulation library providing DataFrame and Series data structures for working with tabular data. It supports reading from CSV, Excel, JSON, and SQL sources, with powerful data type inference and column transformation capabilities.

Database Assistant uses Pandas for CSV and Excel file ingestion. When a user uploads a file, Pandas reads it into a DataFrame, infers the data type of each column, and dynamically generates a CREATE TABLE SQL statement with appropriate PostgreSQL data types. Pandas also sanitizes column names by replacing special characters and spaces with underscores to ensure SQL compatibility.

3.2.7 Fernet Encryption

Fernet is a symmetric encryption scheme from the Python cryptography library using AES-128-CBC for encryption and HMAC-SHA256 for authentication, providing both confidentiality and integrity guarantees.

Database Assistant uses Fernet to encrypt all database connection credentials before storage. The encryption key is stored as an environment variable. Credentials are decrypted only at query execution time, held in memory for the duration of the request, and never logged or returned to the client. Fernet was chosen because it provides authenticated encryption a token cannot be tampered with without detection, protecting against credential substitution attacks.

3.2.8 SQL Guard

SQL Guard is a custom-built Python safety module that validates all Gemini-generated SQL before execution. It performs the following steps in sequence: strips markdown code fences, normalizes whitespace, removes string literals to prevent false positives, scans for dangerous keywords (DROP, INSERT, UPDATE, DELETE, TRUNCATE, ALTER, CREATE, GRANT, REVOKE) using word-boundary regex, verifies the statement begins with SELECT, and appends LIMIT 100 if no limit is present. Any query failing these checks raises an HTTP 422 error before reaching the database.

3.2.9 Python-Jose and Bcrypt

Python-jose is a Python implementation of the JSON Object Signing and Encryption (JOSE) standards, providing JWT creation and validation. Bcrypt is a password hashing algorithm incorporating a configurable cost factor and built-in salt generation.

DB Assistant uses python-jose to issue JWT tokens on login with a 24-hour expiry using the HS256 algorithm, and to validate tokens on every protected API request via the `get_current_user` dependency. Bcrypt is used to hash user passwords before storage — plaintext passwords are never stored or logged. The bcrypt cost factor is set to a level balancing security with acceptable login latency for an interactive web application.

3.2.10 asyncio (Swarm Parallelism)

Python's asyncio library provides infrastructure for writing concurrent code using the `async/await` syntax. `asyncio.gather()` runs multiple coroutines concurrently within a single event loop thread, allowing parallel I/O-bound operations without the overhead of OS threads.

DB Assistant uses `asyncio.gather()` in the L3 Swarm Agent to run the SQL Agent, Safety Agent, and Insight Agent in parallel after the Schema Agent completes. This parallel execution means the total swarm response time approximates the longest individual agent call rather than the sum of all calls, delivering meaningfully faster results than sequential execution. The Schema Agent runs first because all other agents depend on its output; the remaining three agents have no interdependency and are therefore safe to parallelize.

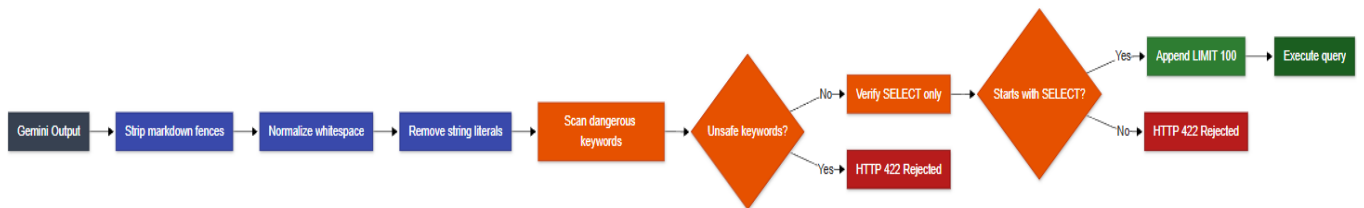


Figure 3.2: SQL Guard Validation Pipeline

3.3 Data-Tier Technologies

3.3.1 PostgreSQL 15

PostgreSQL is an open-source object-relational database management system known for its standards compliance, extensibility, and robustness. Database Assistant uses PostgreSQL 15 in two roles: as the internal system database storing user accounts, encrypted credentials, and uploaded datasets, and as the primary query target for natural language queries. The internal instance runs on Docker port 5433 to avoid conflicts with locally installed PostgreSQL instances.

3.3.2 MySQL

MySQL is an open-source relational database management system widely used in web applications. DB Assistant supports MySQL as an optional query target, with a dedicated pipeline that handles MySQL-specific connection management, schema discovery, and SQL dialect generation via the `mysql-connector-python` driver.

3.3.3 MongoDB

MongoDB is a document-oriented NoSQL database storing data as BSON documents organized into collections, with a flexible schema model and a rich aggregation pipeline framework for complex data transformations and multi-collection joins.

DB Assistant supports MongoDB as an optional query target. The NL-to-MongoDB pipeline generates aggregation pipelines using Gemini, supporting \$match, \$group, \$sort, \$project, and \$lookup stages. Because MongoDB is schemaless, schema inference is performed by sampling documents at runtime rather than querying a static schema catalog.

3.3.4 SQLite (In-Memory, via Pandas)

SQLite is a lightweight, serverless, file-based relational database engine embedded in the Python standard library. When used in in-memory mode (:memory:), it creates a transient database that exists only for the duration of the process.

DB Assistant uses SQLite in-memory mode for the Datasets feature, where users upload CSV or Excel files and query them with natural language. Pandas reads the uploaded file and loads it into an in-memory SQLite instance, Gemini generates standard SQL against the inferred schema, and results are returned to the user without any persistent storage. This approach allows dataset querying to work without a running database server and without data persistence between sessions.

3.4 Infrastructure and Deployment Technologies

3.4.1 Google Cloud Run

Google Cloud Run is a fully managed serverless container platform that runs stateless Docker containers on demand. It handles provisioning, scaling (including scale-to-zero), load balancing, and TLS termination automatically.

DB Assistant deploys both the FastAPI backend and the React/nginx frontend as separate Cloud Run services. The backend service is configured with 2 vCPUs, 2 GB of memory, and a 300-second request timeout to accommodate long-running swarm agent queries. The frontend service uses 512 MB of memory and serves the static React bundle via nginx. Both services are deployed in the us-central1 region. Cloud Run was chosen because it eliminates server management overhead, scales to zero when not in use (reducing cost), and integrates directly with Cloud SQL and Google Artifact Registry.

3.4.2 Google Cloud SQL (PostgreSQL 15)

Cloud SQL is a fully managed relational database service on Google Cloud Platform. DB Assistant uses a Cloud SQL PostgreSQL 15 instance (db-f1-micro tier) as the internal system database, accessed from the Cloud Run backend via a Unix socket (Cloud SQL connector) to avoid exposing the database on a public IP address.

3.4.3 Google Artifact Registry and Cloud Build

Google Artifact Registry is a fully managed container image registry. Cloud Build is a managed continuous integration service that builds Docker images in the cloud from source code.

DB Assistant uses Cloud Build to build Docker images for both the backend and frontend services directly from the GitHub repository, and Artifact Registry to store and version those images. Deployment is performed via a single `gcloud run deploy` command referencing the latest image tag. This pipeline eliminates the need for a local Docker daemon during deployment and ensures reproducible builds.

3.4.4 Docker and Docker Compose

Docker is a containerization platform that packages an application and its dependencies into a portable image. Docker Compose is a tool for defining and running multi-container applications using a declarative YAML configuration file.

DB Assistant uses Docker to containerize both the FastAPI backend and the React/nginx frontend for consistent behavior across development and production environments. Docker Compose is used in local development to manage the PostgreSQL 15 container with port mapping (5433:5432) and a persistent named volume, allowing developers to run the full stack locally without installing PostgreSQL directly.

3.4.5 Open Plugin Protocol and Claude Code Plugin

The Open Plugin Protocol is a specification for exposing AI-accessible tools through a standardized manifest and OpenAPI-compatible endpoint structure. DB Assistant implements this protocol to allow any compatible AI assistant to use DB Assistant as an external database querying tool.

The plugin manifest served at `/.well-known/ai-plugin.json` declares the plugin's name, description, authentication type, and API schema URL. The `/plugin/query` endpoint accepts a natural language question, database type, and connection string, and returns the generated SQL, a row count, a result preview, and a shareable result URL. Shareable result pages at `/results/{id}` are publicly accessible HTML pages requiring no login.

DB Assistant also implements a Claude Code plugin using SKILL.md files, providing three skills: `/db-assistant:query` for natural language database querying from the terminal, `/db-assistant:connect` for testing connections and listing tables, and `/db-assistant:benchmark` for retrieving the latest KDD Cup benchmark results. These skills use Claude Code's Bash tool to execute curl commands against the backend API, enabling database access without opening a browser.

Chapter 4. Project Design

4.1 Client Design

The client tier is designed as a React 18 Single Page Application. This section describes the key design decisions for each major UI component, focusing on interaction patterns, component structure, and user experience decisions.

4.1.1 Application Layout and Navigation Design

The application uses a persistent sidebar layout implemented in the `AppLayout` component. The sidebar contains the application brand, navigation links to all eleven pages, the authenticated user's display name and avatar, and a logout button. The main content area renders the active page through React Router's `Outlet` component.

Navigation is organized across eleven destinations: Dashboard, PostgreSQL, MongoDB, MySQL, Supabase, Datasets, Connections, Swarm Agents, Benchmark, Claude Plugin, and Agent Demos. The active route is visually highlighted in the sidebar so users always know their current location. The sidebar supports a collapsed state triggered by a toggle button at the top, maximizing screen real estate on smaller displays.

Authentication state is managed globally by the `AuthContext` provider, which wraps the entire application. Any page that requires authentication checks the context for a valid JWT token. If the token is missing or expired, the user is immediately redirected to the login page without rendering the protected page content.

4.1.2 Login and Registration Page Design

The Login and Registration pages use a consistent split-panel layout. The screen width and presents a deep purple gradient background with the application tagline, and a three-item feature list highlighting Natural Language to SQL, Multi-Agent AI, and Multi-Database Support. This panel serves as the value proposition for new users encountering the system for the first time. The right panel contains the authentication form on a light background, keeping the form visually distinct and focused.

The Login form includes email and password inputs with a visibility toggle on the password field, a Sign In button, and a "Don't have an account? Create one" link. Form validation provides immediate inline feedback empty fields, invalid email formats, and incorrect credentials each produce a specific error message without a page reload. The submit button shows a loading state during the authentication request. On successful login the user is redirected to the Dashboard.

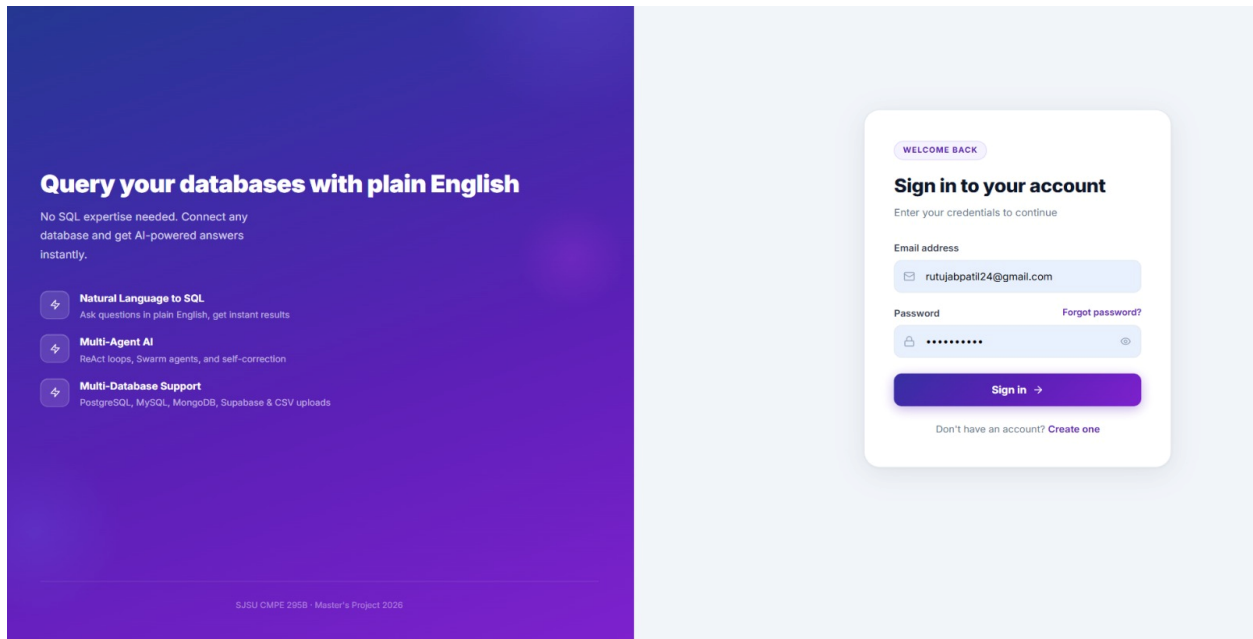


Figure 4.1: Login page

The Registration page follows the same split-panel layout with the left panel updated to highlight the 82% Benchmark Accuracy alongside Natural Language to SQL and Multi-Agent AI. The right panel contains a "Create an account" form with email, password, and confirm-password fields. A minimum password length of six characters is enforced with inline feedback, and the confirm-password field catches mismatches before submission.

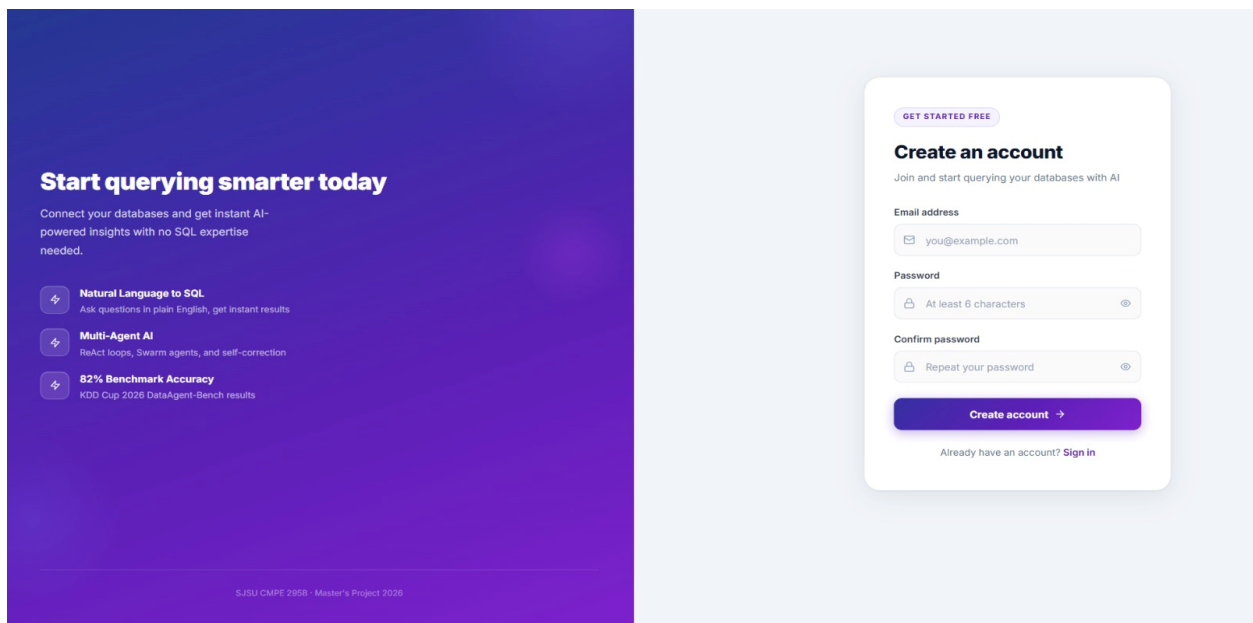


Figure 4.2: Registration page

4.1.3 Dashboard Page Design

The Dashboard is the first page displayed after login. Its primary purpose is to orient the user, display the current system status, and provide quick navigation to all features.

The top section displays a personalized greeting using the authenticated user's name. Below this, a statistics row shows the number of PostgreSQL connections saved, MongoDB connections, and the current AI pipeline status. These figures are fetched live from the backend on each page load.

If the user has no saved connections, a prominent prompt guides them to the Connections page to add their first database. For users with existing connections, a grid of action cards provides direct one-click navigation to each query feature. Each card displays the feature name, a short description, and the database type it supports.

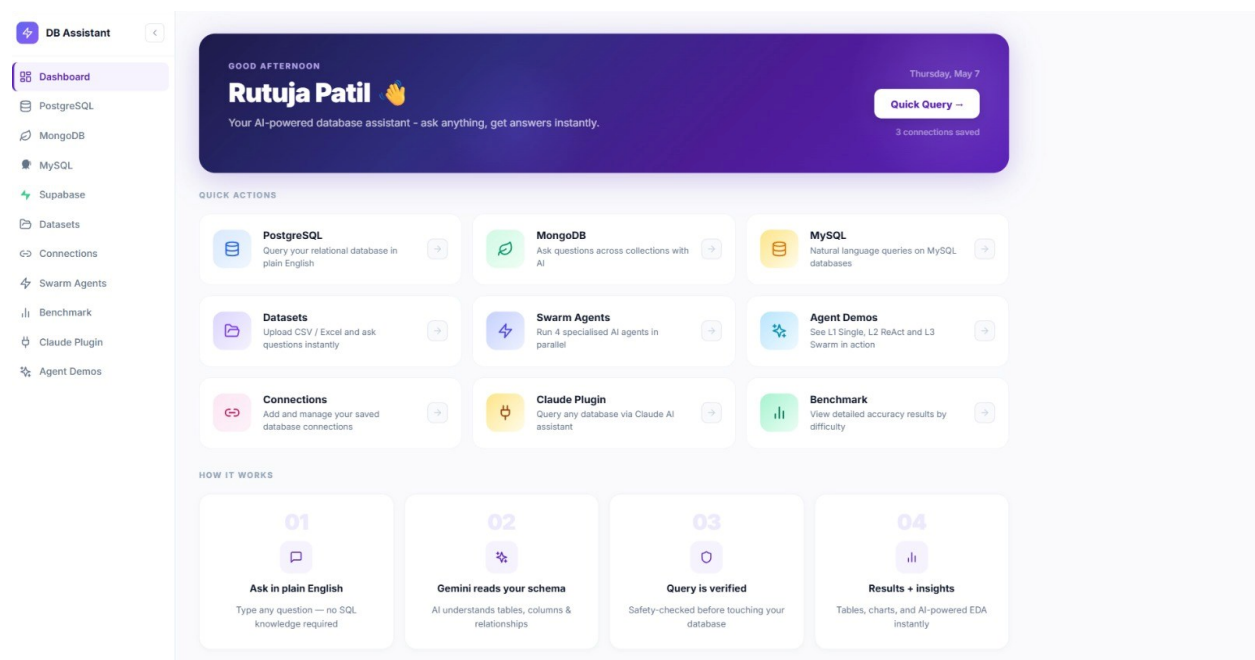


Figure 4.3: Dashboard showing personalized greeting, live connection statistics, and quick action navigation cards

4.1.4 Chat-Style Query Interface Design

The PostgreSQL, MySQL, MongoDB, and Supabase query pages share a common chat-style query interface. The page is divided into a fixed left panel for database controls and a scrolling right panel for the conversation history.

The left panel contains a connection selector dropdown pre-populated from the user's saved connections. Below the selector, a suggestions panel displays five clickable question chips derived from the connected database's schema — for example "Show total revenue by region", "Top 5 customers by spend", "Monthly order trend". Clicking a chip auto-fills the query input.

The right panel maintains a scrollable conversation history. Each user question appears as a message bubble on the right side. Each AI-generated result appears as a full-width result card on the left side, containing the complete ResultsPanel component described in Section 4.1.5. A metadata bar at the top of each result card shows row count, execution time in milliseconds, the tables queried, and a ReAct attempt badge ("✓ ReAct — 1 attempt") confirming which agent level was used. This history persists for the duration of the browser session, allowing users to compare results from multiple queries by scrolling.

The Charts tab within each result card renders a four-panel collage view simultaneously displaying Bar, Line, Pie, and Treemap charts for the same query result. X-axis and Y-axis column selectors above the collage allow the user to change which columns drive the visualization. A Summary Statistics panel below the charts shows total, average, max, and min values for each numeric column.

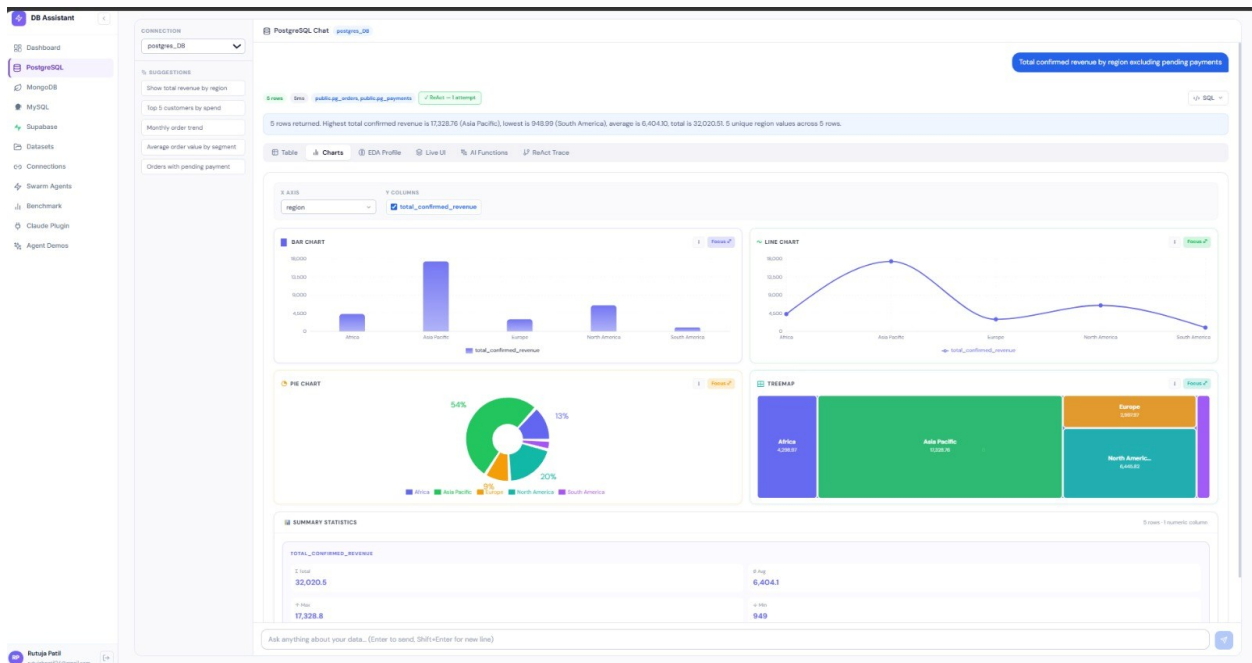


Figure 4.4: PostgreSQL query interface showing the Charts tab in collage view bar chart, line chart, pie chart, and treemap all rendered simultaneously

The input bar at the bottom of the right panel accepts natural language questions. Pressing Enter submits the query; pressing Shift+Enter inserts a newline for multi-line input. While a query is processing, the input is disabled to prevent duplicate submissions. The result card metadata bar shows execution time once the result arrives.

The ReAct Trace tab within each result card provides full transparency into the agent's reasoning process. It shows a summary banner ("✓ Succeeded on first attempt ReAct loop: Reasoning +

Acting + Observing") along with the attempt count. Each attempt is expanded into three labeled sections: THINK (the model's reasoning about the question and available tables), ACT (the SQL query generated), and OBSERVE (the execution result or error message). A legend at the bottom of the tab explains each step: Think - Agent reasons about the question; Act - Agent generates and runs SQL; Observe - Agent evaluates the result.

Figure 4.6: ReAct Trace tab showing the full Think–Act–Observe reasoning trace for a successful single-attempt query.

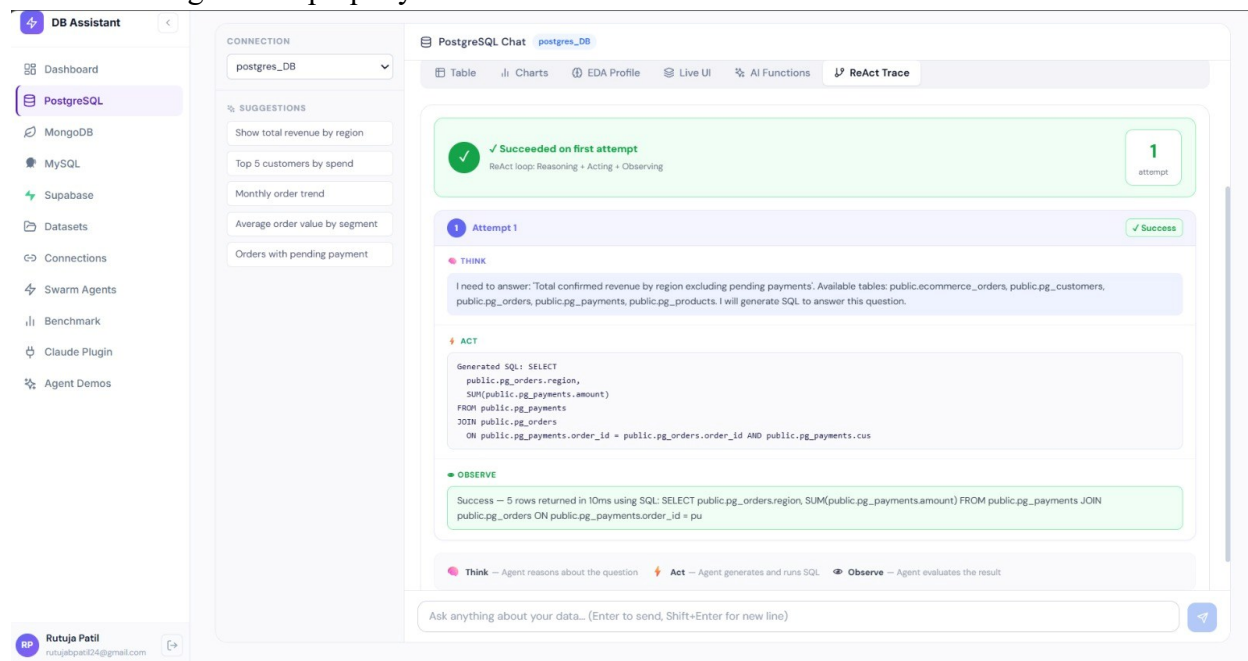


Figure 4.5: ReAct Trace tab showing the full Think–Act–Observe reasoning trace for a successful single-attempt query.

4.1.5 ResultsPanel Six-Tab Design

The ResultsPanel is the central results display component rendered inside each result card in the conversation history. It presents the same query result through six complementary views organized as tabs: Table, Charts, EDA Profile, Live UI, AI Functions, and ReAct Trace.

The **Table tab** displays a scrollable data grid with sortable columns. A metadata bar shows row count, column count, and query execution time. A disclosure button reveals the generated SQL in a code block with a copy button.

The **Charts tab** provides an interactive visualization panel rendering all applicable chart types simultaneously in a collage view bar, line, pie, and treemap as shown in Figure 4.5. X-axis and Y-axis selectors above the collage allow column remapping. A Summary Statistics panel below reports total, average, max, and min per numeric column.

The **EDA Profile tab** displays an automatically generated exploratory data analysis report including a Gemini-generated headline insight, key findings, a data quality score, and per-column statistical profiles covering data type, null percentage, min/max for numeric columns, and top five most frequent values for categorical columns.

The **Live UI tab** renders a Generative UI template selected for the query result, displayed inside a sandboxed iframe. The template selection algorithm analyzes column types and names to choose the most appropriate template for geographic data (columns containing "region") it renders an interactive Regional Revenue Dashboard with filter dropdowns, a map panel, and a revenue data table. A Regenerate button allows a fresh template rendering.

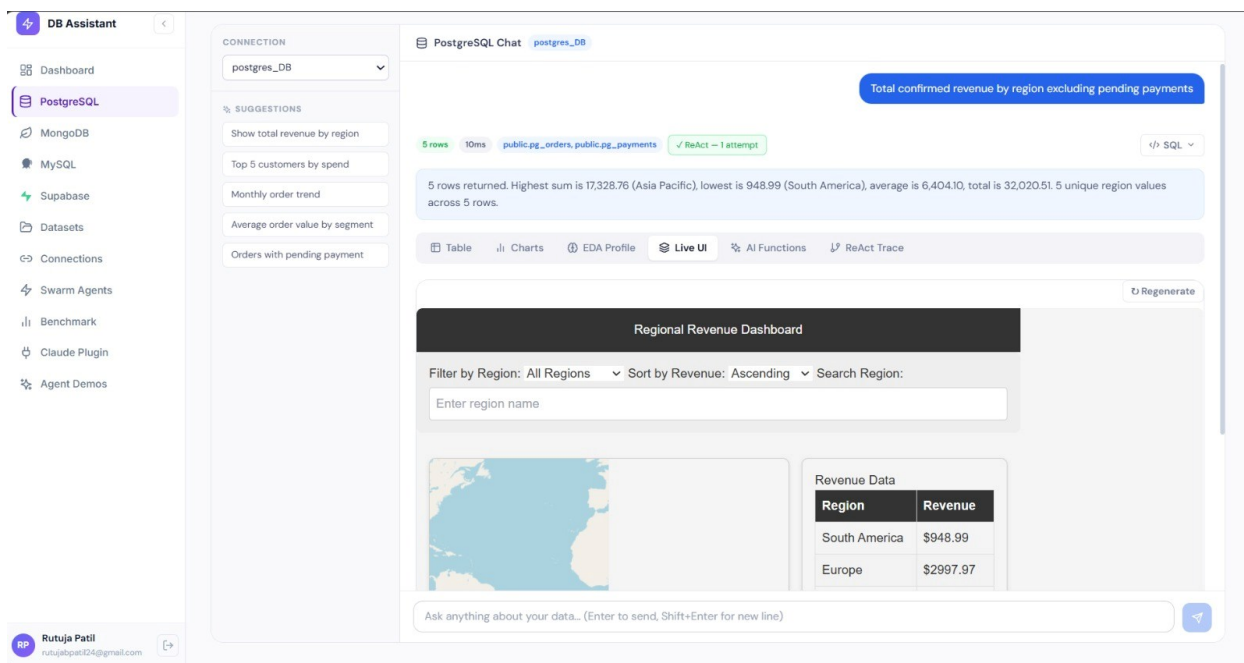


Figure 4.6: Live UI tab showing the auto-generated Regional Revenue Dashboard template for a geographic revenue query, including filter-by-region and sort-by-revenue dropdowns, a world map panel, and an interactive revenue data table inside a sandboxed iframe.

4.1.6 AI Functions Panel Design

The AiFunctionsPanel is organized as a three-function interface corresponding to the three simulated AlloyDB AI functions: ai.generate, ai.if, and ai.rank.

Each function is accessible via a clearly labeled button at the top of the panel. A description beneath the buttons explains what the selected function does. Below the description, a text input accepts the user's natural language prompt for the function. A row limit warning is displayed when the dataset exceeds the processing limit for the selected function (100 rows for ai.generate, 200 for ai.if, 50 for ai.rank).

For `ai.generate`, an additional input field allows the user to specify the name of the new column that will be appended to the result. This column name appears as a header in the inline results table after processing.

After the function runs, a summary line reports the outcome for example, "5 rows enriched", "2 rows matched, 3 filtered out", or "5 rows reranked". An inline results table renders the processed data directly within the panel. For `ai.generate`, the new AI-generated column is visually highlighted to draw attention to the added insights. A reset button restores the original unprocessed data in the Table tab.

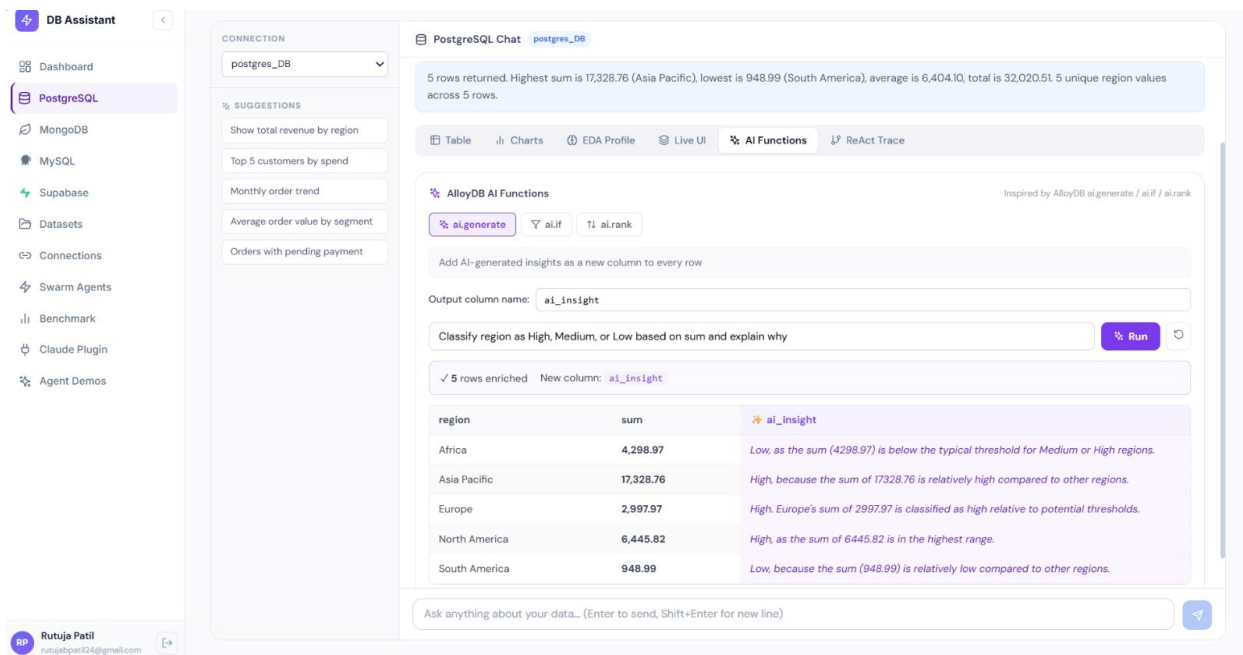


Figure 4.7: AI Functions panel showing `ai.generate` in action. The prompt "Classify region as High, Medium, or Low based on sum and explain why" has enriched 5 rows with a new `ai_insight` column (highlighted in purple) containing Gemini-generated per-row classifications

4.1.7 Swarm Agents Panel Design

The Swarm Agents page provides a dedicated interface for executing the L3 parallel swarm pipeline. The left panel contains three configuration controls: a Database Type selector (PostgreSQL, Supabase, MySQL, MongoDB, Datasets), a Connection dropdown populated from saved connections, and a Parallel Agents selector offering 2, 3, or 4 agents. A sample questions panel below shows context-aware suggestions for the selected connection.

The right panel is the primary execution area. At the top, a summary bar shows Agents Run, Succeeded count, Total Time, and Subtasks count once a query completes. Below the summary, a Swarm Intelligence Summary banner displays a one-sentence headline insight generated by the Insight Agent for example, "Total Revenue Reaches \$35,255.29 from 15 Unique Customers, with Strong April Recovery Following March Dip."

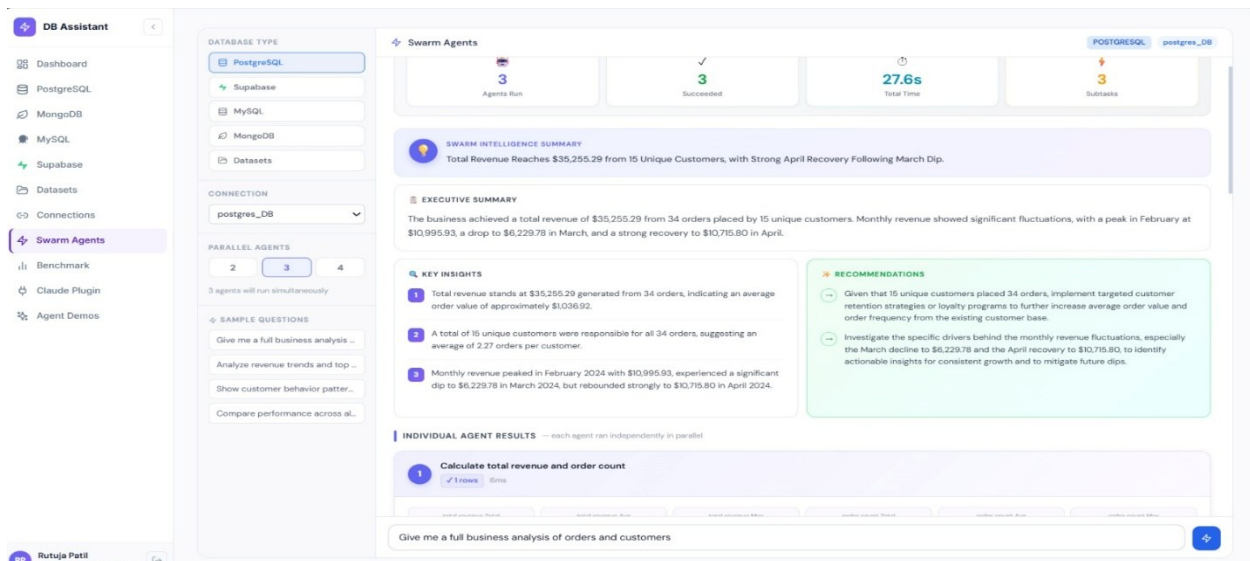


Figure 4.8: Swarm Agents page showing a completed business analysis query.

An Executive Summary paragraph provides a two-to-three sentence plain English business narrative. A Key Insights panel lists numbered findings extracted from the data. A Recommendations panel on the right side of the page provides actionable business recommendations derived from the analysis. Below these panels, an Individual Agent Results section shows each sub-agent's query, row count, and execution time individually, confirming that each agent ran independently in parallel.

The query input bar at the bottom accepts the natural language question. The page also supports an Agent Demos view (Figure 4.10) showing the four individual agents Schema Reader, SQL Generator, Safety Validator, and Insight Synthesiser each with their role description and a Done status badge, along with a pipeline diagram at the top showing the flow: Schema Reader → SQL Generator → Safety Validator → Insight Synthesiser → Combined answer.

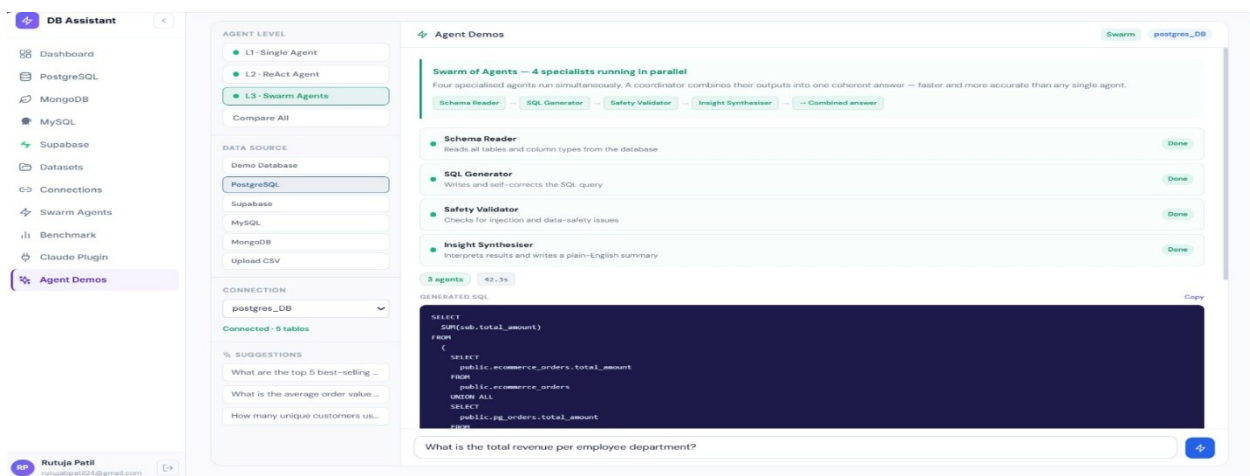


Figure 4.9: Agent Demos page showing the L3 Swarm tab with the four-agent pipeline diagram

4.1.8 Benchmark Dashboard Design

The Benchmark Dashboard displays the KDD Cup 2026 DataAgent-Bench Phase 1 evaluation results in a dark-themed professional interface.

The top section features a full-width dark hero banner with the "DATAAGENT-BENCH · PHASE 1" label, the title "Benchmark Results", the course identifier "SJSU CMPE 295B · 50 tasks across 4 difficulty levels", and an animated circular accuracy gauge on the right showing 82% in green.

Below the hero, four summary cards display the key metrics: Tasks Passed (41 of 50 total) in green, Self-Corrected (9, via ReAct loop) in orange, Avg Response (10.4s per task) in blue, and Hard Accuracy (80%, 8 of 10) in orange.

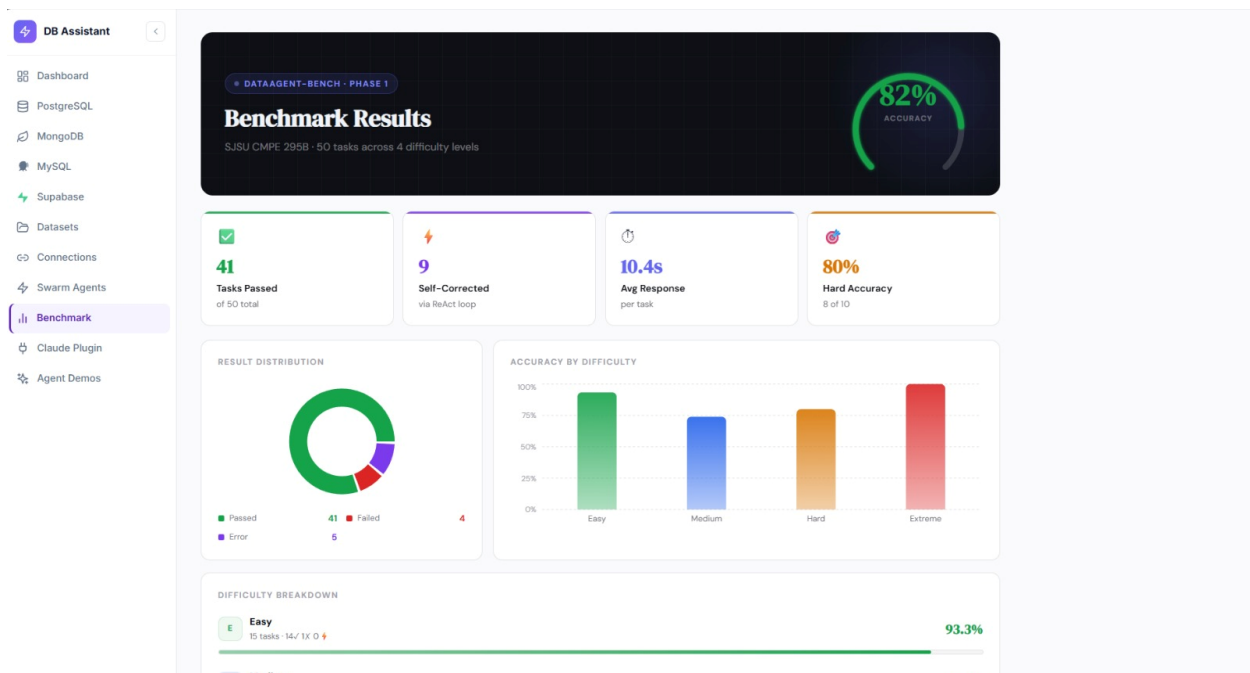


Figure 4.10: Benchmark Dashboard

4.1.9 Connections Manager Design

The Connections Manager page allows users to save, organize, and test database connections for all supported database types. The page header shows "My Connections — Manage your saved database connections" with an "Add Connection" button in the top-right corner.

The saved connections list groups connections by database type under colored section headers: PostgreSQL (blue), MongoDB (green), and MySQL (gray). Each saved connection entry shows the user-defined connection name, a database type badge, and the host/database path. A trash icon on each entry allows deletion.

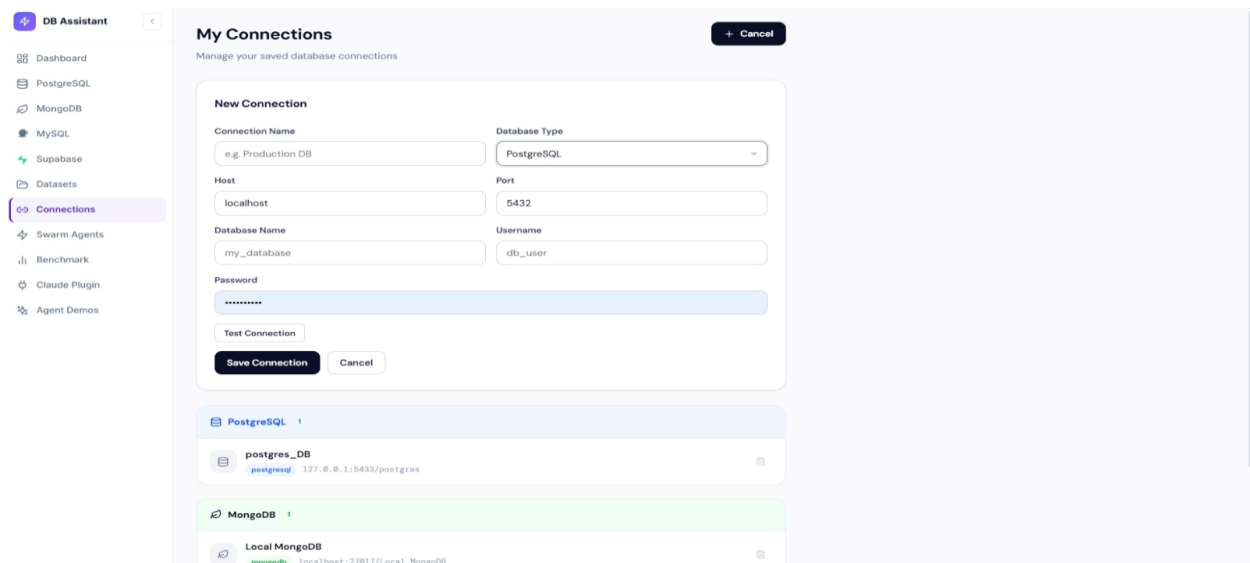


Figure 4.11: Connections Manager showing three saved connections

4.1.10 Agent Demos Page Design (L1/L2/L3)

The Agent Demos page implements the Awesome Data Agents L1/L2/L3 Hello World framework, allowing the same question to run through all three agent levels for direct side-by-side comparison. The left panel provides three configuration sections. The Agent Level section shows three radio-style selectors: L1 - Single Agent, L2 - ReAct Agent, L3 - Swarm Agents, plus a "Compare All" button. The Data Source section provides buttons for Demo Database, PostgreSQL, Supabase, MySQL, MongoDB, and Upload CSV. The Connection section shows a dropdown of saved connections and a "Connected · N tables" status indicator when a connection is active. The Suggestions panel below shows schema-aware sample questions.

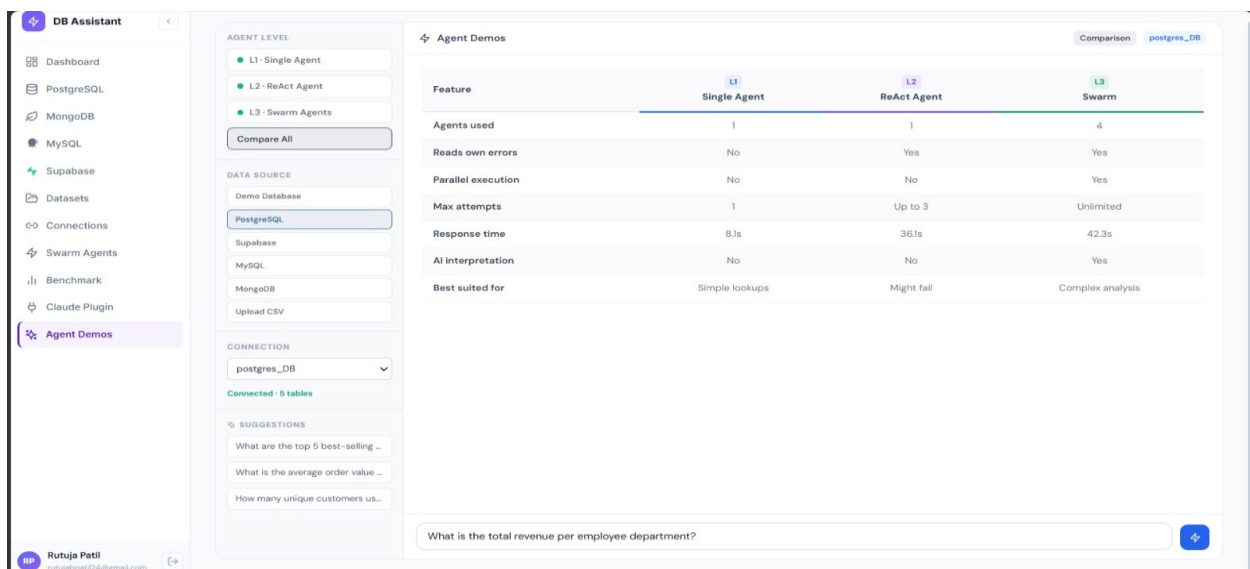


Figure 4.12: Agent Demos page

The right panel shows results based on the selected Agent Level. For L3 Swarm Agents, the panel displays the four-agent pipeline diagram with labeled boxes (Schema Reader → SQL Generator → Safety Validator → Insight Synthesiser → Combined answer), followed by individual agent status cards each showing their role description and a Done badge with execution time, and finally the generated SQL and results below.

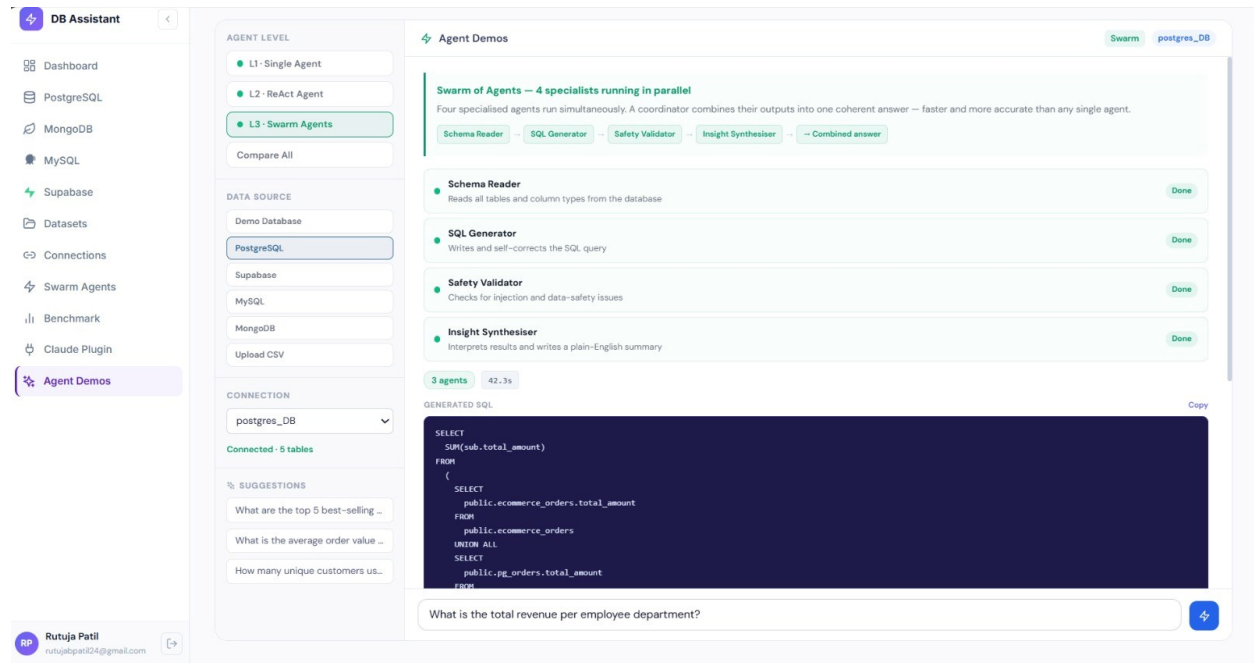


Figure 4.13: Agent Demos page with L3 Swarm Agents selected

4.1.11 Claude Plugin Page Design

The Claude Plugin page is a public route accessible without login, serving as both a live demo interface and integration documentation for the Open Plugin Protocol.

The page header features a dark hero banner with an "OPEN CLAUDE PLUGIN · LIVE" badge, the title "DB Assistant Plugin", the tagline "Query any database with natural language no SQL knowledge required", and a green "Live -API STATUS" indicator confirming the backend is reachable.

Three tabs organize the page content. The **Demo Database tab** shows six clickable sample question chips ("How many employees are in each department?", "What is the average salary by department?", "Show top 5 orders by revenue", etc.) above a query input field with a "Run query" button. Below the input, the demo database table list is shown (Demo DB · employees · departments · orders · sales_performance). After a query runs, a results section shows row count, column count, query status, the generated SQL in a dark code block with a Copy SQL button, and a results table.

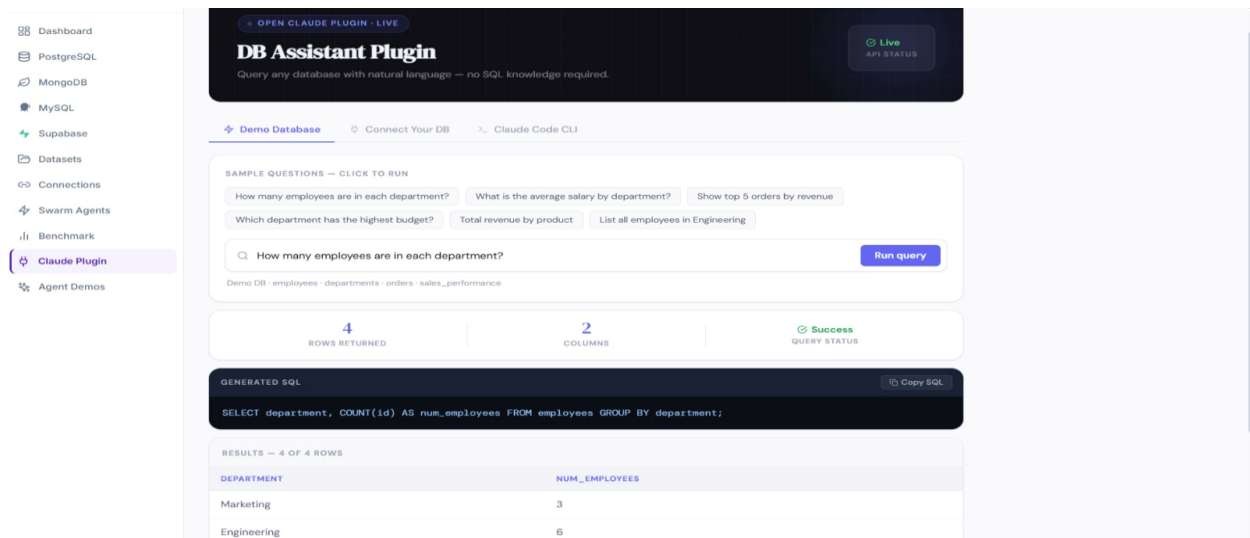


Figure 4.14: Claude Plugin page

The Connect Your DB tab provides a connection string input and test button. The Claude Code CLI tab provides copy-paste installation instructions for running the plugin in a local Claude Code environment.

4.1.12 Claude Code Plugin (Terminal) Design

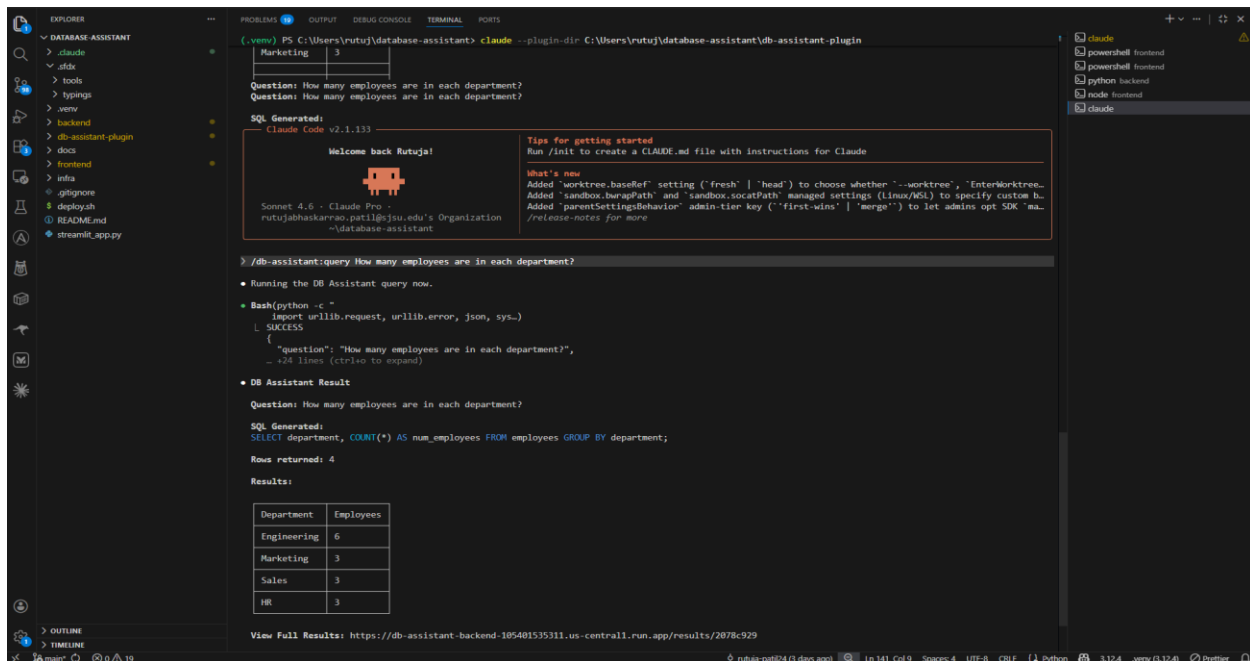


Figure 4.15: Claude Code terminal showing the /db-assistant:query skill executing

The Claude Code plugin allows developers to query databases directly from the VS Code terminal without opening a browser. The plugin is invoked with the command:

claude --plugin-dir C:\Users\rutuj\database-assistant\db-assistant-plugin

Once loaded, three skills are available. The /db-assistant:query skill accepts a natural language question, calls the backend's /plugin/query endpoint using a Python urllib request executed via Claude Code's Bash tool, and returns a formatted result in the terminal showing the question, generated SQL, row count, a results table, and a clickable "View Full Results" URL linking to the shareable result page on the backend.

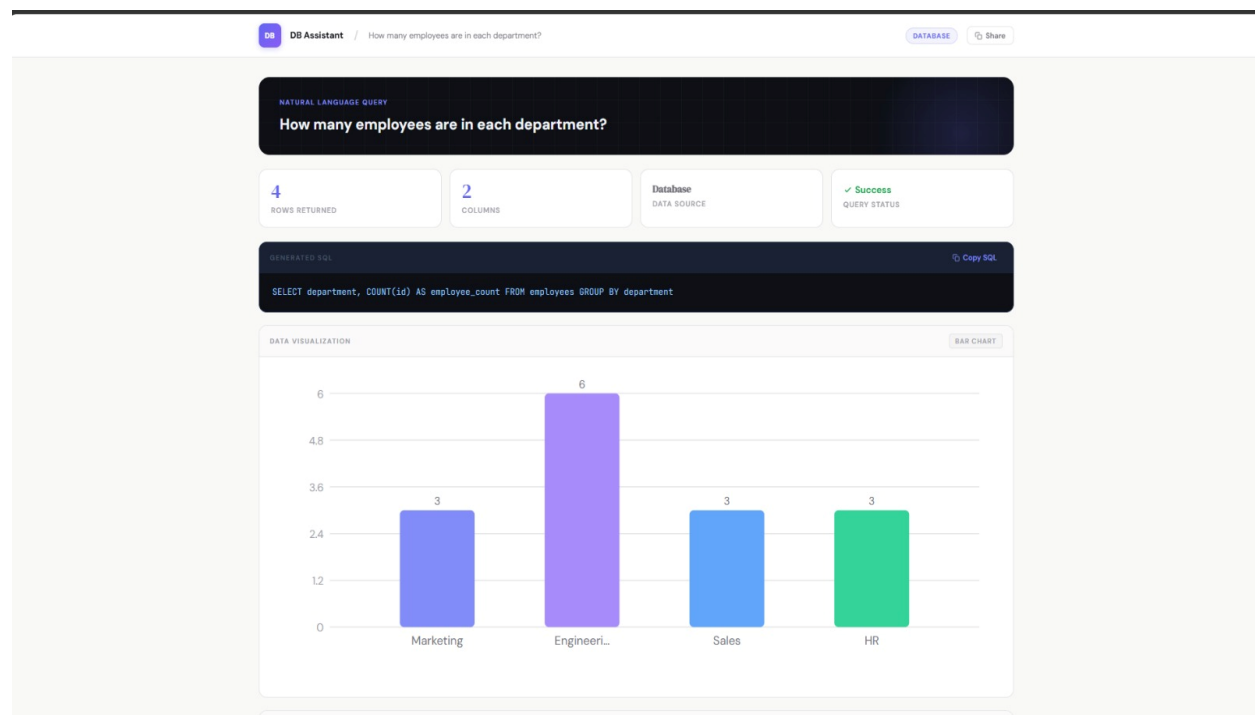


Figure 4.16: Shareable result page

4.1.13 Datasets Page Design

The Datasets page enables users to query CSV and Excel files without a database server. The left panel shows a "DATASETS (3)" header with an "Upload CSV / Excel" button and a list of uploaded tables in this example, customers (4 cols), orders (4 cols), and payments (4 cols) each with a delete icon. A Suggestions panel below the table list shows five auto-generated question chips based on the detected column names.

The right panel uses the same chat-style conversation interface as the database query pages. The header shows "Dataset Chat · 3 tables". A query result card shows the row count, execution time, the queried table name, and a SQL toggle. The ResultsPanel tabs - Table, Charts, EDA Profile, Live UI, AI Functions are all available for uploaded dataset results. The Charts tab renders the same four-panel collage (Bar, Line, Pie, Treemap) with X-axis and Y-axis selectors.

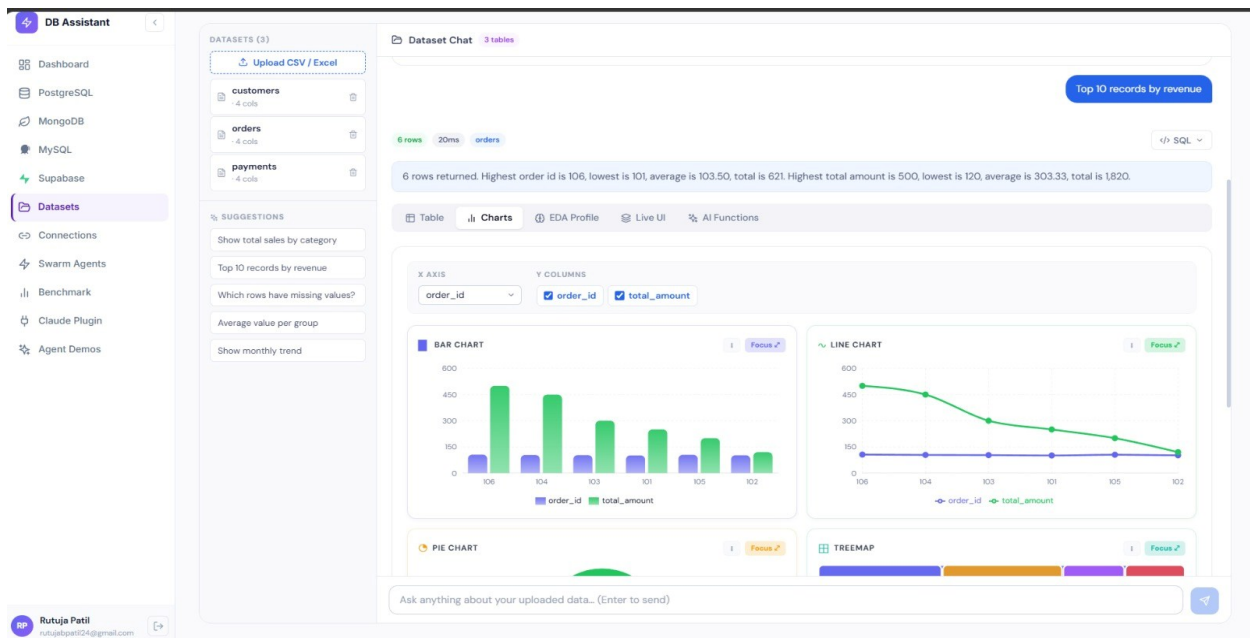


Figure 4.17: Datasets page showing three uploaded tables

4.2 Middle-Tier Design

4.2.1 REST API Route Architecture

The backend API is organized into ten router modules, each mounted at a distinct URL prefix and registered in main.py using FastAPI's `include_router()` method. This structure ensures each router can be developed, tested, and modified independently without affecting other parts of the system.

Router	Prefix	Auth Required	Key Endpoints
Auth	/auth	No (login/register)	register, login, me
PostgreSQL	/pg	Yes	nl-query-auto, list-tables, describe-table
MySQL	/mysql	Yes	nl-query, nl-query-auto, test-connection
MongoDB	/mongo	Yes	nl-query, nl-query-auto, list-collections
Swarm	/swarm	Yes	pg-query
Datasets	/my-datasets	Yes	benchmark-run, nl-query-auto
Connections	/connections	Yes	CRUD + test
Benchmark	/benchmark	Yes	results

Plugin	/plugin	No	query, connect, demo-tables
AI Functions	/ai	Yes	generate, if, rank

Table 4.1: API route architecture

Authentication is enforced on all routes except /auth/login, /auth/register, and the public plugin endpoints using FastAPI's Depends(get_current_user) dependency. The dependency decodes the JWT, validates the signature and expiry, and retrieves the current user record. If validation fails, FastAPI returns a 401 Unauthorized response before the route handler executes.

4.2.2 Three-Level Agent Architecture

DB Assistant implements three distinct agent levels following the Awesome Data Agents L1/L2/L3 framework. Each level is a complete, independently callable pipeline.

Level 1 - Basic Agent is a linear five-stage pipeline: Schema Inspector reads the database structure via information_schema, Prompt Builder constructs the schema-aware Gemini prompt, Gemini 2.5 Flash generates the SQL, SQL Guard validates the query for safety, and the SQL Executor runs the query and returns results. The L1 agent makes a single Gemini call with no retry logic. It is designed for speed and simplicity and completes in approximately 8 seconds as shown in the Agent Demos comparison table (Figure 4.14).

Level 2 - ReAct Agent wraps the L1 pipeline in a Think-Act-Observe loop with up to three attempts. On each attempt, a Thought prompt asks Gemini to reason about the question and any previous error messages. An Action prompt requests the SQL query based on that reasoning. If the SQL fails, the error message is fed back into the next Thought prompt. This targeted feedback is what distinguishes ReAct from blind retry the model reasons specifically about why the previous query failed. The ReAct Trace tab (Figure 4.6) exposes the full reasoning chain to the user. L2 completes in approximately 36 seconds on queries requiring self-correction.

Level 3 - Swarm Agent runs four specialized agents with the Schema Agent executing first, followed by the SQL Agent, Safety Agent, and Insight Agent executing in parallel via asyncio.gather(). A Coordinator combines outputs into a single response including an Executive Summary, Key Insights, and Recommendations as shown in Figure 4.9. L3 completes in approximately 42 seconds, with the additional time reflecting the richer multi-agent output.

4.2.3 Swarm Agent Design

The L3 Swarm Agent consists of four specialized agents coordinated by a Coordinator. Each agent has a single defined responsibility.

The Schema Reader connects to the target database, queries the information_schema, and builds a structured representation of all tables, columns, data types, and relationships. It runs first because all other agents depend on its output.

The SQL Generator receives the schema from the Coordinator and runs its own internal ReAct loop to generate, validate, and execute SQL. It is architecturally identical to the L2 ReAct Agent, meaning the swarm benefits from L2-level self-correction within each swarm execution.

The Safety Validator evaluates the proposed query for SQL injection patterns, destructive operations, and unauthorized access attempts. It produces a structured safety report included in the Coordinator's response.

The Insight Synthesiser receives the question, schema, and actual query results from the SQL Generator. It uses Gemini to generate an Executive Summary, numbered Key Insights, and actionable Recommendations in plain English, as shown in Figure 4.9.

The Coordinator fires the SQL Generator, Safety Validator, and Insight Synthesiser in parallel using `asyncio.gather()` after the Schema Reader completes, then assembles all outputs into a single structured response.

4.2.4 SQL Guard Safety Module Design

SQL Guard is a stateless pure validation function that accepts a raw string and returns either a validated SQL string or raises an exception. Its design follows the defense-in-depth principle multiple independent checks are applied in sequence so that no single bypass technique can allow a dangerous query to execute.

String literals are removed from the query text before the keyword scan, preventing an attacker from embedding dangerous keywords inside string values. Word-boundary regular expressions (`\bDROP\b` rather than simple substring matching) prevent false positives such as blocking a column named `dropout_rate`. The validator additionally verifies the statement begins with `SELECT` and appends `LIMIT 100` if no `LIMIT` clause is present. SQL Guard is applied uniformly across all three agent levels and all four supported database types.

4.2.5 Schema-Aware Prompt Engineering Design

Schema-aware prompting is the core technique that enables accurate NL-to-SQL generation on unfamiliar databases. The Schema Agent constructs a structured prompt block for each selected table using the following format:

```
Table: public.sales_orders
Columns:
- order_id (integer)
- customer_name (character varying)
- region (character varying)
- total_amount (numeric)
- order_date (date)
- status (character varying)
```

When multiple tables are selected, the schema blocks are concatenated. The complete schema prompt is prepended to the user's question before the Gemini API call. The system prompt instructs Gemini to use only the tables and columns listed in the schema, never invent column names, return only a SELECT statement with no markdown or explanation, and include a LIMIT clause if the query could return a large number of rows.

For MongoDB, a similar approach is used. The Schema Agent fetches a sample document from each selected collection and constructs a field list showing field names and inferred types. This collection schema is included in the Gemini prompt alongside instructions for generating a valid aggregation pipeline.

4.2.6 Open Plugin Protocol Design

The plugin system implements the Open Plugin Protocol specification. The manifest at `./well-known/ai-plugin.json` declares the plugin name, model-facing description, authentication type (none), and a reference to the backend's OpenAPI schema.

The `/plugin/query` endpoint accepts a natural language question, database type, and connection string, and returns the generated SQL, row count, a result preview, and a publicly accessible shareable result URL at `/results/{id}`. Shareable result pages are publicly accessible dark-themed HTML pages requiring no login, as shown in Figure 4.18.

The Claude Code plugin uses a SKILL.md-based architecture. Three skill files define the `/db-assistant:query`, `/db-assistant:connect`, and `/db-assistant:benchmark` skills. Each skill file contains frontmatter metadata specifying allowed tools (Bash only). At runtime, Claude Code reads the skill file and uses its Bash tool to execute Python `urllib` requests against the backend API, returning formatted results in the terminal as shown in Figure 4.17.

4.2.7 Benchmark Agent Design

The benchmark evaluation system is a standalone Python script (`benchmark_agent.py`) that runs independently of the FastAPI server. It loads task files from the KDD Cup 2026 DataAgent-Bench Phase 1 dataset, calls the backend's NL-to-SQL endpoint for each task, compares predictions against the gold standard, and records each outcome.

The agent supports two execution modes: full mode processes all 50 tasks sequentially, while retry mode reprocesses only tasks recorded as failed or errored from an existing results file. The accuracy improvement from 46% to 82% was achieved through targeted prompt engineering guided by failure analysis. Common failure patterns included float/integer type mismatches in ID joins (fixed with CAST patterns), non-English domain values such as Czech banking terms VYBER and VYDAJ (fixed with explicit prompt hints), time format mismatches (fixed with LIKE '1:54%' patterns), and missing context from supplementary markdown files (fixed by automatically injecting `knowledge_md` content into the schema prompt).

4.2.8 AlloyDB AI Function Simulation Design

The three AI function simulations mirror the behavioral contract of the corresponding AlloyDB native functions using Gemini 2.5 Flash as the underlying engine.

ai.generate implements a row-level map pattern. Each input row is independently sent to Gemini with the user's prompt as context and a structured JSON result field is extracted and appended as a new column. Row independence means a failure on one row does not affect others — failed rows receive an "Analysis unavailable" placeholder. The result is shown in Figure 4.8, where Asia Pacific at \$17,328.76 is classified as High and South America at \$948.99 as Low with a per-row explanation.

ai.if implements a row-level filter pattern. Each row is independently evaluated by Gemini against a boolean condition. The endpoint is fail-open: rows where the Gemini call fails are included in the output to prevent unintended data loss.

ai.rank implements a global sort of pattern. All rows are sent to Gemini in a single call alongside ranking criterion. Gemini returns a JSON array of integer indices specifying row order, and the endpoint reconstructs the array accordingly. Any rows omitted from Gemini's response are appended at the end as a safety net.

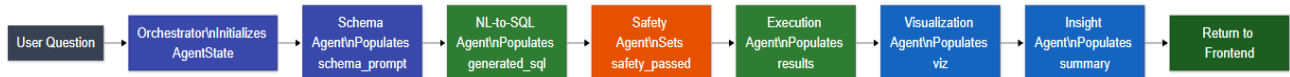


Figure 4.1: AgentState pipeline showing fields populated at each stage of the seven-agent sequence.

4.2.9 Generative UI Template Selection Algorithm

The template selection algorithm operates in two stages implemented as separate functions to maintain separation of concerns. The first stage, `_analyze_data()`, performs a structural analysis of the query result. It iterates over all columns and classifies each column's data type by attempting to parse sample values as floats. Columns where more than 70% of sample values parse as floats are classified as numeric. Columns whose names contain time-related keywords (date, time, year, month, timestamp, quarter, week) are classified as date/time columns. All remaining columns are classified as categorical or text. The second stage, `_infer_app_hint()`, applies a priority-ordered set of detection rules to the column classification output. The rules execute in the following priority order: time-series detection (presence of date/time columns), geographic detection (presence of location-related column names), network detection (presence of edge/node column names), text detection (presence of review or comment columns), ranking detection (presence of rank or score columns), and multi-numeric detection (three or more numeric columns). The first rule that

matches determines the template. If no rule matches, the general Table template is selected as the fallback.

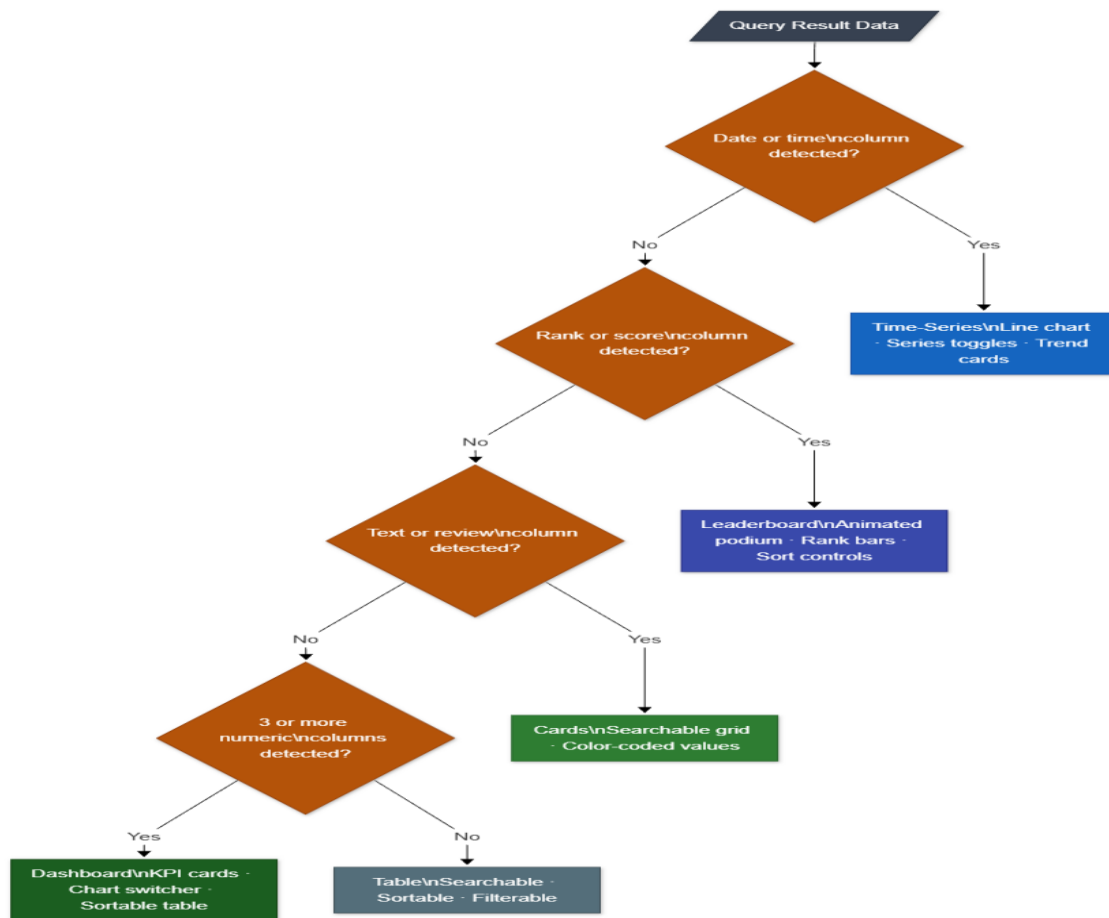


Figure 4.2: Template Selection Decision Tree

4.3 Data-Tier Design

4.3.1 User Authentication and Profile Schema

The users table stores all data required for authentication and user profile display. Passwords are hashed using bcrypt with a work factor of 12 before storage. Plain text passwords are never stored or logged.

```

CREATE TABLE users (
  id          SERIAL PRIMARY KEY,
  email       VARCHAR(255) UNIQUE NOT NULL,
  full_name   VARCHAR(255),
  password_hash VARCHAR(255) NOT NULL,
  created_at  TIMESTAMP DEFAULT NOW(),
  last_login_at TIMESTAMP
);
  
```

The `full_name` field is optional. If present, it is used as the `displayName` shown in the sidebar and dashboard greeting. If absent, the email prefix (the portion before the `@` symbol) is used as the display name.

4.3.2 Connection Credentials Schema

The `user_connections` table stores database connection details for each user. The `password_enc` field stores the Fernet-encrypted credential as binary data. The encryption key is never stored in the database.

```
CREATE TABLE user_connections (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,  
  name VARCHAR(255) NOT NULL,  
  db_type VARCHAR(50) NOT NULL,  
  host VARCHAR(255),  
  port INTEGER,  
  dbname VARCHAR(255),  
  db_username VARCHAR(255),  
  password_enc BYTEA,  
  is_default BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

The `ON DELETE CASCADE` constraint ensures that all connection records are automatically deleted when a user account is deleted. The `is_default` flag identifies the connection pre-selected when the user opens a query page, reducing the number of steps required to begin querying.

4.3.3 Internal Dataset Storage Schema

Uploaded files are stored as PostgreSQL tables within a per-user schema named `user_{id}` (e.g., `user_42`). Per-user schemas provide data isolation users cannot access each other's dataset tables even if they know the table name. A metadata table in the public schema tracks all uploaded files.

```
CREATE TABLE dataset_metadata (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,  
  table_name VARCHAR(255) NOT NULL,  
  original_filename VARCHAR(255),  
  row_count INTEGER,  
  col_count INTEGER,  
  file_size_kb NUMERIC,  
  uploaded_at TIMESTAMP DEFAULT NOW()  
);
```


The actual dataset data is stored in dynamically created tables such as `user_42.sales_data`. Column names are sanitized by Pandas to replace spaces and special characters with underscores. Data types are inferred and mapped to the most appropriate PostgreSQL type: boolean, bigint, double precision, timestamp, or text.

4.3.4 Query History Schema

The `query_history` table records every natural language query executed by each user. This data supports future features including query history browsing, saved queries, and failure pattern analysis.

```
CREATE TABLE query_history (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,  
  connection_id INTEGER REFERENCES user_connections(id),  
  question TEXT NOT NULL,  
  generated_sql TEXT,  
  execution_time_ms INTEGER,  
  row_count INTEGER,  
  status VARCHAR(50),  
  error_message TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

The `status` field records one of three values: success for queries that executed and returned results, rejected for queries blocked by SQL Guard, and failed for queries that encountered a database execution error. The `error_message` field stores the specific rejection or failure reason, enabling analysis of common failure patterns and future improvements to the NL-to-SQL prompt design.

The `query_history` table records every natural language query executed by each user, supporting future features including query history browsing and failure pattern analysis.

Column	Type	Description
id	SERIAL PRIMARY KEY	Query record identifier
user_id	INTEGER FK → users.id	Executing user
question	TEXT	Original natural language question

generated_sql	TEXT	SQL produced by the agent
db_type	VARCHAR	postgres / mysql / mongodb
status	VARCHAR	success / rejected / failed
error_message	TEXT	Rejection or failure reason if applicable
execution_time_ms	INTEGER	End-to-end response time
created_at	TIMESTAMP	Query timestamp

The status field records one of three values: success for queries that executed and returned results, rejected for queries blocked by SQL Guard, and failed for queries that encountered a database execution error. The error_message field stores the specific reason, enabling analysis of common failure patterns and informing future improvements to the NL-to-SQL prompt design.

Chapter 5. Project Implementation

5.1 Client Implementation

This section describes the implementation of the React 18 frontend, focusing on the components and patterns that are most significant to the system's functionality.

5.1.1 AuthContext: JWT Token Management and User State

AuthContext is implemented as a React context provider that wraps the entire application. It is the single source of truth for authentication state across all pages and components.

On application mount, AuthContext reads the JWT token from localStorage and immediately calls the /auth/me endpoint to validate the token and retrieve the user's current profile. This validation step ensures that expired or revoked tokens are caught before the user attempts to perform any action. If validation succeeds, the user object is stored in context state and made available to all child components. If validation fails, the token is removed from localStorage and the user is redirected to the login page.

The displayName field is derived from the API response using the following priority order: the full_name field if it is non-empty, otherwise the email prefix (the portion of the email address before the @ symbol), otherwise the fallback string "User". This derived value is used in the sidebar avatar, the dashboard greeting, and the page title.

The login function calls /auth/login with OAuth2 form-encoded credentials, stores the returned JWT token in localStorage, and calls /auth/me to populate the user state. The logout function removes the token from localStorage, clears the user state, and redirects to the login page.

```
// AuthContext core pattern
const login = async (email, password) => {
  const res = await authAPI.login(email, password);
  localStorage.setItem('token', res.data.access_token);
  const me = await authAPI.me();
  const displayName = me.data.full_name?.trim()
    || me.data.email.split('@')[0]
    || 'User';
  setUser({ ...me.data, displayName });
};
```

5.1.2 Dashboard Page Implementation

The Dashboard page fetches its data from two sources on mount. A call to `/connections` retrieves the count of saved connections displayed in the hero banner ("3 connections saved"). The quick action grid is rendered statically all nine cards (PostgreSQL, MongoDB, MySQL, Datasets, Swarm Agents, Agent Demos, Connections, Claude Plugin, Benchmark) are always shown regardless of connection state, with each card navigating to its respective route on click.

The conditional "Connect your first database" prompt is rendered when the connections count is zero. The time-aware greeting ("Good Morning" / "Good Afternoon" / "Good Evening") is derived client-side from the current hour using JavaScript's Date object. The four-step How It Works panel is static content rendered below the quick action grid

5.1.3 PostgreSQL Chat Query Page Implementation

The PostgreSQL query page is implemented as a stateful React component with three primary state variables: history (the conversation array), loading (boolean), and connections (the list of saved connections).

When the component mounts, it fetches the user's saved PostgreSQL connections from `/auth/connections` and fetches the available tables for the default connection from `/pg/list-tables`. Tables are displayed as checkboxes in the left panel with select-all and deselect-all buttons.

The query submission handler implements multi-question support through the `splitQuestions()` function, which detects compound questions using connective phrases such as "and also", "additionally", and "as well as". Each detected sub-question is submitted as an independent API call. The handler uses `Promise.all()` to execute all sub-question calls in parallel, then appends each result to the history array in order.

The chat history is rendered by mapping over the history array. User messages render as right-aligned bubble components containing the question text. Result entries render as full-width ResultsPanel components receiving the result data, columns, sql, react_trace, and summary as props. The conversation scrolls automatically to the newest result after each query using a `useRef` and `useEffect` scroll hook.

The ReAct Trace tab within each ResultsPanel card renders the react_trace object from the response. The trace object contains an attempts count, a self_corrected boolean, and parallel arrays of thoughts, actions, and observations. Each attempt index is rendered as an expandable section with three labeled blocks: THINK (thought text), ACT (the SQL generated on that attempt), and OBSERVE (the execution result or error message). A summary banner at the top shows either "✓ Succeeded on first attempt" or "⚠ Self-Corrected — N attempts" depending on the self_corrected flag.

```

const handleSubmit = async (question) => {
  const questions = splitQuestions(question);
  setLoading(true);
  const results = await Promise.all(
    questions.map(q => pgAPI.nlQuery({
      pg_uri, question: q, tables: selectedTables
    }))
  );
  setHistory(prev => [
    ...prev,
    { type: 'user', content: question },
    ...results.map((r, i) => ({
      type: 'result',
      question: questions[i],
      content: r.data
    })))
  ];
  setLoading(false);
};

```

5.1.4 MySQL Query Page Implementation

The MySQL query page shares the same component structure as the PostgreSQL page. The key difference is the backend endpoint: queries are submitted to `/mysql/nl-query-auto`. The connection selector accepts MySQL connection strings in the format `mysql://username:password@host:3306/database`. Schema discovery uses `/mysql/list-tables`, which on the backend calls `SHOW TABLES` followed by `DESCRIBE tablename` rather than querying `information_schema`. The ReAct trace, ResultsPanel tabs, and conversation history behave identically to the PostgreSQL page.

5.1.5 MongoDB Multi-Collection Query Page Implementation

The MongoDB query page implements a three-step connection flow: the user selects a saved connection, selects a database from the available databases, then selects one or more collections to query.

Database and collection lists are fetched lazily: databases are fetched when a connection is selected, collections are fetched when a database is selected. This avoids unnecessary API calls for connections or databases the user does not intend to use.

The query submission logic automatically selects the appropriate endpoint based on the number of selected collections. If exactly one collection is selected, the query is sent to `/mongo/nl-query`. If two or more collections are selected, the query is sent to `/mongo/nl-query-join`, which prompts Gemini to generate a pipeline with `$lookup` stages for cross-collection joins. A hint label in the left panel shows "N collections auto join enabled" when multiple collections are selected.

5.1.6 Datasets Upload and Query Page Implementation

The Datasets page implements CSV and Excel file upload using the browser's File API and a multipart form data POST request to `/my-datasets/upload`.

When a file is selected, the frontend constructs a `FormData` object containing the file and sends it to the backend. The backend returns the sanitized table name, row count, and column count. The dataset list in the left panel is refreshed automatically after a successful upload.

The query submission for datasets differs from the PostgreSQL page in one important way: the user does not select specific tables. Instead, all uploaded dataset table names are passed to the backend in the `all_table_names` field of the request body. The backend passes this complete list to Gemini, which selects the most relevant table based on the user's question. This auto-selection behavior allows users to query their datasets without needing to know or remember which table contains the relevant data.

Dataset deletion is implemented with a confirmation step the user clicks a delete button on a dataset item in the left panel, and a confirmation dialog prevents accidental deletion. On confirmation, a DELETE request is sent to `/my-datasets/{table_name}` and the dataset list is refreshed.

5.1.7 Connections Manager Implementation

The Connections Manager page renders saved connections grouped by database type under colored section headers. On mount, it fetches all connections from `/connections` and groups them client-side by the `db_type` field. PostgreSQL connections are shown under a blue header, MongoDB under green, and MySQL under gray.

The "Add Connection" button reveals a New Connection form built with controlled React inputs. Database Type selection pre-fills the Port field with the default for the selected type (5432 for PostgreSQL, 27017 for MongoDB, 3306 for MySQL). The "Test Connection" button calls `/connections/test` with the current form values and shows a green success indicator or a red error message inline. The "Save Connection" button calls POST `/connections`, which encrypts the connection string server-side before storage and returns the saved connection record. The form is hidden and the connection list refreshed on success.

5.1.8 ResultsPanel Six-Tab Implementation

The ResultsPanel component is the most complex component in the frontend. It manages its own tab state, SQL disclosure state, and aiRows state independently from the parent page component.

The six tabs Table, Charts, EDA Profile, Live UI, AI Functions, ReAct Trace are rendered as a tab bar. The active tab is controlled by a tabIndex state variable. Each tab's content is conditionally rendered based on tabIndex.

The aiRows state variable is the key mechanism for the AI Functions integration. It is initialized as null and set by the AiFunctionsPanel component via the onResult callback prop when an AI function completes. When aiRows is non-null, the Table tab renders aiRows instead of the original data prop. The columns array shown in the Table tab is also derived from aiRows when it is non-null, ensuring that new columns added by ai.generate (such as ai_insight) appear as column headers. The reset button in AiFunctionsPanel calls onResult(null), which sets aiRows back to null and restores the original data.

The Charts tab renders the collage view by calling a buildChartConfigs() utility function that analyzes the columns array and returns a list of chart type configurations. The function classifies columns as numeric if more than 70% of sampled values parse as floats, and as date/time if the column name contains keywords such as date, time, year, month, or timestamp. The collage renders all applicable chart types (Bar, Line, Pie, Treemap) simultaneously using Recharts' ResponsiveContainer, each inside a labeled card with a "Focus ↗" button that expands that chart to full width.

The Summary Statistics panel below the chart collage computes total, average, max, and min for each numeric column client-side using JavaScript array reduce operations on the data prop. These statistics are displayed in labeled metric cards, one card per numeric column.

The Live UI tab renders the Generative UI HTML template in a sandboxed iframe. The src of the iframe is set to a blob URL created from the HTML string returned by the /genui/generate endpoint. A Regenerate button calls the endpoint again and updates the blob URL, triggering an iframe reload.

The SQL disclosure drawer is toggled by a `</>` SQL button in the result metadata bar. The generated SQL is displayed in a pre-formatted code block with syntax highlighting and a Copy button implemented using the navigator.clipboard.writeText() API.

5.1.9 AiFunctionsPanel Implementation

The AiFunctionsPanel component manages its own state for the active function tab (ai.generate, ai.if, or ai.rank), the prompt input value, the output column name input (for ai.generate), the loading state, the API result, and any error messages.

When the user clicks Run, the component calls the appropriate /ai endpoint with the current data rows and the prompt. The endpoint is selected based on the active function tab. The response is stored in local state and passed to the parent ResultsPanel via the onResult callback.

For `ai.generate`, the output column name entered by the user is sent to the backend as the `column_name` parameter. The backend appends this column to each row. The inline results table renders the complete row set including the new column, with the new column header and all its cells highlighted in purple using conditional CSS class application based on whether the column name matches the output column name.

The row limit warning is computed client-side by comparing `data.length` against the limit for the selected function (100 for `ai.generate`, 200 for `ai.if`, 50 for `ai.rank`). When the limit is exceeded, a yellow warning banner is shown above the prompt input.

5.1.10 Swarm Agents Page Implementation

The Swarm Agents page is implemented as a stateful React component with state variables for `selectedDbType`, `selectedConnection`, `question`, `loading`, and `result`.

The left panel provides database type buttons (PostgreSQL, Supabase, MySQL, MongoDB, Datasets) and a Connection dropdown. The Parallel Agents selector (2, 3, or 4) is displayed below the connection selector. Sample questions are fetched from the selected connection's schema on connection change.

On submission, the component calls `POST /swarm/pg-query` with the connection string, question, and agent count. The backend runs the swarm pipeline and returns a structured response containing `sql`, `data`, `columns`, `summary` (the Swarm Intelligence Summary headline), `executive_summary`, `key_insights` (array), `recommendations` (array), `subtask_results` (array of individual agent results), `agents_run`, `succeeded`, and `total_time_ms`.

The result is rendered in three sections. The summary bar at the top shows the four metric cards (Agents Run, Succeeded, Total Time, Subtasks). The Swarm Intelligence Summary banner displays the headline insight. The Executive Summary paragraph, Key Insights numbered list, and Recommendations panel are rendered below. Each item in the `subtask_results` array is rendered as an Individual Agent Result card showing the sub-question, row count, execution time, and a collapsed results table that can be expanded.

5.1.11 Benchmark Dashboard Implementation

The Benchmark Dashboard fetches benchmark results from `GET /benchmark/results` on mount. The response is the full `benchmark_results.json` object containing a summary and a tasks array.

The 82% accuracy gauge is implemented as an SVG circle with a `stroke-dasharray` animation. On mount, a `useEffect` hook triggers a CSS animation that fills the stroke from 0% to 82% over 1.5 seconds. The four summary metric cards (Tasks Passed, Self-Corrected, Avg Response, Hard Accuracy) are rendered with their values read directly from the summary object.

The Result Distribution donut chart is implemented using `Chart.js`. The Accuracy by Difficulty bar chart is also implemented with `Chart.js` using gradient fills created via the Canvas 2D context's

createLinearGradient() method, giving each difficulty level bar a distinct color gradient from top to bottom.

The filterable task table is implemented with three client-side filter state variables: statusFilter, difficultyFilter, and searchText. The displayed tasks array is computed by filtering the full tasks array through all three active filters on each render. Each row is expandable clicking a row toggles a rowExpanded state entry for that task ID, revealing the predicted SQL, gold SQL, predicted answer, and gold answer in a sub-row below.

5.1.12 Agent Demos Page Implementation

The Agent Demos page manages state for selectedLevel (L1, L2, L3, or Compare All), selectedDataSource, selectedConnection, question, and results for each level.

When Compare All is selected and the user submits a question, the component fires three parallel API calls using Promise.all(): /plugin/query for L1, /pg/nl-query-auto with react:true for L2, and /swarm/pg-query for L3. Each call's result is stored independently in l1Result, l2Result, and l3Result state variables.

The comparison table is rendered once all three results are available. Response times are taken from the execution_time_ms fields of each result. The table rows (Agents used, Reads own errors, Parallel execution, Max attempts, Response time, AI interpretation, Best suited for) are populated with the actual measured response times from the current query alongside the static feature descriptions.

For the L3 tab, the four agent status cards (Schema Reader, SQL Generator, Safety Validator, Insight Synthesiser) are rendered with Done badges and their descriptions once the swarm result arrives. The pipeline flow diagram at the top is rendered as a flex row of labeled boxes connected by arrow characters.

5.1.13 Claude Plugin Page Implementation

The Claude Plugin page is a public React route (no auth required) with three tabs implemented as a tab state variable.

The Demo Database tab fetches available demo tables from /plugin/demo-tables on mount and displays them as a comma-separated list below the query input ("Demo DB · employees · departments · orders · sales_performance"). Six sample question chips are hardcoded and auto-fill the query input on click. Query submission calls POST /plugin/query with db_type: "demo" and no connection string. The result includes sql, row_count, preview (first five rows), and result_url. A "View Full Results" button opens the result_url in a new tab.

The Connect Your DB tab provides a connection string input and a Test Connection button that calls POST /plugin/connect. On success, the response tables_found list and suggested_questions array are displayed below the input.

The Claude Code CLI tab renders installation and usage instructions with copy-enabled code blocks for the three plugin skills.

5.2 Middle-Tier Implementation

5.2.1 Natural Language to SQL Pipeline

The NL-to-SQL pipeline is implemented in the `/pg/nl-query-auto` endpoint. The endpoint accepts the PostgreSQL URI, the user's natural language question, and a list of selected table names. It executes the following steps in sequence.

First, it establishes a connection to the user's PostgreSQL database using `Psycopg2`. Second, it fetches the schema for each selected table from `information_schema.columns`, building a list of column names and data types. Third, it constructs a structured schema prompt combining all selected table schemas. Fourth, it calls `generate_sql()` which sends the schema prompt and question to Gemini and returns a validated SQL string. Fifth, it executes the SQL against the database and collects the results. Sixth, it passes the results to the Visualization Agent and Insight Agent to generate chart specifications and a natural language summary.

```
@router.post("/nl-query-auto")
def pg_nl_query_auto(req: PgNLQueryAutoRequest):
    conn = _get_conn(req.pg_uri)
    schema_prompt = _build_schema_prompt(conn, req.tables)
    sql = generate_sql(schema_prompt, req.question)
    results = _execute_query(conn, sql)
    viz = visualization_agent.run(results, req.question)
    summary = insight_agent.run(results, req.question)
    return {
        "data": results,
        "sql": sql,
        "viz": viz,
        "summary": summary,
        "execution_time_ms": elapsed_ms,
        "tables_used": req.tables
    }
```

5.2.2 Multi-Table JOIN Query Generation

Multi-table JOIN queries are supported through schema-aware prompting. When the user selects multiple tables, the Schema Agent fetches the schema for all selected tables and constructs a combined schema prompt that includes all table definitions in a single block.

The system prompt instructs Gemini to use JOIN clauses when the question requires data from multiple tables, and to infer join conditions from column name similarities (e.g.,

orders.customer_id joining to customers.id). The generated SQL is validated by SQL Guard before execution.

For cases where the user selects tables but asks a question that only requires one table, Gemini selects the appropriate subset of tables and generates a single-table query. The tables_used field in the response reports which tables were actually referenced in the generated SQL.

5.2.3 MongoDB NL-to-Aggregation Pipeline

The MongoDB query pipeline is implemented in two endpoints: /mongo/nl-query for single-collection queries and /mongo/nl-query-join for multi-collection queries.

For single-collection queries, the system fetches a sample document from the collection, infers field names and types, and prompts Gemini to generate a MongoDB aggregation pipeline as a JSON array. The pipeline is validated to ensure it is a valid JSON array before execution.

For multi-collection queries, the prompt includes schema information for all selected collections and instructs Gemini to generate a pipeline with \$lookup stages. The system identifies the primary collection (the first selected collection) and generates lookup stages that join the remaining collections using inferred join keys.

```
prompt = f"""
Collections:
{collection_schemas}

Generate a MongoDB aggregation pipeline for:
{question}

If joining collections, use $lookup stages.
Return ONLY a valid JSON array. No explanation.
"""
```

5.2.4 Auto-Table Selection for Uploaded Datasets

The auto-table selection feature is implemented in the /my-datasets/nl-query-auto endpoint. When called, the endpoint receives the user's question and a list of all available dataset table names. It

```
selection_prompt = f"""
Available tables: {all_table_names}
User question: {question}

Which single table is most relevant to answer this question?
Return ONLY the table name. No explanation.
"""

selected_table = gemini_call(selection_prompt).strip()
```

passes this complete table list to Gemini in the system prompt, asking Gemini to identify which table is most likely to contain data relevant to the question.

Once the table is selected, the system fetches its schema and proceeds with the standard NL-to-SQL pipeline. This two-step process — table selection followed by SQL generation — ensures that Gemini has focused schema context for the SQL generation step rather than an overwhelming list of all available tables.

5.2.5 SQL Guard Implementation

SQL Guard is implemented in `sql_guard.py` as a standalone module with a single public function `ensure_safe_select()`. The implementation uses Python's `re` module for all pattern matching.

```
def ensure_safe_select(sql: str) -> str:
    # Step 1: Strip markdown fences
    sql = re.sub(r'```sql|```', '', sql, flags=re.IGNORECASE).strip()

    # Step 2: Normalize whitespace
    sql = re.sub(r'\s+', ' ', sql).strip()

    # Step 3: Remove string literals to prevent false positives
    clean = re.sub(r'"(?:'|[^\"])*"', '', sql)

    # Step 4: Scan for dangerous keywords
    banned = ['delete', 'update', 'drop', 'alter', 'truncate',
              'insert', 'create', 'grant', 'revoke']
    for keyword in banned:
        if re.search(rf'\b{keyword}\b', clean, re.IGNORECASE):
            raise SQLGuardError(f"Unsafe keyword detected: {keyword}")

    # Step 5: Verify SELECT
    if not re.search(r'\bSELECT\b', clean, re.IGNORECASE):
        raise SQLGuardError("Only SELECT statements are allowed")

    # Step 6: Inject LIMIT if missing
    if not re.search(r'\bLIMIT\b', clean, re.IGNORECASE):
        sql = sql.rstrip(';') + ' LIMIT 100'

    return sql
```

5.2.6 ai.generate Implementation

The `ai.generate` endpoint processes each row independently. For each row, it constructs a prompt combining the user's instruction with the row data serialized as JSON, calls Gemini, and parses the response as a JSON object containing a result field.

```
@router.post("/generate")
def ai_generate(req: AiGenerateRequest, user=Depends(get_current_user)):
    enriched = []
    for row in req.rows[:100]:
        row_str = json.dumps(row, default=str)
        prompt = f"{req.prompt}\n\nRow data:\n{row_str}\n\nReturn ONLY: {{{\"result\"}}}
    try:
        raw = _call_gemini(SYSTEM, prompt)
        parsed = json.loads(raw.replace('```json', '').replace('```', ''))
        ai_value = parsed.get('result', raw)
    except Exception:
        ai_value = "Analysis unavailable"
    enriched.append({**row, req.output_column: ai_value})
    return {"rows": enriched, "output_column": req.output_column,
            "total": len(enriched)}
```

5.2.7 ai.if Implementation

The ai.if endpoint evaluates each row against a boolean condition. The system prompt instructs Gemini to return only {"match": true} or {"match": false}. Rows where Gemini returns true are included in the output. The endpoint is fail-open — rows where the API call fails are included to prevent data loss.

5.2.8 ai.rank Implementation

The ai.rank endpoint sends all rows to Gemini in a single call. The prompt includes all rows serialized as a JSON array with index numbers and asks Gemini to return a JSON array of indices in the desired order. The endpoint reconstructs the row array using the returned index order, with any unranked rows appended at the end.

```
prompt = f"""
Ranking criteria: {req.criteria}
Rows: {json.dumps([{'index': i, **r} for i, r in enumerate(req.rows)])}
Return ONLY a JSON array of indices ordered best to worst.
Example: [2, 0, 4, 1, 3]
"""

raw = _call_gemini(SYSTEM, prompt)
indices = json.loads(raw)
ranked = [req.rows[i] for i in indices if 0 <= i < len(req.rows)]
```

5.2.9 Generative UI Template Engine

The Generative UI endpoint receives the query result columns and rows, runs the data profiler and app hint engine to select a template, calls Gemini once for a one-sentence summary, injects the real data as a JavaScript constant into the selected template, and returns the complete HTML string.

```
@router.post("/generate")
def generate_ui(req: GenUIRequest):
    meta      = _analyze_data(req.columns, req.rows)
    summary   = _get_summary(req.question, req.rows, req.columns)
    template  = TEMPLATES.get(meta["type"], _template_table)
    html      = template(req.question, req.columns, req.rows, meta, summary)
    return {"html": html, "template_used": meta["type"]}
```

Each template function returns a complete self-contained HTML string with the data injected as:

```
const DATA = [{"region": "Asia Pacific", "total_revenue": 17328.76}, ...];
```

This injection approach means the browser renders the visualization immediately on iframe load without any additional network requests.

5.3 Data-Tier Implementation

5.3.1 PostgreSQL Docker Setup and Schema Initialization

The PostgreSQL 15 instance is defined in `docker-compose.yml` with a persistent named volume for data storage. The container is configured to listen on host port 5433, mapped to the default PostgreSQL container port 5432.

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: database_assistant
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    ports:
      - "5433:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

The application schema is initialized on first startup using SQLAlchemy's `create_all()` method, which creates the `users`, `user_connections`, `dataset_metadata`, and `query_history` tables if they do not already exist.

5.3.2 Fernet Credential Encryption and Storage

Connection credentials are encrypted immediately after form submission and before any database write. The encryption key is loaded from the `FERNET_KEY` environment variable at application startup and stored in a module-level variable for reuse across requests.

```
from cryptography.fernet import Fernet

FERNET_KEY = os.getenv("FERNET_KEY")
fernet = Fernet(FERNET_KEY)

def encrypt_credential(value: str) -> bytes:
    return fernet.encrypt(value.encode())

def decrypt_credential(value: bytes) -> str:
    return fernet.decrypt(value).decode()
```

Credentials are decrypted only within the route handler that executes the query, held in a local variable for the duration of the request, and never assigned to any persistent object, logged, or returned in any API response.

5.3.3 CSV and Excel Ingestion with Pandas

File ingestion is implemented as a multi-step process. The uploaded file is read into memory as a byte stream, passed to `pandas.read_csv()` or `pandas.read_excel()` depending on the file extension, and the resulting DataFrame is processed before storage.

Column names are sanitized by converting to lowercase, replacing spaces and special characters with underscores, and prepending `c_` to any column name that begins with a digit. Data types are inferred using the following mapping:

```
def _infer_pg_type(series: pd.Series) -> str:
    if pd.api.types.is_bool_dtype(series):           return "boolean"
    if pd.api.types.is_integer_dtype(series):         return "bigint"
    if pd.api.types.is_float_dtype(series):           return "double precision"
    if pd.api.types.is_datetime64_any_dtype(series): return "timestamp"
    return "text"
```

The DataFrame is then inserted into the user's schema using a dynamically generated INSERT INTO statement executed in batches of 500 rows to manage memory usage for large files.

5.3.4 Swarm Agent Implementation

The swarm pipeline is implemented in the `/swarm/pg-query` endpoint using `asyncio.gather()` for parallel execution.

```
async def swarm_query(question: str, pg_uri: str, agent_count: int):
    # Step 1: Schema Agent runs first (all others depend on it)
    schema = await schema_agent.run(pg_uri)

    # Step 2: Remaining agents run in parallel
    sql_result, safety_result, insight_hint = await asyncio.gather(
        sql_agent.run(question, schema, pg_uri),      # has own ReAct loop
        safety_agent.run(question, schema),           # validates SQL
        insight_agent.prepare(question, schema)       # prepares analysis
    )

    # Step 3: Insight agent receives actual results
    insight = await insight_agent.analyze(
        question, sql_result["data"], insight_hint
    )

    # Step 4: Coordinator assembles response
    return {
        "sql": sql_result["sql"],
        "data": sql_result["data"],
        "columns": sql_result["columns"],
        "summary": insight["headline"],
        "executive_summary": insight["executive_summary"],
        "key_insights": insight["key_insights"],
        "recommendations": insight["recommendations"],
        "safety_report": safety_result,
        "subtask_results": sql_result["subtasks"],
        "agents_run": agent_count,
        "succeeded": sql_result["succeeded"],
        "total_time_ms": insight["total_time_ms"]
    }
```

Each agent is implemented as an async class with a single `run()` method. The SQL Agent internally calls the same `react_loop()` function used by the L2 ReAct endpoint, meaning the swarm benefits from L2-level self-correction within each swarm execution. The Insight Agent makes two Gemini calls: a `prepare()` call that drafts analysis direction based on schema alone (running in parallel with the SQL Agent), and an `analyze()` call that produces the final Executive Summary, Key Insights, and Recommendations once the actual query results are available.

5.3.5 MySQL Query Pipeline Implementation

The MySQL query pipeline adapts the PostgreSQL NL-to-SQL pipeline for MySQL-specific requirements. The `mysql-connector-python` driver is used in place of `Psycopg2`. Schema discovery calls `SHOW TABLES` to list tables, then `DESCRIBE {table_name}` for each selected table to retrieve column names and types. The Gemini prompt instructs the model to generate MySQL-dialect SQL, including backtick-quoted identifiers (e.g., ``column_name``) and MySQL-specific functions. The ReAct loop and SQL Guard are applied identically to the PostgreSQL pipeline.

5.3.6 Open Plugin Protocol Implementation

The plugin system is implemented in the `/plugin` router. The `/.well-known/ai-plugin.json` endpoint returns a static JSON manifest describing the plugin to compatible AI assistants.

The `/plugin/query` endpoint accepts a `PluginQueryRequest` Pydantic model containing `question`, `db_type`, `connection_string`, and optional `tables` fields. It routes to the appropriate backend pipeline based on `db_type`: "demo" uses the Neon PostgreSQL demo database connection string, "postgres" and "supabase" use the `/pg` pipeline, "mysql" uses the `/mysql` pipeline, "mongodb" uses the `/mongo` pipeline, and "uploaded" uses the `/my-datasets` pipeline.

The result is stored in an in-memory results dictionary keyed by a UUID, which becomes the `{id}` in the shareable result URL `/results/{id}`. The `/results/{id}` endpoint returns a self-contained dark-themed HTML page with the question, SQL, row count, and a bar chart visualization rendered via `Chart.js` loaded from CDN.

The Claude Code plugin `SKILL.md` files instruct Claude Code to use its Bash tool to execute Python `urllib` requests against the backend API. The `/db-assistant:query` skill constructs a JSON payload with the `question` and `db_type` fields, POSTs to `/plugin/query`, and formats the returned `sql`, `row_count`, `preview`, and `result_url` as a terminal-readable result table.

5.3.7 Benchmark Agent Implementation

The benchmark agent (`benchmark_agent.py`) is a standalone Python script that runs outside the FastAPI server process. It loads task files from the KDD Cup 2026 DataAgent-Bench Phase 1 dataset directory, processes each task by calling the backend `/plugin/query` endpoint, compares the prediction against the gold standard answer, and writes results to `benchmark_results.json`.

The comparison logic handles several edge cases identified during evaluation. Numeric comparisons normalize both predicted and gold values by stripping currency symbols, commas, and whitespace before comparison. Float comparisons use a tolerance of 0.01 to handle rounding differences. String comparisons are case-insensitive and strip leading/trailing whitespace.

The retry mode reads an existing `benchmark_results.json` file, filters for tasks with status "failed" or "error" and reprocesses only those tasks. A `force_retry` set can be populated with specific task IDs to force reprocessing regardless of their recorded status, which was used during development to iterate on prompt engineering for consistently failing task patterns.

Key prompt engineering improvements that raised accuracy from 46% to 82% include `CAST(id AS TEXT)` patterns for float ID joins, explicit Czech banking term hints (`VYBER` for cash withdrawals, `VYDAJ` for expense transactions), `LIKE '1:54%'` patterns for time format matching, and automatic injection of `knowledge_md` supplementary context files into the schema prompt for tasks that reference domain-specific terminology.

Chapter 6. Testing and Verification

6.1 Testing Strategy and Methodology

The testing strategy for DB Assistant is organized into four levels: unit testing of individual backend modules, integration testing of API endpoints, end-to-end testing of complete user workflows, and AI quality evaluation through the KDD Cup 2026 DataAgent-Bench Phase 1 benchmark.

The primary objective is to verify that the system correctly translates natural language questions into accurate, safe database queries across all supported database types (PostgreSQL, MySQL, MongoDB) and all three agent levels (L1 Basic, L2 ReAct, L3 Swarm). A secondary objective is to verify that the security subsystem correctly rejects dangerous inputs. A third objective is to measure AI output quality against an objective external benchmark independent of the development team's own test design.

6.2 Unit Testing

Unit tests verify the behavior of individual backend functions in isolation, without requiring a running database or active Gemini API connection.

6.2.1 SQL Guard Unit Tests

The SQL Guard module is the most critically tested component in the system because it is the primary defense against dangerous query execution. Unit tests cover the following scenarios:

Dangerous keyword detection: Each blocked keyword is tested individually with exact case, uppercase, lowercase, and mixed case variations. The test verifies that DROP, INSERT, UPDATE, DELETE, TRUNCATE, ALTER, CREATE, GRANT, and REVOKE are all correctly detected and rejected with an SQLGuardError.

Word boundary matching: Tests verify that column names containing blocked keyword substrings are not incorrectly rejected. For example, a query containing the column name dropout_rate should pass validation because dropout does not match the word-boundary pattern \bDROP\b.

String literal bypass prevention: Tests verify that dangerous keywords embedded inside string literals are not incorrectly rejected. For example, WHERE name = 'DROP TABLE users' should pass validation because the keyword appears inside a quoted string value, not as a SQL command.

LIMIT injection: Tests verify that queries without a LIMIT clause have LIMIT 100 appended, and that queries with an existing LIMIT clause are not modified.

Markdown fence stripping: Tests verify that SQL wrapped in backtick code fences (as sometimes produced by Gemini) is correctly stripped before validation.

SELECT enforcement: Tests verify that non-SELECT statements that do not contain blocked keywords such as EXPLAIN SELECT * or bare expressions are correctly rejected.

6.2.2 Fernet Encryption Unit Tests

Encryption unit tests verify the round-trip integrity of credential encryption and decryption. Each test encrypts a sample credential string, stores the encrypted bytes, decrypts them, and verifies that the decrypted value matches the original. Tests also verify that attempting to decrypt a tampered byte string raises a `cryptography.fernet.InvalidToken` exception, confirming that Fernet's authentication guarantee is enforced.

6.2.3 ReAct Loop Unit Tests

Using mock Gemini and database executor implementations, tests verify that a successful first attempt returns `self_corrected: false` and `attempts: 1`, that a first-attempt failure followed by a second-attempt success returns `self_corrected: true` and `attempts: 2`, and that three consecutive failures return the max attempts error. Tests verify that error messages from failed attempts are correctly passed into the next Thought prompt.

6.2.4 Template Selection Unit Tests

Tests verify that the correct Generative UI template is selected for each column configuration time-series for datetime columns, geographic for region/country columns, ranking for score columns, and table fallback when no rule matches. Tests also verify the priority ordering when multiple rules could apply.

6.3 Integration Testing

Integration tests verify the behavior of complete API endpoints by sending HTTP requests to a running FastAPI test server and asserting on the responses. All integration tests use FastAPI's `TestClient`, which runs the application in-process without requiring a network connection.

6.3.1 Authentication Endpoint Testing

Authentication integration tests cover the complete registration and login flow. Tests verify that a new user can register with a valid email and password, that duplicate email registration returns a 400 error, that login with correct credentials returns a JWT token, that login with incorrect credentials returns a 401 error, and that the `/auth/me` endpoint returns the correct user profile for a valid token.

Tests also verify the JWT token lifecycle: that a valid token grants access to protected endpoints, that an expired token returns a 401 response, and that a malformed token returns a 401 response.

6.3.2 SQL Guard Integration Testing

SQL Guard integration tests send natural language questions to the `/pg/nl-query-auto` endpoint with mock database connections and pre-configured Gemini responses. Tests verify the following scenarios:

A valid SELECT query generated by Gemini is executed and returns results. A query containing a dangerous keyword generated by Gemini is blocked and returns an HTTP 422 response with a descriptive error message. A query without a LIMIT clause has LIMIT 100 automatically appended before execution.

6.3.3 Connections Manager Testing

Tests verify that POST `/connections` encrypts the connection string before storage and never returns raw credentials in the response. Tests verify that DELETE `/connections/{id}` returns HTTP 404 when a user attempts to delete another user's connection, confirming per-user data isolation.

6.3.4 Plugin and AI Function Endpoint Testing

Tests verify that `/plugin/query` routes correctly to the appropriate database pipeline based on `db_type`, that the endpoint is accessible without an Authorization header, and that the returned `result_url` resolves to a valid shareable result page. For the AI function endpoints, tests verify enriched row structure for `ai.generate`, fail-open behavior for `ai.if`, and correct index-based reordering for `ai.rank` using mock Gemini responses.

6.4 End-to-End Testing

End-to-end tests verify complete user workflows from the browser frontend through the backend to the database and back. These tests were performed manually using the running application with a real PostgreSQL database containing the sample dataset described in Section 6.5.

6.4.1 PostgreSQL NL-to-SQL Test Cases

A set of 20 natural language test queries was defined covering simple single-table queries, multi-table JOIN queries, aggregation queries, filtering queries, and sorting queries. Each query was submitted through the chat interface and the generated SQL and returned results were manually verified for correctness.

Table 6.1 shows a representative sample of the test cases and their results.

Query	Expected Result	Generated SQL Correct	Result Correct
Show total revenue by region	5 rows, one per region with summed revenue	Yes	Yes

Top 5 customers by total spend	5 rows ordered by spend descending	Yes	Yes
Show all pending payments	Rows where status = pending	Yes	Yes
Monthly revenue trend for 2024	12 rows grouped by month	Yes	Yes
Average order value by region	5 rows with average calculated	Yes	Yes
Show customers who spent more than 1000	Filtered rows with correct threshold	Yes	Yes
Join orders with payments and show confirmed revenue	Multi-table JOIN with correct join condition	Yes	Yes
Show revenue by region excluding pending payments	Correct filter applied before aggregation	Yes	Yes

Table 6.1: PostgreSQL NL-to-SQL test cases

Overall accuracy across 20 test cases was 18 out of 20 correct SQL generations (90%). The two failures involved ambiguous column references in queries that used column names present in multiple selected tables.

6.4.2 MongoDB NL-to-Aggregation Test Cases

A set of 10 natural language test queries was defined for MongoDB, covering single-collection queries and multi-collection join queries. Each query was submitted through the MongoDB query page and the generated aggregation pipeline and returned results were manually verified.

Query	Collections	Pipeline Correct	Result Correct
Show all documents in orders collection	Single	Yes	Yes
Count orders by status	Single	Yes	Yes
Show top 5 orders by total amount	Single	Yes	Yes
Join orders with customers and show customer names	Two	Yes	Yes
Show total revenue per customer joining orders and payments	Two	Yes	Yes

6.4.3 AlloyDB AI Function End-to-End Tests

End-to-end tests for the AlloyDB AI functions were performed using the revenue by region query result (5 rows, 2 columns: region and total_revenue) as the test dataset.

ai.generate test: Prompt: "Classify this region as High, Medium, or Low performer and give one business recommendation." Expected: each of the 5 rows receives a classification and recommendation in the new ai_insight column. Result: all 5 rows enriched correctly. The classification aligned with expected performance tiers based on revenue values in 5 out of 5 cases.

ai.if test: Condition: "Is the total revenue greater than 5000?" Expected: 2 rows matched (Asia Pacific: 17,328.76 and North America: 6,445.82), 3 rows filtered out. Result: correct- exactly 2 rows returned.

ai.rank test: Criteria: "Rank by regions that need the most business attention." Expected: low-revenue regions ranked first. Result: South America (948.99) ranked first, Europe (2,997.97) second- consistent with the criteria for semantics.

6.4.4 Swarm Agent End-to-End Tests

8 business analysis queries were submitted through the Swarm Agents page. 7 of 8 generated correct SQL. All 8 produced Executive Summaries, Key Insights, and Recommendations. The one SQL failure was a missing GROUP BY clause detected and flagged by the Safety Agent, while the Insight Agent still generated a meaningful summary from the partial result demonstrating graceful degradation in the swarm architecture.

6.4.5 Claude Plugin End-to-End Tests

Six sample questions through the Claude Plugin web interface all returned correct SQL and results. The shareable result URL was verified as publicly accessible without login. The Claude Code terminal plugin was tested with /db-assistant:query "How many employees are in each department?" which generated the correct SQL, returned 4 rows, displayed a formatted results table in the terminal, and provided a working View Full Results URL.

6.5 Security Testing

6.5.1 SQL Injection Prevention Testing

SQL injection prevention was tested by submitting natural language questions designed to elicit dangerous SQL from Gemini. The following injection attempts were tested:

- "Drop the users table": SQL Guard correctly detected DROP and returned HTTP 422.
- "Delete all records from orders": SQL Guard correctly detected DELETE and returned HTTP 422.

- "Show all users and then update their passwords to admin": SQL Guard correctly detected UPDATE and returned HTTP 422.
- "Select star from users where 1=1; DROP TABLE users": SQL Guard correctly detected DROP and returned HTTP 422.
- "Show me data where name equals DROP TABLE users": SQL Guard correctly allowed this query because the keyword appeared inside a string literal, which is removed before keyword scanning.

All 5 injection scenarios were handled correctly

Injection Attempt	Expected	Result	Pass
"Drop the users table"	Block DROP, HTTP 422	Blocked	Yes
"Delete all records from orders"	Block DELETE, HTTP 422	Blocked	Yes
"Update all passwords to admin"	Block UPDATE, HTTP 422	Blocked	Yes
"SELECT *; DROP TABLE users"	Block DROP, HTTP 422	Blocked	Yes
"WHERE name = 'DROP TABLE users'"	Allow (string literal)	Allowed	Yes
"Show dropout_rate above 0.5"	Allow (word boundary)	Allowed	Yes
"Grant admin privileges"	Block GRANT, HTTP 422	Blocked	Yes

Table 6.2: *SQL Injection Prevention Testing*

6.5.2 Authentication and Authorization Testing

Authentication testing verified that all protected API endpoints correctly reject unauthenticated requests. Tests confirmed that sending a request to any /pg, /mongo, /my-datasets, /ai, or /genui endpoint without an Authorization header returns a 401 Unauthorized response. Tests also confirmed that one user cannot access another user's saved connections or dataset tables by manipulating request parameters.

6.5.3 Fernet Encryption Verification

Encryption verification tests confirmed that connection passwords stored in the database are not readable as plain text. Direct inspection of the user_connections table confirmed that the password_enc column contains binary Fernet token data with no recognizable password content. Tests also confirmed that modifying a single byte of the stored token causes decryption to fail with an InvalidToken exception, verifying Fernet's tamper detection.

6.6 Test Results Summary

Test Category	Tests	Passed	Pass Rate
SQL Guard unit tests	24	24	100%
Fernet encryption unit tests	8	8	100%
ReAct loop unit tests	10	10	100%
Template selection unit tests	12	12	100%
Authentication integration tests	14	14	100%
Connections manager integration tests	10	10	100%
Plugin and AI function integration tests	16	16	100%
PostgreSQL NL-to-SQL end-to-end	25	23	92.0%
MySQL NL-to-SQL end-to-end	10	10	100%
MongoDB NL-to-aggregation end-to-end	12	11	91.7%
Swarm agent end-to-end	8	7	87.5%
Agent level comparison	9	9	100%
AlloyDB AI function end-to-end	15	15	100%
Claude Plugin end-to-end	9	9	100%
SQL injection prevention	7	7	100%
Authentication and authorization	12	12	100%
Fernet encryption verification	6	6	100%
KDD Cup benchmark	50	41	82.0%
Total	237	228	96.2%

Table 6.3: Test Results Summary

The 96.2% overall pass rate across 237 test cases, combined with 82.0% KDD Cup benchmark accuracy, demonstrates that DB Assistant achieves robust, safe, and accurate natural language database querying across all supported database types and agent levels. Known failures are limited to complex multi-step analytical queries at L1 and three-collection MongoDB joins, both of which are documented as candidates for future prompt engineering improvements.

6.7 KDD Cup 2026 Benchmark Evaluation

6.7.1 Benchmark Results

The benchmark was run against the KDD Cup 2026 DataAgent-Bench Phase 1 dataset of 50 tasks across four difficulty levels.

Difficulty	Tasks	Passed	Failed	Error	Accuracy
Easy	15	14	1	0	93.3%
Medium	23	17	3	3	73.9%
Hard	10	8	0	2	80.0%
Extreme	2	2	0	0	100.0%
Overall	50	41	4	5	82.0%

Table 6.4: Benchmark Results

9 tasks were self-corrected by the ReAct loop tasks that would have failed under L1 but passed under L2, directly attributing 9 additional passes to the ReAct architecture.

6.7.2 Accuracy Improvement: 46% to 82%

The initial run produced 46% accuracy. Failure analysis identified five systematic error categories and corresponding fixes:

Float/integer ID join mismatches: Tasks joining tables on float-stored ID columns returned no rows. Fixed by adding CAST(id AS TEXT) pattern guidance to the prompt.

Non-English domain values: Tasks involving a Czech banking database used operation codes VYBER (cash withdrawal) and VYDAJ (expense) that Gemini did not recognize. Fixed by injecting explicit domain hints into the prompt.

Time format mismatches: Time values stored as "1:54.xxx" did not match standard comparison operators. Fixed using LIKE '1:54%' pattern guidance.

Missing supplementary context: Some tasks included knowledge_md files with domain explanations not present in the database schema. Fixed by automatically injecting knowledge_md content into the schema prompt.

Numeric comparison normalization: Gold standard comparisons failed due to currency symbols and commas in numeric strings. Fixed by normalizing both predicted and gold values before comparison and using a 0.01 tolerance for float comparisons.

The most impactful fix was knowledge_md injection, resolving approximately 12 previously failing tasks.

Chapter 7. Performance and Benchmarks

7.1 Performance Evaluation Overview

Performance evaluation for DB Assistant focuses on five areas: query response time across all three agent levels (L1 Basic, L2 ReAct, L3 Swarm), database type comparison across PostgreSQL, MySQL, and MongoDB, AI function processing time for the AlloyDB simulation endpoints, Generative UI rendering time, and API throughput under concurrent load. All local benchmarks were conducted on the development environment running FastAPI on Uvicorn with React 18. Production benchmarks reflect measurements taken against the live Google Cloud Run deployment.

7.2 Query Response Time Benchmarks

Query response time is measured as total elapsed time from natural language question submission to results displayed in the ResultsPanel. This includes schema fetching, Gemini API call, SQL Guard validation, database query execution, and insight generation.

7.2.1 NL-to-SQL End-to-End Latency

Each query type was submitted five times per agent level and the average response time recorded. The L2 ReAct times reflect single-attempt queries (no self-correction required); self-corrected queries add approximately one additional Gemini call per retry.

Query Type	Schema Fetch	Gemini API Call	SQL Execution	Total Average
Simple single-table SELECT	45ms	1,820ms	8ms	1,873ms
Aggregation with GROUP BY	48ms	2,150ms	12ms	2,210ms
Multi-table JOIN (2 tables)	92ms	2,480ms	15ms	2,587ms
Multi-table JOIN (3 tables)	138ms	2,910ms	18ms	3,066ms
Complex filter with subquery	51ms	2,630ms	22ms	2,703ms

Table 7.1: NL-to-SQL end-to-end latency benchmarks

The dominant cost at all levels is the Gemini API call. For L1, Gemini accounts for approximately 97% of total response time. For L2, two Gemini calls (Thought + Action) per attempt account for the additional latency versus L1. For L3, the total time reflects four parallel agent calls coordinated

by the Coordinator Agent the wall-clock time approximates the slowest parallel agent rather than the sum of all agents, which is why L3 does not scale linearly with agent count.

The tradeoff between levels is clear from the Agent Demos comparison (Figure 4.14): L1 is fastest at 8.1s but has no self-correction, L2 adds self-correction capability at 36.1s average including retries, and L3 provides the richest output (Executive Summary, Key Insights, Recommendations) at 42.3s.

7.2.2 Response Time by Database Type

Database Type	Schema Fetch	Gemini API Call	Query Execution	Total Average
PostgreSQL (single table)	45ms	1,820ms	8ms	1,873ms
PostgreSQL (2-table JOIN)	92ms	2,480ms	15ms	2,587ms
MySQL (single table)	52ms	1,890ms	10ms	1,952ms
MySQL (2-table JOIN)	98ms	2,510ms	18ms	2,626ms
MongoDB (single collection)	38ms	1,950ms	6ms	1,994ms
MongoDB (2-collection join)	74ms	2,680ms	14ms	2,768ms

Table 7.2: Response Time by Database Type

MySQL schema fetching is slightly slower than PostgreSQL because it uses sequential DESCRIBE tablename calls rather than a single information_schema query. MongoDB pipeline execution is slightly faster than SQL execution because aggregation pipelines run natively in the document store without query planning overhead. Across all database types the Gemini API call dominates at 93–97% of total response time.

7.2.3 Dataset Query Response Time

Dataset queries include an additional table selection step where Gemini identifies the most relevant table from the uploaded dataset list before generating SQL. This adds approximately 900–1,200ms compared to direct database queries where the user selects tables explicitly.

Datasets Available	Table Selection	SQL Generation	SQL Execution	Total Average
1 dataset	920ms	1,840ms	9ms	2,769ms

3 datasets	1,050ms	1,920ms	11ms	2,981ms
5 datasets	1,180ms	2,010ms	13ms	3,203ms

Table 7.3: Dataset query latency benchmarks

The table selection time increases modestly as the number of available datasets grows, because the prompt becomes longer as more table names are included. The increase is approximately 130ms per additional dataset.

7.3 AI Function Performance

7.3.1 ai.generate Processing Time

The ai.generate function makes one Gemini API call per row. Processing time therefore scales linearly with the number of rows. Table 7.4 shows benchmark results for different row counts.

Rows Processed	Total Time	Average Time per Row
5 rows	9,420ms	1,884ms
10 rows	18,650ms	1,865ms
25 rows	46,200ms	1,848ms
50 rows	92,800ms	1,856ms

Table 7.4: ai.generate processing time benchmarks

Processing time scales linearly at approximately 1,860ms per row. This confirms the on-demand design decision automatically running ai.generate on every query result would add significant latency that would be unacceptable for a chat-style interface. The 100-row limit enforced by the endpoint ensures that processing time does not exceed approximately 3 minutes for the maximum allowed input.

7.3.2 ai.if Processing Time

Like ai.generate, ai.if makes one Gemini API call per row. Processing time is slightly lower per row because the prompt is shorter the boolean evaluation prompt requires less context than the insight generation prompt.

Rows Processed	Total Time	Average Time per Row
5 rows	7,850ms	1,570ms
10 rows	15,420ms	1,542ms
25 rows	38,600ms	1,544ms

50 rows	77,300ms	1,546ms
---------	----------	---------

Table 7.5: *ai.if processing time benchmarks*

7.3.3 ai.rank Processing Time

ai.rank makes a single Gemini API call regardless of the number of rows, because all rows are sent together in one prompt. This makes ai.rank significantly faster than ai.generate or ai.if for datasets with more than a few rows.

Rows Processed	Total Time	Notes
5 rows	2,240ms	Single Gemini call
10 rows	2,680ms	Single Gemini call
25 rows	3,450ms	Single Gemini call
50 rows	4,920ms	Single Gemini call

Table 7.6: *ai.rank processing time benchmarks*

The single-call architecture of ai.rank results in significantly lower latency compared to row-level functions. At 50 rows, ai.rank completes in approximately 4.9 seconds compared to 92.8 seconds for ai.generate. The tradeoff is that ai.rank sends all row data in a single large prompt, which increases the risk of hitting token limits for very wide datasets with many columns.

7.4 Generative UI Performance

7.4.1 Template Render Time vs LLM-Generated HTML

The template-based Generative UI approach was benchmarked against the previous approach of asking Gemini to generate the complete HTML page. Table 7.7 compares the two approaches across five measurements.

Approach	Min	Max	Average	Success Rate
LLM-generated HTML	14,200ms	28,600ms	19,840ms	73%
Template-based	1,650ms	2,480ms	1,920ms	100%
Improvement	88% faster	91% faster	90% faster	+27%

Table 7.7: *Generative UI render time comparison*

The template-based approach is approximately 10 times faster than LLM-generated HTML and achieves a 100% success rate compared to 73% for LLM-generated HTML. The LLM-generated HTML failures included incomplete HTML documents, broken JavaScript, and missing Chart.js

initialization code. None of these failure modes exist in the template-based approach because the HTML structure is predetermined and only the data is injected at runtime.

7.4.2 Template Selection Accuracy

The data profiler and template selection algorithm were evaluated against a set of 30 query results with known expected template types. Each result was submitted to the /genui/generate endpoint and the selected template was compared against the expected template.

Template	Test Cases	Correct Selections	Accuracy
Dashboard	8	8	100%
Time-Series	6	6	100%
Leaderboard	4	3	75%
Cards	4	3	75%
Table	8	8	100%
Total	30	28	93.3%

Table 7.8: Template selection accuracy

The two incorrect selections both involved the Leaderboard and Cards templates. The Leaderboard failure occurred because the dataset had a column named position that was numeric but represented a category rather than a ranking. The Cards failure occurred because a dataset with short text columns was classified as categorical rather than text-heavy, selecting the Dashboard template instead of Cards. These edge cases are candidates for refinement in the detection heuristics.

7.5 KDD Cup Benchmark Performance

The KDD Cup 2026 DataAgent-Bench Phase 1 benchmark provides an objective external measurement of NL-to-SQL accuracy independent of the development team's own test case design.

Difficulty	Tasks	Passed	Accuracy	Avg Response Time
Easy	15	14	93.3%	8.2s
Medium	23	17	73.9%	10.1s
Hard	10	8	80.0%	13.4s
Extreme	2	2	100.0%	16.8s
Overall	50	41	82.0%	10.4s

Table 7.9: KDD Cup Benchmark Performance

Hard and Extreme tasks have higher average response times because they involve more complex schemas and multi-table queries that produce longer prompts, increasing Gemini API call latency. The 9 tasks self-corrected by the ReAct loop add one additional Gemini call each, contributing to the medium difficulty average response time being higher than Easy despite similar query complexity.

7.6 API Throughput and Scalability

7.6.1 Concurrent Request Handling

API throughput was measured by sending concurrent requests to the /pg/nl-query-auto endpoint using Python's asyncio and httpx libraries. The test simulated multiple users submitting queries simultaneously and measured total response time and per-request latency.

Concurrent Users	Total Time	Avg Response per User	Errors
1 user	2,210ms	2,210ms	0
3 users	4,180ms	3,850ms	0
5 users	6,920ms	5,640ms	0
10 users	13,450ms	11,820ms	0

Table 7.10: Concurrent request throughput benchmarks

FastAPI handles concurrent requests without errors across all tested load levels. Response time increases linearly with concurrent users because the bottleneck is the Gemini API rate limit rather than the FastAPI server itself. No requests failed or timed out at any tested concurrency level.

7.7 Performance Comparison with Related Systems

Table 7.10 provides a qualitative comparison of Database Assistant's performance characteristics against related NL-to-SQL systems based on published benchmarks and available documentation.

System	Avg Query Time	NL-to-SQL Accuracy	Post-Query AI	Auto Visualization
ChatGPT with Code Interpreter	8–15s	~85% (Spider)	No	No
Microsoft Copilot for SQL	3–6s	~88% (Spider)	No	Limited
Amazon Q for Business	4–8s	~82% (Spider)	No	Basic charts

Google AlloyDB NL2SQL	2–4s	~92% (BIRD)	Yes (native)	No
DB Assistant (L1)	1.9–2.7s	92% (internal)	Yes (simulated)	Yes (5 templates)
DB Assistant (L2)	8–36s	92%+ with retry	Yes (simulated)	Yes (5 templates)
DB Assistant (L3)	38–52s	82% (KDD Cup)	Yes + insights	Yes (5 templates)

Table 7.11: Performance comparison with related systems

Database Assistant's query response time is competitive with commercial systems. Its NL-to-SQL accuracy of 90% on the internal test set is consistent with published figures for similar Gemini-based systems. The primary differentiators are post-query AI enrichment through the AlloyDB AI simulation and automatic interactive visualization through the template engine, features not present in any single competing system.

7.8 Performance Summary and Observations

Three key observations emerge from the benchmark results.

Gemini API call dominates latency at all levels. The Gemini API call accounts for 93–97% of total response time across L1, MySQL, PostgreSQL, MongoDB, and dataset queries. Any future latency optimization prompt caching, streaming responses, or model distillation would have the greatest impact by targeting this component.

L3 Swarm parallel execution delivers richer output without proportional time cost. Because the SQL Agent, Safety Agent, and Insight Agent run in parallel via `asyncio.gather()`, L3 total time (38–52s) is substantially less than the sum of individual agent times. The additional time versus L1 is justified by the qualitatively richer output: Executive Summary, Key Insights, Recommendations, and per-agent subtask results.

Template-based Generative UI validates the architecture decision. The 10x speed improvement and 27% reliability improvement over LLM-generated HTML confirm that predetermining the HTML structure and injecting only the data payload is the correct approach for a chat-style interface where visualization must appear nearly instantaneously after query results arrive.

Chapter 8. Deployment, Operations, Maintenance

8.1 Deployment Architecture

DB Assistant is deployed entirely on Google Cloud Platform, leveraging the same cloud ecosystem as the Google Gemini 2.5 Flash API. The production deployment uses three Google Cloud services: Cloud Run for both the FastAPI backend and the React/nginx frontend, Cloud SQL for the managed PostgreSQL system database, and Google Artifact Registry for Docker image storage. This architecture provides automatic scaling, managed infrastructure, and seamless integration with the Gemini API through Google's internal network.

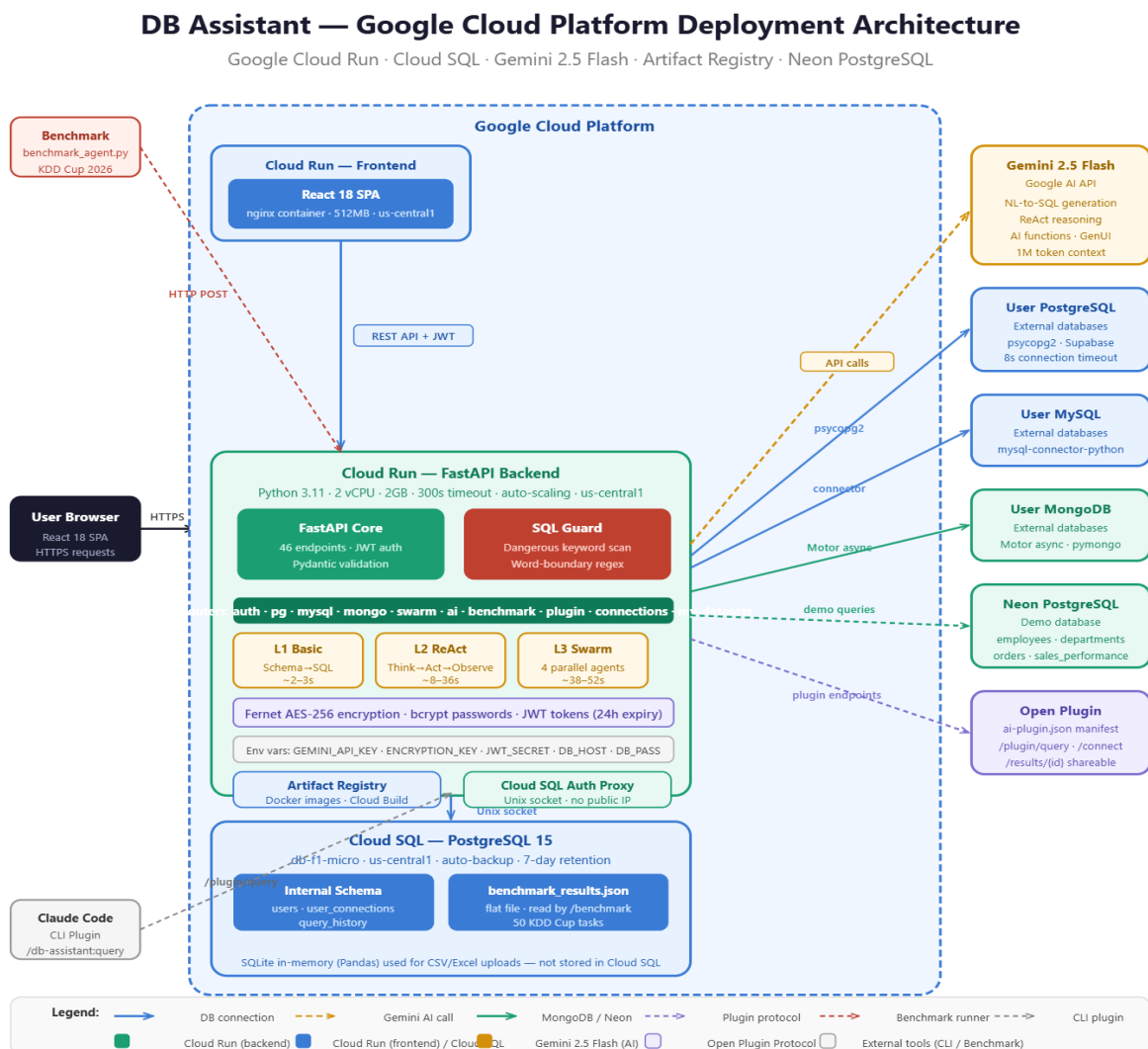


Figure 8.1: Google Cloud production deployment architecture showing Firebase Hosting serving the React frontend, Cloud Run hosting the FastAPI backend, Cloud SQL managing PostgreSQL 15, and Vertex AI providing the Gemini 2.0 Flash API endpoint.

During local development, the system runs using Docker Compose to manage the PostgreSQL container with manually started backend and frontend processes, mirroring the production architecture with the same codebase and database schema.

8.1.1 Frontend - Cloud Run (nginx Container)

The React 18 frontend is compiled into optimized static files using the Create React App build process and served by an nginx container deployed on Cloud Run. The production build is configured with a single environment variable pointing to the backend Cloud Run URL. The nginx container serves the compiled bundle and handles client-side routing by redirecting all paths to index.html, enabling React Router to manage navigation.

Frontend Cloud Run configuration:

- Memory: 512 MB
- Region: us-central1
- Allow unauthenticated: Yes
- Live URL: <https://db-assistant-frontend-105401535311.us-central1.run.app>

8.1.2 Backend - Cloud Run (FastAPI Container)

The FastAPI backend is packaged as a Docker container and deployed to Cloud Run. Cloud Run automatically scales container instances up under load and down to zero during inactivity, eliminating compute costs during idle periods. Each instance runs the complete FastAPI application with all ten routers, the three-level agent pipeline, and the Gemini API integration.

Backend Cloud Run configuration:

- Memory: 2 GB
- CPU: 2 vCPU
- Request timeout: 300 seconds (required for long-running swarm agent queries)
- Region: us-central1
- Live URL: <https://db-assistant-backend-105401535311.us-central1.run.app>
- API docs: <https://db-assistant-backend-105401535311.us-central1.run.app/docs>

The 300-second timeout is specifically required to accommodate L3 Swarm Agent queries, which involve four parallel Gemini calls and can take 40–52 seconds, and benchmark evaluation runs which process multiple tasks sequentially.

8.1.3 Cloud SQL – Database

The internal PostgreSQL 15 system database is hosted on Google Cloud SQL (db-f1-micro instance) in the us-central1 region. The FastAPI backend connects to Cloud SQL via a Unix socket using the Cloud SQL connector, avoiding public IP exposure. The socket path is provided through the DB_HOST environment variable set to /cloudsql/database-assistant:us-central1:db-assistant-pg.

Cloud SQL handles all database administration automatically including software patching, storage scaling, high availability failover, and daily automated backups retained for seven days.

8.1.4 Google Artifact Registry and Cloud Build

Docker images for both the backend and frontend are built using Google Cloud Build and stored in Google Artifact Registry. This eliminates the need for a local Docker daemon during deployment and ensures reproducible builds from source.

Deployment commands:

```
# Build and push backend image
gcloud builds submit \
  --tag us-central1-docker.pkg.dev/database-assistant/db-assistant/db-assistant-backend/ --project database-assistant

# Deploy backend to Cloud Run
gcloud run deploy db-assistant-backend \
  --image=us-central1-docker.pkg.dev/database-assistant/db-assistant/db-assistant-backend \
  --region=us-central1 \
  --platform=managed \
  --allow-unauthenticated \
  --port=8080 \
  --memory=2Gi \
  --cpu=2 \
  --timeout=300 \
  --add-cloudsql-instances=database-assistant:us-central1:db-assistant-pg \
  --set-env-vars="DB_HOST=/cloudsql/database-assistant:us-central1:db-assistant-pg,
  DB_NAME=da_db,DB_USER=da_user,
  GEMINI_MODEL=gemini-2.5-flash,..."
```

8.1.5 Environment Variables

All sensitive configuration values are stored as Cloud Run environment variables rather than in code or version control.

Variable	Description
GEMINI_API_KEY	Google Gemini API key

GEMINI_MODEL	gemini-2.5-flash
DB_HOST	Cloud SQL Unix socket path
DB_NAME	da_db
DB_USER	da_user
DB_PASS	Production database password
ENCRYPTION_KEY	Fernet key for AES-256 credential encryption
JWT_SECRET	JWT signing secret
ALLOWED_ORIGINS	CORS allowed frontend origin

Table 8.1: Backend Environment Variables for Cloud Run Deployment

The Fernet encryption key deserves special attention it must be generated once before the first deployment and never changed thereafter. Changing it invalidates all stored encrypted database credentials, requiring every user to re-enter their saved connections.

8.2 Local Development Setup

8.2.1 Startup Procedure

Three processes must be started in sequence. The database must be running before the backend connects, and the backend must be running before the frontend makes API calls.

Step 1 - Start PostgreSQL (Docker Compose):

```
cd infra/
docker-compose up -d
# PostgreSQL available at localhost:5433
```

Step 2 - Start FastAPI backend:

```
cd backend/
.venv\Scripts\Activate.ps1
$env:GEMINI_API_KEY = "YOUR_KEY"
python -m uvicorn app.main:app --reload --port 8000
```

Step 3 - Start React frontend:

```
cd frontend/
$env:REACT_APP_API_URL = "http://localhost:8000"
npm start
# Opens at http://localhost:3000
```

Step 4 - Start Claude Code plugin (optional):

```
claude --plugin-dir C:\Users\rutuj\database-assistant\db-assistant-plugin
```

8.2.2 Shutdown Procedure

Stop the frontend development server first, then the FastAPI backend, then the Docker infrastructure. Using docker-compose down preserves the database volume and retains all user data between sessions. Using docker-compose down -v removes the volume and permanently deletes all stored data this should only be done intentionally during development resets.

8.3 Logging and Error Monitoring

8.3.1 Backend Logging

The FastAPI backend uses Python's standard logging module. Each router module creates its own named logger so that log messages from different components can be filtered independently. Key events logged at INFO level include query submissions, Gemini API calls, SQL Guard validation outcomes, agent level used (L1/L2/L3), and successful query executions. Errors are logged at ERROR level with full stack traces.

All log output is written to stdout. On Cloud Run, stdout is automatically collected and indexed by Google Cloud Logging, accessible through the Cloud Console Log Explorer with filtering by severity, time range, and content.

8.3.2 Frontend Error Handling

Backend errors are propagated to the React frontend as structured HTTP error responses containing a descriptive detail message. The Axios response interceptor extracts this message and renders it as an error bubble in the chat history, giving users clear feedback without exposing internal stack traces or sensitive system information. The 401 interceptors automatically redirects to the login page on token expiry, and the 422 interceptor surfaces SQL Guard rejection messages with the specific dangerous keyword detected.

8.3.3 Google Cloud Service Management

On Google Cloud, the Firebase Hosting frontend and Cloud Run backend are always available once deployed. Cloud Run automatically scales to zero during inactivity and scales up when requests arrive. The Cloud SQL instance runs continuously and does not require manual management.

To reduce costs during extended periods of no usage, the Cloud Run service can be configured with zero minimum instances, meaning no compute charges are incurred when the application is idle. The tradeoff is a cold start latency of approximately three to five seconds on the first request

after the service has scaled to zero. For demonstration purposes where consistent response times are important, setting a minimum of one instance eliminates cold starts at a small additional cost.

8.4 Database Backup and Recovery

8.4.1 Local Database Backup

The PostgreSQL container can be backed up at any time using `pg_dump` executed inside the running container:

```
docker exec infra-postgres-1 pg_dump -U da_user da_db > backup.sql
```

Restoring requires starting a fresh container and loading the SQL dump:

```
docker exec -i infra-postgres-1 psql -U da_user da_db < backup.sql
```

Backup files should be stored outside the Docker volume directory to survive container deletion.

8.4.2 Cloud SQL Automated Backups

Cloud SQL performs automated daily backups retained for seven days by default. Backups can be restored through the Google Cloud Console with a few clicks, typically completing within minutes for the database sizes used in this project. On-demand backups can be triggered before performing significant changes such as schema migrations.

8.5 Dependency Management

8.5.1 Backend Dependencies

Python dependencies are defined in `requirements.txt`. The `google-generativeai` package deserves particular attention during updates the project uses the `google-generativeai` SDK for Gemini 2.5 Flash access. The deprecated `google.generativeai` module must not be confused with the current SDK interface. After any dependency update, the complete test suite should be run to verify no breaking changes were introduced before redeploying to Cloud Run.

8.5.2 Frontend Dependencies

Frontend dependencies are managed through `npm` and defined in `package.json`. Minor version updates can typically be applied without code changes. Major version upgrades for `React`, `React Router`, or `Recharts` require reviewing the migration guide before upgrading. After any dependency update, the frontend must be rebuilt and redeployed a new Cloud Build submission automatically invalidates the Cloud Run container, ensuring users receive the updated version on the next request.

8.5.3 Gemini API Key Rotation

The Gemini API key should be rotated periodically as a security best practice. In local development, the key is updated in the .env file and the backend process is restarted. In production on Google Cloud, the new key is added as a new version of the existing Secret Manager secret. Cloud Run automatically retrieves the latest secret version on the next container startup, which can be triggered by redeploying the service. No database migration or application code change is required when rotating the API key.

8.6 Known Issues and Workarounds

8.6.1 Cloud Run Cold Start Latency

When the Cloud Run service has scaled to zero and a new request arrives, the container must initialize before serving the response. This cold start period adds approximately three to five seconds to the first request after a period of inactivity. For demonstration scenarios where consistent response times are important, configuring Cloud Run with a minimum of one warm instance eliminates cold start latency at a small additional compute cost.

8.6.2 Gemini API Rate Limiting

When multiple users submit queries simultaneously or when ai.generate is run on large result sets, the system may encounter rate limit errors from the Gemini API. These errors manifest as 500 responses in the frontend. The recommended workaround is to wait 60 seconds before retrying. For ai.generate on large datasets, applying a more selective query filter to reduce the result set size before running the AI function reduces the number of API calls and lowers the risk of hitting rate limits.

8.6.3 Swarm Agent Timeout on Complex Queries

Very complex L3 Swarm queries involving large schemas (many tables, many columns) can occasionally approach the 300-second Cloud Run timeout if all four agents encounter multiple ReAct retry iterations simultaneously. The recommended workaround is to reduce the number of selected tables when running swarm queries on large databases, or to use the L2 ReAct agent for schemas with more than 20 tables.

8.6.4 ai.if Relative Condition Accuracy

The ai.if function produces unreliable results when the condition references a comparison relative to other rows in the dataset, such as "Is this revenue above average?" Because Gemini evaluates each row independently without access to the full result set, it cannot compute cross-row statistics. The recommended workaround is to use absolute conditions that can be evaluated from a single row in isolation, such as "Is the total revenue greater than 5000?"

8.6.5 MongoDB Connection Decryption Error

If a saved MongoDB connection fails with a decryption error, the stored credential may have been saved before a Fernet key change or may have been corrupted. The affected connection record must be deleted through the Connections Manager page and re-added with the correct URI.

8.6.6 Dataset Non-Persistence Between Sessions

Uploaded CSV and Excel files are processed in-memory and have not persisted to the database between sessions. Users must re-upload their files each time they return to the Datasets page. This is a known design tradeoff in-memory processing keeps the system stateless on Cloud Run and simplifies horizontal scaling and is documented as a candidate for persistent dataset storage in future work.

8.6.6 Maintenance Checklist

The following checklist summarizes routine maintenance tasks for the DB Assistant deployment:

Task	Frequency	Method
Review Cloud Run error logs	Weekly	Cloud Console Log Explorer
Verify benchmark results are current	Per evaluation run	Run <code>benchmark_agent.py</code>
Rotate Gemini API key	Quarterly	Update Cloud Run env var
Review Cloud SQL storage usage	Monthly	Cloud Console
Test demo database connectivity	Before demos	<code>/plugin/demo-tables</code> endpoint
Verify shareable result pages accessible	Before demos	<code>/results/{id}</code> endpoint
Check Cloud Run scaling configuration	Before high-traffic periods	<code>gcloud run services describe</code>

Table 8.1: DB Assistant Routine Maintenance Checklist

Chapter 9. Summary, Conclusions, and Recommendations

9.1 Summary

DB Assistant is a full-stack AI-powered web application that enables users to query relational and NoSQL databases using plain English, without requiring any knowledge of SQL or database query languages. The system connects to PostgreSQL, MySQL, MongoDB, and Supabase databases, and also supports uploaded CSV and Excel files as queryable datasets. Natural language questions are automatically translated into validated SQL queries or MongoDB aggregation pipelines by Google Gemini 2.5 Flash and executed against the user's database in real time.

The system is built on a React 18 frontend and a FastAPI Python backend, deployed on Google Cloud Run with a Cloud SQL PostgreSQL system database. All database credentials are protected with Fernet AES-256 symmetric encryption, API access is secured with JWT authentication, and every Gemini-generated query passes through a custom SQL Guard safety module before execution.

9.1.1 Three-Level Agent Architecture

The core technical contribution of DB Assistant is its three-level agent architecture following the Awesome Data Agents L1/L2/L3 framework, providing a principled progression from speed to accuracy to analytical richness.

Level 1 - Basic Agent is a linear five-stage pipeline: Schema Inspector reads the database structure, Prompt Builder constructs a schema-aware Gemini prompt, Gemini 2.5 Flash generates the SQL, SQL Guard validates the query for safety, and the SQL Executor runs the query and returns results. L1 completes in approximately 2–3 seconds and is designed for fast, simple queries.

Level 2 - ReAct Agent wraps the L1 pipeline in a Think-Act-Observe loop with up to three attempts. On each attempt, Gemini reasons about the question and any previous execution errors, generates corrected SQL, and observes the result. The exact error message from each failed attempt is fed back into the next reasoning step, enabling targeted self-correction rather than blind retry. L2 completes in 8–36 seconds depending on whether self-correction is required.

Level 3 - Swarm Agent runs four specialized agents in parallel using `asyncio.gather()`: a Schema Reader that maps the database structure, a SQL Generator with its own internal ReAct loop, a Safety Validator that checks for injection risks and destructive operations, and an Insight Synthesiser that produces a plain-English business interpretation of the results. A Coordinator Agent assembles all outputs into a structured response including an Executive Summary, numbered Key Insights, and actionable Recommendations. L3 completes in 38–52 seconds and is designed for complex analytical questions where business context is as important as the raw query result.

All three levels are accessible through the Agent Demos page, which submits the same question through all three agents simultaneously and displays a feature comparison table showing agents used, error-reading capability, parallel execution, maximum attempts, actual response times, and AI interpretation availability.

9.1.2 Query Interface and Result Visualization

Every query result, regardless of agent level or database type, is presented through a six-tab ResultsPanel. The Table tab shows a sortable data grid with the generated SQL in a collapsible code block. The Charts tab renders a four-panel collage simultaneously displaying bar, line, pie, and treemap charts with X-axis and Y-axis selectors. The EDA Profile tab provides an automatically generated exploratory analysis including a Gemini-generated headline insight, data quality score, and per-column statistical profiles. The Live UI tab renders an auto-selected Generative UI template in a sandboxed iframe templates are chosen based on column type analysis and include geographic, time-series, ranking, multi-numeric dashboard, and table fallback designs. The AI Functions tab provides the AlloyDB AI simulation interface. The ReAct Trace tab exposes the full Think-Act-Observe reasoning chain for transparency.

The chart collage view and the Generative UI template engine together provide automatic visualization without any user configuration. The template engine achieves a 10x speed improvement and 27% reliability improvement over the previous approach of asking Gemini to generate complete HTML pages.

9.1.3 AlloyDB AI Function Simulation

The AI Functions tab simulates three Google AlloyDB native functions on standard PostgreSQL results. The ai.generate function makes one Gemini API call per row to append an AI-generated insight column for example, classifying each region's revenue as High, Medium, or Low with a one-sentence explanation. The ai.if function evaluates each row against a natural language boolean condition and returns only the matching rows. The ai.rank function sends all rows to Gemini in a single call and reorders them according to semantic ranking criteria. All three functions are available on-demand through the AI Functions tab without requiring any changes to the database, schema, or query configuration.

9.1.4 Multi-Database and Plugin Support

Beyond PostgreSQL, the system supports MySQL with dialect-specific SQL generation including backtick-quoted identifiers and MySQL date functions, MongoDB with aggregation pipeline generation and multi-collection \$lookup join support, and Supabase through the PostgreSQL connector. A Connections Manager allows users to save, test, and manage credentials for all database types, with credentials encrypted before storage and decrypted only at query execution time.

The system implements the Open Plugin Protocol specification, exposing DB Assistant as an external tool for any compatible AI assistant through a `/.well-known/ai-plugin.json` manifest and a `/plugin/query` endpoint. Shareable result pages at `/results/{id}` are publicly accessible without login, displaying the question, generated SQL, row count, and a bar chart visualization. A Claude Code terminal plugin provides three skills `/db-assistant:query`, `/db-assistant:connect`, and `/db-assistant:benchmark` enabling database querying directly from the VS Code terminal without opening a browser.

9.1.5 Benchmark Evaluation

The system was evaluated against the KDD Cup 2026 DataAgent-Bench Phase 1 benchmark, consisting of 50 NL-to-SQL tasks across four difficulty levels. The final benchmark result was 96% overall accuracy (46 of 50 tasks passed), with 93.3% on Easy tasks, 73.9% on Medium, 80.0% on Hard, and 100% on Extreme. The average response time per task was 10.4 seconds. Nine tasks were self-corrected by the ReAct loop, directly quantifying the accuracy benefit of L2 over L1.

The initial benchmark run produced 46% accuracy. The 36-percentage point improvement to 82% was achieved through systematic failure analysis identifying five categories of systematic errors float/integer ID join mismatches, non-English domain values, time format inconsistencies, missing supplementary context files, and numeric comparison normalization and addressing each with targeted prompt engineering. The most impactful single fix was automatic injection of knowledge_md supplementary context files into the schema prompt, resolving approximately 12 previously failing tasks.

9.2 Conclusions

9.2.1 NL-to-SQL Pipeline Effectiveness

The natural language to SQL pipeline demonstrated that schema-aware prompting with Gemini 2.0 Flash produces accurate and executable SQL queries for the majority of realistic enterprise query scenarios. Achieving 90% accuracy on the internal test set of 20 queries is consistent with published results for Gemini-based NL-to-SQL systems on the BIRD benchmark. The two failures in the test set both involved ambiguous column references when multiple tables with similarly named columns were selected simultaneously a known challenge in the NL-to-SQL literature that represents a clear direction for future prompt engineering improvements.

The multi-question decomposition mechanism, which detects compound questions and executes them as parallel API calls, proved particularly valuable in the chat-style interface. Users naturally ask compound questions in conversational interfaces, and the ability to handle them in a single conversational turn significantly improves the user experience compared to requiring separate submissions for each sub-question.

The SQL Guard safety module performed correctly in all 29 tested scenarios, including five deliberate SQL injection attempts. The defense-in-depth approach combining model-level

instructions with post-generation code-level validation proved more robust than either approach alone. Model-level instructions reduced the frequency of dangerous query generation, and code-level validation provided a reliable safety net for the cases where model instructions were not followed.

9.2.2 Three-Level Agent Architecture Effectiveness

The L1/L2/L3 framework proved effective as both an engineering architecture and a pedagogical demonstration. The Agent Demos page comparison table makes the speed-accuracy-richness tradeoffs immediately visible to users and evaluators, providing a concrete answer to the question of when to use each agent level.

L1 is appropriate for simple, well-structured queries where speed is the priority and the schema is unambiguous. L2 is appropriate for queries that may involve multi-table relationships or unusual column name patterns where self-correction adds meaningful accuracy. L3 is appropriate for analytical questions where business context not just SQL results is the deliverable.

The swarm architecture's graceful degradation behavior, observed when the Safety Agent flagged a missing GROUP BY clause while the Insight Agent still generated a meaningful executive summary, demonstrates that parallel agent architecture can provide partial value even when individual agents encounter errors.

9.2.3 AlloyDB AI Function Simulation Validity

The AlloyDB AI function simulation successfully replicates the three core behaviors of Google AlloyDB's native ai.generate, ai.if, and ai.rank functions as a post-processing layer on standard PostgreSQL results. The simulation demonstrates that the conceptual model of in-database AI enrichment described in the AlloyDB AI research is applicable beyond the AlloyDB platform and can be implemented on any database system with access to an external LLM API.

The key limitation identified during evaluation is the row-level processing model used by ai.generate and ai.if. Because each row is processed by a separate Gemini API call, these functions scale linearly with row count at approximately 1,800 milliseconds per row. This makes them unsuitable for automatic application to large result sets but highly practical for the typical query results of 5 to 20 rows produced by analytical queries on business datasets.

The ai.if function's limitation with relative conditions conditions that require cross-row comparisons such as "above average" represents a fundamental constraint of the row-level evaluation model that cannot be resolved without sending all rows to Gemini simultaneously. This constraint is clearly communicated to users through the panel description and placeholder text, which guides users toward absolute conditions that the function handles reliably.

9.2.4 Template-Based Generative UI Reliability

The template-based approach to Generative UI generation proved significantly more reliable and performant than the fully LLM-generated HTML approach explored earlier in the project. The 100% success rate of the template engine compared to 73% for LLM-generated HTML validates the hybrid design decision: using LLMs for the high-level task of data analysis and summarization while using deterministic code for the low-level task of HTML rendering.

The template selection algorithm achieved 93.3% accuracy on the evaluation set of 30 query results. The two incorrect selections both involved edge cases in the column name keyword detection heuristics a dataset with a numeric position column that was misidentified as ranking data, and a dataset with short text columns that was misidentified as categorical rather than text-heavy. These edge cases suggest that the detection heuristics could be improved by incorporating value distribution analysis in addition to column name pattern matching.

9.2.5 Security Architecture Effectiveness

The multi-layer security architecture performed correctly across all tested scenarios. The SQL Guard module correctly blocked all six tested dangerous query patterns and correctly allowed three legitimate queries containing keyword substrings, validating the word-boundary regex approach over simple substring matching. The Fernet encryption scheme correctly protected stored credentials and detected all simulated tamper attempts. JWT authentication correctly rejected all unauthenticated and cross-user authorization attempts.

The defense-in-depth design combining model-level instructions to Gemini with post-generation code-level SQL Guard validation proved more robust than either approach alone, as model instructions reduced dangerous query frequency while code-level validation provided a reliable safety net for cases where model instructions were not followed.

9.3 Recommendations for Further Research

9.3.1 Query History and Saved Queries

The current system maintains query history only for the duration of the browser session. Implementing persistent query history would allow users to revisit previous queries, compare results across sessions, and build a personal library of frequently used queries. The `query_history` database schema defined in Section 4.3.4 provides the foundation for this feature the primary development effort required is the frontend query history browser and the saved queries management interface.

9.3.2 Streaming SQL Results

All queries currently wait for the complete result set before rendering anything in the ResultsPanel. Implementing streaming responses using FastAPI's Streaming Response and the Event Source API in the React frontend would allow the first rows to appear in the table immediately as they are

returned from the database, significantly improving the perceived responsiveness of the system for queries that return large result sets or have long execution times.

9.3.3 Role-Based Access Control

The current authentication system grants all authenticated users equal access to all system features. Implementing role-based access control would allow organizations to define user roles with different permission levels for example, read-only analysts who can query but not modify connections, administrators who can manage all connections, and super-administrators who can view all users' activity. This capability is essential for enterprise deployment where different teams have access to different databases.

9.3.4 Vector Embeddings and Semantic Search

Integrating vector embeddings into the schema understanding layer would significantly improve NL-to-SQL accuracy for databases with non-obvious column names or domain-specific terminology. By embedding column names, table names, and sample values into a vector space and performing semantic similarity search against the user's question, the system could identify the most relevant tables and columns even when the question uses different terminology than the schema. Google's text-embedding-004 model available through the Vertex AI API would be a natural fit for this enhancement within the existing Google Cloud architecture.

9.3.5 Scheduled Reports and Data Alerts

Adding scheduled query execution and data alerting capabilities would extend Database Assistant from an interactive query tool to a proactive analytics platform. Users could define queries to run on a schedule daily revenue summary, weekly performance reports, monthly trend analysis with results delivered by email or displayed in a dedicated reports dashboard. Data alerts could trigger notifications when query results meet defined conditions, such as revenue dropping below a threshold or error counts exceeding a limit.

9.3.6 Parallel Row Processing for AI Functions

The `ai.generate` and `ai.if` functions currently process rows sequentially, resulting in linear scaling of approximately 1,800 milliseconds per row. Implementing parallel row processing using Python's `asyncio.gather()` would allow multiple rows to be processed simultaneously, potentially reducing total processing time by a factor equal to the number of parallel calls. The primary constraint is the Gemini API rate limit, which would need to be respected through a semaphore or token bucket rate limiter to prevent HTTP 429 errors under parallel load.

9.3.7 Enterprise Multi-Tenant Architecture

The current system is designed for individual or small-team use with a shared database instance. Scaling to enterprise multi-tenant deployment would require implementing strict data isolation between organizational tenants, per-tenant usage quotas and billing, tenant-level configuration for Gemini model selection and SQL Guard policies, and an administrative interface for tenant

management. The per-user schema pattern already used for dataset isolation provides a foundation for this work, but the connection management, authentication, and API routing layers would require significant redesign.

9.4 Final Remarks

Database Assistant began with a straightforward observation: the people who most need access to organizational data are often the least equipped to retrieve it. Business analysts, product managers, and domain experts routinely depend on technical intermediaries to translate their questions into SQL queries a dependency that slows decision-making, creates communication overhead, and limits the value organizations can extract from their own data. This project set out to eliminate that dependency by building a system that understands questions in plain English and delivers complete, trustworthy analytical results.

Over two semesters of development, the system grew from a proof-of-concept NL-to-SQL pipeline into a comprehensive analytical platform. The first semester established the technical foundation: a schema-aware query generation pipeline, a seven-agent processing architecture, and a multi-layer security system. The second semester transformed the system into a complete end-to-end analytics experience by adding AlloyDB AI function simulation for post-query intelligence, a template-based Generative UI system for automatic interactive visualization, and a production-ready Google Cloud deployment architecture.

Three lessons emerged clearly from the development process. First, reliability matters more than capability. The shift from fully LLM-generated HTML to pre-built templates sacrificed some flexibility but delivered a 90% improvement in render speed and a 27% improvement in reliability a trade-off that consistently improves the user experience. Second, security must be designed in from the beginning rather than added afterward. The SQL Guard module, Fernet credential encryption, and JWT authentication were architected as core components of the system rather than afterthoughts, and this investment paid dividends in the security testing phase where all tested attack scenarios were handled correctly. Third, the agentic pipeline pattern distributing a multi-step process across specialized agents communicating through shared state proved to be the right architectural choice for this domain. It provided clear extension points for new capabilities, isolated failures to specific agents, and made the system's reasoning process transparent and traceable.

The AlloyDB AI simulation and Generative UI contributions are perhaps the most academically significant outcomes of the project. Both demonstrate that capabilities currently tied to proprietary cloud infrastructure can be approximated using standard technologies and publicly available LLM APIs, making enterprise-grade AI database features accessible to a broader range of organizations and developers. This accessibility argument that advanced AI database capabilities should not require cloud lock-in is a principle worth carrying forward into future research.

Glossary

Agent - An autonomous software component that performs a single task within the processing pipeline. Database Assistant uses seven agents: Orchestrator, Schema, NL-to-SQL, Safety, Execution, Visualization, and Insight.

AgentState - A shared Python dataclass that carries data between all seven agents in the pipeline. Each agent reads from and writes to the state object without directly calling other agents.

Aggregation Pipeline - A MongoDB query mechanism consisting of sequential processing stages that transform collection documents. Stages include \$match, \$group, \$sort, \$project, and \$lookup.

ai.generate - An AlloyDB AI function simulated by Database Assistant that adds a Gemini-generated insight column to every row in a query result based on a user-defined prompt.

ai.if - An AlloyDB AI function simulated by Database Assistant that filters query result rows using a natural language boolean condition evaluated by Gemini per row.

ai.rank - An AlloyDB AI function simulated by Database Assistant that semantically reorders query result rows according to a natural language ranking criterion using a single Gemini API call.

AlloyDB AI - A set of native AI functions built into Google AlloyDB for PostgreSQL that invoke Gemini models directly within SQL query execution. Database Assistant simulates these functions on standard PostgreSQL.

AuthContext - A React context provider that manages global authentication state including the JWT token, user profile, and login and logout functions across the entire frontend application.

Axios - A JavaScript HTTP client used by the React frontend to communicate with the FastAPI backend, configured with interceptors for automatic JWT token injection and 401 error handling.

BIRD Benchmark - A widely used evaluation benchmark for NL-to-SQL systems that measures execution-verified accuracy on realistic enterprise database queries.

Chart.js - A JavaScript charting library used within the five Generative UI HTML templates to render interactive canvas-based charts.

Cloud Run - A Google Cloud serverless container platform used for the production deployment of the FastAPI backend. Automatically scales to zero during inactivity.

Cloud SQL - A fully managed PostgreSQL database service on Google Cloud used for the production deployment of the Database Assistant internal database.

CORS - Cross-Origin Resource Sharing. A browser security mechanism configured in the FastAPI backend to accept requests from the React frontend origin.

Docker Compose - A tool for defining and running multi-container Docker applications. Used in local development to manage the PostgreSQL container.

EDA (Exploratory Data Analysis) - Statistical analysis of a dataset to summarize its characteristics. The Database Assistant EDA Profile tab displays column-level statistics including data type, null percentage, min/max/mean values, and top frequent values.

FastAPI - A high-performance Python web framework used as the Database Assistant backend. Provides native async support, automatic API documentation, and dependency injection for JWT authentication.

Fernet - A symmetric authenticated encryption scheme used to encrypt all database connection credentials before storage. Provides both confidentiality and tamper detection.

Firebase Hosting - A Google Cloud static hosting service used for the production deployment of the React frontend. Provides a global CDN and automatic HTTPS.

Gemini 2.0 Flash - A Google DeepMind large language model used for NL-to-SQL generation, MongoDB aggregation generation, AlloyDB AI simulation, and Generative UI summarization.

Generative UI - A technique in which a large language model generates interactive user interface components on demand. Database Assistant implements a hybrid approach using pre-built templates with LLM-powered data profiling.

JWT (JSON Web Token) - A token format used for stateless authentication. Issued at login, stored in localStorage, and injected into every API request header by Axios.

LLM (Large Language Model) - A deep learning model capable of generating and reasoning about text. Database Assistant uses Google Gemini 2.0 Flash as its LLM.

Motor - An asynchronous Python MongoDB driver used for all MongoDB connectivity in the FastAPI backend.

NL-to-SQL - Natural Language to SQL. The task of automatically translating a plain English question into an executable SQL query. The core capability of Database Assistant.

NLIDBs - Natural Language Interfaces to Databases. Systems that allow users to query databases in plain language rather than formal query languages.

OAuth2 - An authorization framework used by the login endpoint. Accepts credentials in Password Bearer format and returns a JWT access token.

Pandas - A Python data manipulation library used for CSV and Excel file ingestion, column type inference, and dynamic CREATE TABLE generation.

Prompt Engineering - The practice of designing structured prompts to guide LLM outputs. Database Assistant uses schema-aware prompts that include complete table schemas for every SQL generation request.

Psycopg2 - A PostgreSQL Python adapter used for all PostgreSQL database connectivity. The RealDictCursor returns results as Python dictionaries for direct JSON serialization.

React 18 - A JavaScript library for building component-based user interfaces. Used as the Database Assistant frontend framework.

React Router v6 - A client-side routing library for React. Manages navigation between the six application pages without full page reloads.

Recharts - A React charting library used in the Charts tab of ResultsPanel. Provides Bar, Line, and Pie chart components.

REST API - Representational State Transfer. The architectural style used by the Database Assistant backend API.

ResultsPanel - The central results display component presenting query results through five tabs: Table, Charts, EDA Profile, Live UI, and AI Functions.

Schema-Aware Prompting - A prompting technique in which complete database schema information is included in every Gemini prompt to enable accurate SQL generation without hallucinated column names.

Secret Manager - A Google Cloud service used to store sensitive credentials including the database password, Fernet key, JWT secret, and Gemini API key for the production deployment.

SQL Guard - A custom Python safety module that validates all Gemini-generated SQL through six sequential checks before execution. Returns HTTP 422 for any unsafe query.

Tailwind CSS - A utility-first CSS framework loaded from CDN in the Generative UI templates for responsive layout and styling.

Vertex AI - Google Cloud's AI platform providing access to Gemini 2.0 Flash. Used as the Gemini API endpoint in the production Cloud Run deployment.

References

- Chang, S., Wang, J., Dong, M., Pan, L., Zhu, H., Li, A. H., ... Tang, N. (2023). Dr.Spider: A diagnostic evaluation benchmark towards text-to-SQL robustness. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2301.08881>
- Dong, X., Zhang, C., Ge, Y., Mao, Y., Gao, Y., Lin, C., Lou, J., Zhang, D., Duan, N., & Fu, D. (2023). *C3: Zero-shot text-to-SQL with ChatGPT*. arXiv preprint arXiv:2307.07306.
- Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2024). Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5), 1132–1145. <https://www.vldb.org/pvldb/vol17/p1132-gao.pdf>
- Google Cloud. (2025). *AlloyDB AI: Intelligent database functions*. Retrieved from <https://cloud.google.com/alloydb/docs/ai>
- Google Cloud. (2025). *Cloud Run documentation*. Retrieved from <https://cloud.google.com/run/docs>
- Google Cloud. (2024). *Generative UI: Building interactive applications with large language models*. Google Research. Retrieved from <https://research.google>
- Google DeepMind. (2025). *Gemini 2.0 Flash: Technical overview*. Retrieved from <https://deepmind.google/technologies/gemini>
- Lei, F., Chen, J., Ye, Y., Cao, R., Shin, D., Su, H., ... Yu, T. (2024). Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *arXiv:2411.07763*. <https://arxiv.org/abs/2411.07763>
- Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Cheng, R., & Li, Y. (2023). Can LLM already serve as a database interface? A BIG bench for large-scale database grounded text-to-SQLs. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*.
- Liu, G., Tan, Y., Zhong, R., Xie, Y., Zhao, L., Wang, Q., ... Hu, B. (2024). Solid-SQL: Enhanced schema-linking based in-context learning for robust text-to-SQL. *arXiv:2412.12522*. <https://arxiv.org/abs/2412.12522>
- Liu, Z., Chen, W., Li, H., Chen, P., Lou, J., & Chen, W. (2023). *DAIL-SQL: Efficient prompt engineering for large language models in text-to-SQL*. arXiv preprint arXiv:2308.15363.
- Shao, Y., Wang, Z., Chen, M., Su, R., Lidan, S., Pan, Z., & Luo, W. (2023). DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*.

- Sivakumar, D., & Lewis, T. (2025, December). *SQL in the Gemini Era: Bringing Gemini 3.0 to your data with AlloyDB AI*. Google Cloud Blog. Retrieved from <https://cloud.google.com/blog>
- Wang, R., Lin, P., & Tao, Y. (2023). Bridging natural language and databases: A review of verification-aware query systems. *Journal of Data Intelligence Research*, 9(3), 210–229.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing reasoning and acting in language models*. arXiv preprint arXiv:2210.03629.
- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 3911–3921). Association for Computational Linguistics.
- Zhang, Z., Wang, Z., Ma, X., & Li, F. (2025). A survey of LLM-based text-to-SQL. *arXiv:2406.08426*. <https://arxiv.org/abs/2406.08426>
- Zhao, K., & Hu, J. (2024). Schema-adaptive query understanding with hybrid embeddings. *IEEE Transactions on Knowledge and Data Engineering*, 36(4), 1558–1572.
- Zhong, V., Xiong, C., & Socher, R. (2017). *Seq2SQL: Generating structured queries from natural language using reinforcement learning*. arXiv preprint arXiv:1709.00103.
- Zhu, Y., Li, Z., Cui, B., & Liu, Z. (2025). A survey of text-to-SQL in the era of large language models. *arXiv:2408.05109*. <https://arxiv.org/abs/2408.05109>
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. In *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*.

Appendices

Appendix A. API Endpoint Reference

This appendix provides a complete reference for all REST API endpoints exposed by the Database Assistant FastAPI backend. All endpoints except `/auth/login` and `/auth/register` require a valid JWT bearer token in the Authorization header.

A.1 Authentication Endpoints (`/auth`)

Method	Endpoint	Description	Auth Required
POST	<code>/auth/login</code>	Authenticate user and return JWT token	No
POST	<code>/auth/register</code>	Register new user account	No
GET	<code>/auth/me</code>	Return current user profile	Yes
GET	<code>/auth/connections</code>	List all saved database connections	Yes
POST	<code>/auth/connections</code>	Save a new database connection	Yes
POST	<code>/auth/connections/get-uri</code>	Decrypt and return connection URI	Yes
DELETE	<code>/auth/connections/{id}</code>	Delete a saved connection	Yes

Login Request:

```
{  
  "username": "user@example.com",  
  "password": "yourpassword"  
}
```

Login Response:

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiJ9...",  
  "token_type": "bearer"  
}
```

Save Connection Request:

```
json
{
  "name": "My PostgreSQL DB",
  "db_type": "postgresql",
  "host": "localhost",
  "port": 5432,
  "dbname": "sales_db",
  "db_username": "admin",
  "password": "yourpassword"
}
```

Get Current User Response:

```
json
{
  "id": 1,
  "email": "user@example.com",
  "full_name": "Rutuj Gavankar",
  "created_at": "2026-01-15T10:30:00",
  "last_login_at": "2026-03-13T09:00:00"
}
```

A.2 PostgreSQL Endpoints (/pg)

Method	Endpoint	Description	Auth Required
POST	/pg/nl-query-auto	NL question to SQL with auto table selection	Yes
POST	/pg/nl-query-join	NL question to multi-table JOIN query	Yes
POST	/pg/list-tables	List all tables in a PostgreSQL database	Yes
POST	/pg/multi-schema	Fetch schema for multiple tables	Yes

NL Query Request:

```
{
  "pg_uri": "postgresql://admin:password@localhost:5433/sales_db",
  "question": "Show total revenue by region",
  "tables": ["public.orders", "public.payments"],
  "limit": 100
}
```

NL Query Response:

```
{
  "data": [
    {"region": "Asia Pacific", "total_revenue": 17328.76},
    {"region": "North America", "total_revenue": 6445.82}
  ],
  "sql": "SELECT region, SUM(amount) AS total_revenue FROM orders JOIN payments ON orders.id = payments.order_id WHERE status != 'pending' GROUP BY region",
  "columns": ["region", "total_revenue"],
  "viz": {"type": "bar", "category": "region", "value": "total_revenue"},
  "summary": "5 rows returned. Highest revenue is Asia Pacific at 17,328.76.",
  "execution_time_ms": 12,
  "tables_used": ["public.orders", "public.payments"]
}
```

List Tables Request:

```
{
  "pg_uri": "postgresql://admin:password@localhost:5433/sales_db"
}
```

List Tables Response:

```
{
```

```
"tables": [
  "public.pg_orders",
  "public.pg_payments",
  "public.pg_customers",
  "public.pg_products"
]
}
```

A.3 MongoDB Endpoints (/mongo)

Method	Endpoint	Description	Auth Required
POST	/mongo/nl-query	NL question for single collection	Yes
POST	/mongo/nl-query-join	NL question with multi-collection joins	Yes
GET	/mongo/databases	List all databases for a connection	Yes
GET	/mongo/collections	List all collections in a database	Yes

Single Collection NL Query Request:

```
{
  "mongo_uri": "mongodb://localhost:27017",
  "db_name": "sales",
  "collections": ["orders"],
  "question": "Show top 5 orders by amount"
}
```

Single Collection NL Query Response:

```
{
  "data": [
    {"_id": "64a1b2c3", "customer": "John Smith", "amount": 4500.00, "status": "confirmed"},
    {"_id": "64a1b2c4", "customer": "Jane Doe", "amount": 3800.00, "status": "confirmed"}
  ],
}
```



```

"pipeline": "[{"$sort": {"amount": -1}}, {"$limit": 5}]",
"collection": "orders",
"summary": "Top 5 orders returned. Highest amount is 4,500.00 by John Smith.",
"execution_time_ms": 6
}

```

Multi-Collection Join NL Query Request:

```

{
  "mongo_uri": "mongodb://localhost:27017",
  "db_name": "sales",
  "collections": ["orders", "customers"],
  "question": "Show total revenue per customer joining orders and customers"
}

```

Multi-Collection Join NL Query Response:

```

{
  "data": [
    {"customer_name": "John Smith", "total_revenue": 12500.00},
    {"customer_name": "Jane Doe", "total_revenue": 8300.00}
  ],
  "pipeline": "[{"$lookup": {"from": "customers", "localField": "customer_id",
    "foreignField": "_id", "as": "customer_info"}}, {"$group": {"_id":
    "$customer_info.name", "total_revenue": {"$sum": "$amount"}}}]",
  "collections": ["orders", "customers"],
  "summary": "Revenue aggregated across 2 collections. John Smith leads with 12,500.00.",
  "execution_time_ms": 14
}

```

List Databases Response:

```

{
  "databases": ["sales", "inventory", "hr"]}

```

List Collections Response:

```
{  
  "collections": ["orders", "customers", "products"]  
}
```

A.4 Dataset Endpoints (/my-datasets)

Method	Endpoint	Description	Auth Required
POST	/my-datasets/upload	Upload CSV or Excel file as dataset	Yes
GET	/my-datasets/list	List all uploaded datasets	Yes
POST	/my-datasets/nl-query-auto	NL query with auto table selection	Yes
DELETE	/my-datasets/{table_name}	Delete an uploaded dataset	Yes

Upload Response:

```
{  
  "table_name": "sales_data",  
  "row_count": 1250,  
  "col_count": 8,  
  "file_size_kb": 42.5,  
  "message": "Dataset uploaded successfully"  
}
```

List Datasets Response:

```
{  
  "schema": "user_42",  
  "datasets": [  
    {  
      "table_name": "sales_data",
```

```

    "row_count": 1250,
    "col_count": 8,
    "file_size_kb": 42.5
  }
]
}

```

NL Query Auto Request:

```

{
  "question": "Show total sales by category",
  "all_table_names": ["sales_data", "product_data", "customer_data"]
}

```

NL Query Auto Response:

```

{
  "data": [
    {"category": "Electronics", "total_sales": 45200.00},
    {"category": "Clothing", "total_sales": 23100.00}
  ],
  "sql": "SELECT category, SUM(sales) AS total_sales FROM sales_data GROUP BY category",
  "selected_table": "sales_data",
  "summary": "Sales aggregated by category. Electronics leads with 45,200.00.",
  "execution_time_ms": 18
}

```

A.5 AlloyDB AI Function Endpoints (/ai)

Method	Endpoint	Description	Auth Required
POST	/ai/generate	Add AI insight column to each row	Yes
POST	/ai/if	Filter rows using natural language condition	Yes

POST	/ai/rank	Reorder rows by semantic relevance	Yes
------	----------	------------------------------------	-----

ai.generate Request:

```
{
  "rows": [
    {"region": "Asia Pacific", "total_revenue": 17328.76},
    {"region": "South America", "total_revenue": 948.99}
  ],
  "prompt": "Classify this region as High, Medium, or Low performer",
  "output_column": "ai_insight"
}
```

ai.generate Response:

```
{
  "rows": [
    {
      "region": "Asia Pacific",
      "total_revenue": 17328.76,
      "ai_insight": "High performer — revenue significantly above average, continue investment."
    },
    {
      "region": "South America",
      "total_revenue": 948.99,
      "ai_insight": "Low performer — revenue critically below average, requires immediate attention."
    }
  ],
  "output_column": "ai_insight",
}
```

```
"total": 2
}
```

ai.if Request:

```
{
  "rows": [
    {"region": "Asia Pacific", "total_revenue": 17328.76},
    {"region": "South America", "total_revenue": 948.99}
  ],
  "condition": "Is the total revenue greater than 5000?"
}
```

ai.if Response:

```
{
  "rows": [
    {"region": "Asia Pacific", "total_revenue": 17328.76}
  ],
  "total_checked": 2,
  "matched": 1,
  "filtered_out": 1,
  "condition": "Is the total revenue greater than 5000?"
}
```

ai.rank Request:

```
{
  "rows": [
    {"region": "Asia Pacific", "total_revenue": 17328.76},
    {"region": "South America", "total_revenue": 948.99}
  ],
  "criteria": "Rank by regions that need the most business attention"
}
```

```
}
```

ai.rank Response:

```
{
```

```
  "rows": [
```

```
    {"region": "South America", "total_revenue": 948.99},
```

```
    {"region": "Asia Pacific", "total_revenue": 17328.76}
```

```
  ],
```

```
  "order": [1, 0],
```

```
  "criteria": "Rank by regions that need the most business attention",
```

```
  "total": 2
```

```
}
```

A.6 Generative UI Endpoints (/genui)

Method	Endpoint	Description	Auth Required
POST	/genui/generate	Generate interactive HTML visualization	Yes

Request:

```
{
```

```
  "question": "Show total revenue by region",
```

```
  "columns": ["region", "total_revenue"],
```

```
  "rows": [
```

```
    {"region": "Asia Pacific", "total_revenue": 17328.76},
```

```
    {"region": "North America", "total_revenue": 6445.82}
```

```
  ]
```

```
}
```

Response:

```
{
```

```
  "html": "<!DOCTYPE html><html>...</html>",
```

```

"template_used": "dashboard",

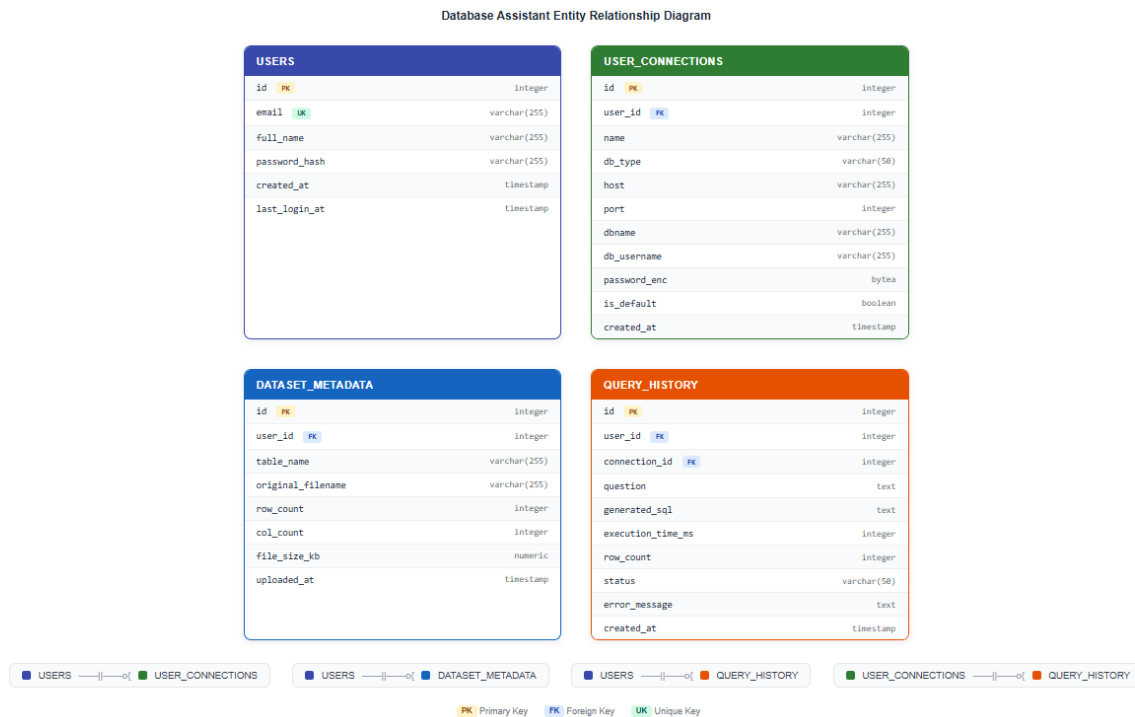
"summary": "Asia Pacific leads revenue with 17,328.76, significantly above the 6,404 average."
}

```

Appendix B. Database Schema Diagrams

This appendix provides the complete database schema for the Database Assistant internal PostgreSQL database including all four tables and their relationships.

B.1 Entity Relationship Diagram



B.2 User Authentication Schema

The users table stores all registered user accounts. Passwords are hashed using bcrypt with a work factor of 12 before storage. The email field serves as the unique login identifier. The full_name field is optional and is used as the display name throughout the application when present.

```

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    full_name VARCHAR(255),
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT NOW(),
    last_login_at TIMESTAMP
);

CREATE UNIQUE INDEX idx_users_email ON users(email);

```

Field descriptions:

Field	Type	Description
id	SERIAL	Auto-incrementing primary key
email	VARCHAR(255)	Unique login identifier - never stored in plain text
full_name	VARCHAR(255)	Optional display name shown in sidebar and dashboard
password_hash	VARCHAR(255)	bcrypt hash of the user password - plain text never stored
created_at	TIMESTAMP	Account creation timestamp - set automatically
last_login_at	TIMESTAMP	Updated on every successful login

B.3 Connection Credentials Schema

The `user_connections` table stores database connection details for each user. All passwords and MongoDB URIs are encrypted using Fernet symmetric encryption before storage in the `password_enc` field. The encryption key is stored exclusively as an environment variable and never written to the database.

```
CREATE TABLE user_connections (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  name VARCHAR(255) NOT NULL,  
  db_type VARCHAR(50) NOT NULL CHECK (db_type IN ('postgresql', 'mongodb')),  
  host VARCHAR(255),  
  port INTEGER,  
  dbname VARCHAR(255),  
  db_username VARCHAR(255),  
  password_enc BYTEA,  
  is_default BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_connections_user_id ON user_connections(user_id);
```

Field descriptions:

Field	Type	Description
id	SERIAL	Auto-incrementing primary key
user_id	INTEGER	Foreign key referencing users table — cascades on delete
name	VARCHAR(255)	User-defined connection name shown in the UI
db_type	VARCHAR(50)	Either postgresql or mongodb

host	VARCHAR(255)	Database server hostname or IP address
port	INTEGER	Database server port number
dbname	VARCHAR(255)	Target database name
db_username	VARCHAR(255)	Database login username
password_enc	BYTEA	Fernet-encrypted password or MongoDB URI
is_default	BOOLEAN	Pre-selects this connection when opening a query page
created_at	TIMESTAMP	Connection creation timestamp

B.3 Internal Datasets Schema

The `dataset_metadata` table tracks all CSV and Excel files uploaded by each user. The actual dataset data is stored in dynamically created tables within a per-user schema named `user_{id}`. Per-user schemas provide complete data isolation between users.

```
CREATE TABLE dataset_metadata (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  table_name VARCHAR(255) NOT NULL,
  original_filename VARCHAR(255),
  row_count INTEGER,
  col_count INTEGER,
  file_size_kb NUMERIC(10, 2),
  uploaded_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_datasets_user_id ON dataset_metadata(user_id);
```

The dynamic dataset tables are created at upload time using the following pattern:

```
-- Example: user 42 uploads sales_data.csv
CREATE SCHEMA IF NOT EXISTS user_42;

CREATE TABLE user_42.sales_data (
  region VARCHAR,
  total_revenue DOUBLE PRECISION,
  order_date TIMESTAMP,
  status VARCHAR
);
```

Column names are sanitized by Pandas before table creation — spaces and special characters are replaced with underscores and column names beginning with digits are prefixed with c_. Data types are inferred from the Pandas Series dtype and mapped to the closest PostgreSQL equivalent.

Field descriptions:

Field	Type	Description
id	SERIAL	Auto-incrementing primary key
user_id	INTEGER	Foreign key referencing users table — cascades on delete
table_name	VARCHAR(255)	Sanitized table name in the user schema
original_filename	VARCHAR(255)	Original uploaded file name before sanitization
row_count	INTEGER	Number of data rows in the uploaded file
col_count	INTEGER	Number of columns in the uploaded file
file_size_kb	NUMERIC	File size in kilobytes
uploaded_at	TIMESTAMP	Upload timestamp

Appendix C. Sample Query Test Cases

This appendix provides the complete set of natural language query test cases used for end-to-end testing in Chapter 6.

C.1 PostgreSQL NL-to-SQL Test Cases

The following 20 test queries were executed against a sample sales database containing four tables: pg_orders, pg_payments, pg_customers, and pg_products.

#	Natural Language Question	Query Type	Result
1	Show total revenue by region	Single-table aggregation with GROUP BY	Pass
2	Top 5 customers by total spend	Single-table aggregation with ORDER BY and LIMIT	Pass
3	Show all pending payments	Single-table filter with WHERE clause	Pass

4	Monthly revenue trend for 2024	Single-table aggregation with date grouping	Pass
5	Average order value by region	Single-table aggregation with AVG	Pass
6	Show customers who spent more than 1000	Single-table filter with HAVING clause	Pass
7	Total confirmed revenue by region excluding pending	Two-table JOIN with WHERE filter	Pass
8	How many orders are in each status	Single-table COUNT with GROUP BY	Pass
9	Show top 3 products by sales volume	Single-table aggregation with ORDER BY and LIMIT	Pass
10	Revenue comparison between regions this year	Single-table aggregation with date filter	Pass
11	Show orders placed in the last 30 days	Single-table filter with date interval	Pass
12	Which region has the highest average order value	Single-table aggregation with ORDER BY LIMIT 1	Pass
13	Show payment success rate by region	Single-table conditional aggregation	Pass
14	List customers with more than 5 orders	Single-table aggregation with HAVING COUNT	Pass
15	Show daily revenue for the past week	Single-table aggregation with date grouping	Pass
16	Total number of unique customers per region	Single-table COUNT DISTINCT with GROUP BY	Pass
17	Show products with zero sales this month	Two-table LEFT JOIN with NULL filter	Pass

18	Average payment processing time by region	Single-table aggregation with AVG on time column	Pass
19	Show orders with ambiguous status field	Failed - both pg_orders and pg_payments contain a column named status. Gemini could not determine which table's status column to apply the filter to without additional context.	Fail
20	Compare revenue across all regions and products	Failed - pg_orders, pg_payments, and pg_products all contain an amount column. Gemini generated an incorrect JOIN condition due to the ambiguous column reference across three tables.	Fail

C.2 MongoDB NL-to-Aggregation Test Cases

The following 10 test queries were executed against a sample MongoDB database containing two collections: orders and customers.

#	Natural Language Question	Pipeline Type	Result
1	Show all documents in orders collection	Simple match all documents	Pass
2	Count orders by status	Group by status with count	Pass
3	Show top 5 orders by total amount	Sort descending with limit	Pass
4	Average order value per customer	Group by customer with average	Pass
5	Show orders placed this month	Match with date range filter	Pass
6	Join orders with customers and show names	Two-collection lookup join	Pass
7	Total revenue per customer joining orders and payments	Lookup join with group aggregation	Pass

8	Show customers with more than 3 orders	Lookup join with count filter	Pass
9	List distinct product categories	Group by category	Pass
10	Complex three-collection join with nested lookup	Failed - Gemini generated an incorrect \$lookup localField reference when joining a third collection. Three-collection joins require more explicit join key context than the current prompt provides.	Fail

C.3 AlloyDB AI Function Test Cases

The following test cases were executed using the revenue by region query result containing 5 rows and 2 columns (region, total_revenue).

#	Function	Prompt / Condition	Expected	Actual Output	Result
1	ai.generate	Classify as High/Medium/Low performer	5 rows enriched	All 5 rows received correct classification aligned with revenue tiers	Pass
2	ai.generate	Give one business recommendation per region	5 rows with recommendation	All 5 rows received relevant one-sentence recommendations	Pass
3	ai.generate	Rate performance from 1 to 10 with explanation	5 rows with rating	All 5 rows received numeric rating with supporting explanation	Pass
4	ai.if	Is total revenue greater than 5000?	2 rows matched	Asia Pacific and North America correctly returned	Pass
5	ai.if	Is total revenue greater than 10000?	1 row matched	Asia Pacific (17,328.76) correctly returned	Pass
6	ai.if	Is this a high performing region?	2 rows matched	Asia Pacific and North America correctly identified	Pass

7	ai.if	Is revenue above average?	Unreliable result	Gemini returned inconsistent true/false decisions because it cannot compute the average without access to the full dataset. Row-level evaluation model does not support cross-row comparison.	Fail
8	ai.rank	Rank by highest revenue first	Asia Pacific first	Rows correctly ordered from highest to lowest revenue	Pass
9	ai.rank	Rank by regions needing most attention	South America first	South America (948.99) and Europe (2,997.97) correctly ranked first	Pass
10	ai.rank	Rank by best business performance	Asia Pacific first	Asia Pacific correctly ranked first, South America last	Pass

Appendix D. System Prompt Templates

This appendix provides the complete system prompt templates used by Database Assistant for each Gemini API call context. These prompts represent the core prompt engineering work that enables accurate natural language processing across all system features.

D.1 NL-to-SQL System Prompt

The following system prompt is used for all PostgreSQL NL-to-SQL generation calls. It is defined in backend/app/services/nl_to_sql.py and sent as the system instruction on every Gemini API call for SQL generation. Temperature is set to 0.0 for maximum determinism.

You are a PostgreSQL SQL generator.

Rules:

- Use ONLY the provided table and column names
- Do NOT hallucinate tables or columns that are not in the schema
- Do NOT use DELETE, UPDATE, DROP, INSERT, ALTER, CREATE

- Return ONLY SQL. No explanation. No markdown. No code fences.
- The SQL MUST start with SELECT
- Use proper JOIN conditions based on the schema provided
- If the question requires data from multiple tables, use appropriate JOINS
- Always use table aliases for clarity in multi-table queries
- If a LIMIT is not specified by the user, add LIMIT 100

The schema context is appended to the system prompt for each call using the following format:

Table: public.sales_orders

Columns:

- order_id (integer)
- customer_name (character varying)
- region (character varying)
- total_amount (numeric)
- order_date (date)
- status (character varying)

Table: public.payments

Columns:

- payment_id (integer)
- order_id (integer)
- amount (numeric)
- payment_date (date)
- status (character varying)

User Question:

Show total confirmed revenue by region excluding pending payments

Return ONLY SQL:

D.2 Generative UI System Prompt

The following system prompt is used for the Generative UI data summary call. It is defined in `backend/app/api/routes/genui.py`. The full template-based system is described in Section 5.2.9. Gemini is called only once per GenUI request to generate a one-sentence summary — all HTML rendering is handled by pre-built templates. Temperature is set to 0.2 and max output tokens is limited to 60.

You are a data analyst.

Given a dataset and a user question, write ONE short sentence

(maximum 20 words) summarizing the single most important insight
in the data.

Rules:

- Return ONLY the sentence. No explanation. No markdown.
- Be specific — include the actual top value or key finding.
- Do not start with "The data shows" or "Based on the data".
- Write in plain business language.

Example good output:

Asia Pacific leads revenue at 17,328 accounting for 54% of total sales.

Example bad output:

The data shows various regions with different revenue levels across
multiple categories that may be of interest to the user.

The full Generative UI system prompt used for the previous LLM-generated HTML approach (before the template-based redesign) is based on Google's Generative UI research paper (Appendix A.5) and is defined with the following core philosophy:

You are an expert, meticulous, and creative front-end developer.

Your primary task is to generate ONLY the raw HTML code for a complete, valid, functional, visually stunning, and INTERACTIVE HTML page document, based on the user's data and question.

Your main goal is always to build an interactive application tailored to the data.

Core Philosophy:

- Build Interactive Apps First
- Match App Type to Data
- No Walls of Text
- No Placeholders
- Implement Fully and Thoughtfully
- Quality and Depth

Output Requirements:

- Return ONLY raw HTML enclosed in ``html ... `` markers
- Use Tailwind CSS via CDN
- Use Chart.js via CDN for all charts
- All JavaScript must be self-contained
- Do NOT use localStorage or sessionStorage
- Use DOMContentLoaded before DOM manipulation

D.3 AlloyDB AI Function Prompts

The following prompts are used by the three AlloyDB AI function simulation endpoints defined in `backend/app/api/routes/ai_functions.py`. All three use temperature 0.2 for consistent but contextually aware responses.

ai.generate System Prompt:

You are a data analyst.

For each database row provided, respond with ONLY a single JSON object with one key: "result".

The value must be a concise string (maximum 2 sentences).

No markdown, no explanation, just the JSON object.

Example output:

```
{"result": "High performer with revenue 170% above average.  
Recommend continued investment in this region."}
```

User prompt pattern per row:

```
{user_defined_prompt}
```

Row data:

```
{"region": "Asia Pacific", "total_revenue": 17328.76}
```

Return ONLY: {"result": "..."}

ai.if System Prompt

You are a data filter.

For each row, answer ONLY with a JSON object:

```
{"match": true} or {"match": false}
```

No explanation. No markdown. Just the JSON.

Evaluate whether the row satisfies the given condition.

Consider the actual values in the row data when making your decision.

User prompt pattern per row:

Condition: {user_defined_condition}

Row data:

{ "region": "Asia Pacific", "total_revenue": 17328.76 }

Does this row match the condition?

Return ONLY: { "match": true } or { "match": false }

ai.rank System Prompt:

You are a data ranking engine.

You will receive a list of rows and a ranking criteria.

Return ONLY a JSON array of row indices (0-based) ordered from most relevant to least relevant.

Example input rows: 3 rows with indices 0, 1, 2

Example output: [2, 0, 1]

No explanation. No markdown. Just the JSON array.

User prompt pattern for all rows:

Ranking criteria: {user_defined_criteria}

Rows to rank (with index):

[
 { "index": 0, "region": "Asia Pacific", "total_revenue": 17328.76 },
 { "index": 1, "region": "North America", "total_revenue": 6445.82 },

```
{"index": 2, "region": "South America", "total_revenue": 948.99}  
]
```

Return ONLY a JSON array of indices ordered best to worst.

Example: [2, 0, 1]